Deakin University

Faculty of Science, Engineering and Built Environment

School of IT

Performance of State-of-the-Art Machine Learning Methods on Out-of-Distribution Data in Tabular Regression: A Survey

Supervisor: Prof. Gleb Beliakov and A/Prof. Simon James

Author's Name: Bao Minh Tran

Email: s224236373@deakin.edu.au

November 12^{th}, 2025

Contents

Abstract

$$P^{train} \neq P^{test}$$

1 Introduction

Artificial Intelligence (AI) [1] is a broad computer science field that studies about softwares that can mimic the human's capability of learning and problem solving. Machine Learning (ML) is a sub-field of AI, which [2] applies statistical methods to explore the underlying correlation among the features and the target. Nonetheless, ML faces difficulties when dealing with high data volume and dimensions. A sub-field of ML has been studied to overcome these challenges, i.e. Deep Learning [3], which has proven its superiority over conventional ML approaches, and yielded prospective results in Natural Language Processing (NLP), Computer Vision (CV), Large Language Models (LLMs), and also being applied to other industries ([4], [5]), e.g. healthcare, finance, energy, and agriculture.

Since 1940s [6], several ML/DL models have been proposed, and various benchmarks have been introduced to compare the performance among them. A critical fault is that these benchmarks often ([7], [8], [9], [10]) overlook the distributional shifts between the training data and deployment data, alternatively, they assume that the traing and testing sets are independent and identically distributed (i.i.d). These inherent distributional shifts are due to various factors, e.g. [8] sample biases, environment changing, data generation errors, and [11] learning spurious correlations. Many studies have been contributed to showing [11] distributional shifts in real data are roadblock to the ML/DL applications to other fields, and many [12] benchmarks and [13] AI practioners failed to analyse the models' performance on Out-of-Distribution (OOD) data. In Computer Vision, [14] showed that recent image classifiers fail to OOD generalization and [15] introduced a benchmark for performance evaluation among object detectors, namely ObjectNet, which showed performance drop of top models, from 2012 to 2018, in comparision to other standard benchmarks, e.g. ImageNet. Regarding Natural Language Processing, [16] proposed a benchmark, called CheckList, that leads to failure in semantic analysis due to tiny variations in testing texts compared to those in the training set.

Although ([17], [18]) tabular data is the primary data structure to retain data for training models in numerous areas, e.g. finance ([19], [20]), healthcare ([21], [22]) and manufacturing sectors [23], the number of standardized tabular regression benchmarks are very limited ([24], [17]). Therefore, this report serves as a benchmark consisting of various train/test splitting strategies on real datasets to evaluate model's capability towards OOD generalization on tabular regression tasks. The key contributions of our work are:

- Introduction to a comprehensive benchamrk consisting of 26 real-world, low dimensional (up to 20 features) tabular datasets specifically for regression tasks. These datasets are subsequently partitioned into several train/test pairs aiming for evaluating the models' OOD generalization. To our knowledge, this marks the first benchamrk that encompasses certain approaches to split datasets for OOD extrapolation assessment.
- Conduct an experimental study to quantify and rank the robustness of a broad spectrum of standard ML/DL models on unseen distributions.

- Give insightful analysis to explore what characteristics of the proposed ML/DL methods impact their performance when generalising to unseen data regions.

Before proceeding, it is recommended to see the Appendices for the use of Abbreviations and Mathematics Notations within this report.

2 Related works

The aim of this report is to investigate the performance of widely used ML models on Out-of-Distribution (OOD) tabular data. Thus, plenty of papers were introduced to formally define what it means for models to extrapolate on unseen data. Many papers ([25], [26], [27], [28]) have supported that extrapolation to OOD data refers to models predicting data points lying beyond the **convex hull of training samples**. Nevertheless, a more prevalent definition of OOD data highlights the **distributional shifts between the training and testing sets**, and is commonly used to compare models' generalization on OOD data ([9], [7], [12], [24], [11], [17]). To our knowledge, there are none of benchmarks for tabular regression tasks on OOD data that is built upon the first definition. Meanwhile, there are contributions, though still very limited, to benchmarking models on OOD data, based on distributional shifts, in tabular regression tasks, e.g. **Wild-Tab** [17], **Shifts Dataset** [7], and an unnamed benchmark provided by [24].

The **Wild-Tab** benchmark [17] evaluates models' robustness under covariate, subpopulation, and hybrid shifts of large-sized real-world datasets. This report also conducted an empirical study by implementing baseline DL models, e.g. MLP-PLR, FT-Transformer, and 10 other OOD generalization techniques, e.g. IRM, ERM, and GroupDRO, that are specifically designed for tabular regression tasks, and gave certain insightful analysis, i.e. ERM outperforms other models, and the impact of model architecture and hyperparameter tuning methods on models' performance. In contrast, [24] offers a benchmark to compare between the tree-based models and DL models on OOD data in tabular regression tasks. More specifically, [24] leverages 45 tabular datasets, mostly from OpenML [29], and cleans and prepares properly. Later, they benchmarked 3 tree-based models, i.e. Random Forest, XGBoost, and Gradient Boosting Trees (or HistGradient Boosting Trees in case of categorical inputs) and 4 distinct DL models, i.e. classical MLP, ResNet, FT-Transformer, and SAINT. Also, they clearly interpreted why tree-based models outperform DL models in tabular regression on OOD data regardless of models' complexity. Despite limited available benchamrks for tabular regression on OOD data, these prior works inspired us to conduct a more comprehensive empirical study to compare the performance of various ML/DL models on OOD data that are created by applying diverse splitting strategies to real-world datasets.

3 Problem settings

3.1 Regression formulation

Throughout this report, we are dealing with tabular regression tasks, let us first formally recall the formulation of this problem. To begin with, we are given an input space or domain, denoted as $\mathcal{X}$, the objective is to [30] induce an approximation function, denoted as $\hat{f} : \mathcal{X} \mapsto \mathcal{Y} \subseteq \mathbb{R}$, such that, $\hat{f} \approx f$, where f is the true function representing the

relationship $\mathcal{X} \mapsto \mathcal{Y}$. To quantify how well the predictor/model $\hat{f}$ performs, several metrics have been proposed, typically Mean Squared Error (MSE), Mean Absolute Error (MAE), and Root Mean Squared Error (RMSE). In practice, these evaluation metrics are leveraged as objective functions to train the models. Formally, the models repeatedly update their parameters to [11] diminish the population risk.

Definition 1. (True) Population Risk [8]

$$\mathcal{R}_{\text{true}}\left(\hat{f}\right) \equiv \mathcal{R}_{\mathbf{x}\in\mathcal{X}}\left(\hat{f}\right) \stackrel{\text{def}}{=} \mathbb{E}_{\mathbf{x}\in\mathcal{X}}\Big[\ell\Big(\hat{f}(\mathbf{x}), f(\mathbf{x})\Big)\Big]$$

Nonetheless, in real-world scenarios, it is almost impossible to capture all behaviours of $\mathcal{X}$ to train the models, e.g. self-driving car systems [11], dog/cat classification [8], and disease diagnosis for patients in all hospitals [10]. Alternatively, the model $\hat{f}$ is trained on a **sub-domain** $\mathcal{X}^{\text{tr}} \subset \mathcal{X}$, and hence, the model $\hat{f}$ is trying to optimize the sample risk or empirical risk, not the population risk as expected. In other words, while training on $\mathcal{X}^{\text{tr}} \subset \mathcal{X}$, the predictor $\hat{f}$ is expected to perform on $\mathcal{X}^{\text{te}} \subset \mathcal{X}$ as well as it does on $\mathcal{X}^{\text{tr}}$.

Definition 2. Empirical Risk [8]

$$\mathcal{R}_{\text{emp}}\left(\hat{f}\right) \equiv \mathcal{R}_{\mathbf{x}\in\mathcal{X}^{\text{tr}}}\left(\hat{f}\right) \stackrel{\text{def}}{=} \mathbb{E}_{\mathbf{x}\in\mathcal{X}^{\text{tr}}}\Big[\ell\Big(\hat{f}(\mathbf{x}), f(\mathbf{x})\Big)\Big]$$

Despite the impossibility to gather the whole domain $\mathcal{X}$, practioners and researchers have endeavoured to collect samples to form the training set $\mathcal{X}^{\text{tr}}$ that behaves somewhat similar to the whole population or domain $\mathcal{X}$. Thereby, the model $\hat{f}$ is expected to [8] make the **Empirical Risk** $\mathcal{R}_{\text{emp}}$ approximates the **True Risk** $\mathcal{R}_{\text{true}}$. Formally, this is

$$\arg\min_{\hat{f}} \mathcal{R}_{\text{emp}}\left(\hat{f}\right) \approx \arg\min_{\hat{f}} \mathcal{R}_{\text{true}}\left(\hat{f}\right)$$

To measure the similarity between the sample training set $\mathcal{X}^{\text{tr}}$ and the true population $\mathcal{X}$, we formally present two commonly used concepts, i.e. **distributional shifts** and **convex hull of data** as below.

3.2 Distributional shifts

Throughout this section, we aim to estimate the similarity between $\mathcal{X}^{\text{tr}}$ and $\mathcal{X}^{\text{te}}$ based on their statistical properties, more precisely, their **joint distributions**, i.e. $\mathbf{P}^{\text{tr}}(\mathbf{X}, \mathbf{y})$ and $\mathbf{P}^{\text{te}}(\mathbf{X}, \mathbf{y})$. More detailed, the **marginal distribution** of $\mathcal{X}^{\text{tr}}$ and $\mathcal{X}^{\text{te}}$ are written as $\mathbf{P}^{\text{tr}}(\mathbf{X})$ and $\mathbf{P}^{\text{te}}(\mathbf{X})$, respectively. On the other hand, the **conditional distribution** interprets distribution of $\mathcal{Y}$ with regards to $\mathcal{X} \mapsto \mathcal{Y}$, written as $\mathbf{P}(\mathbf{y} \mid \mathbf{X})$.

A pervasive hypothesis ([7], [8]) while training and validating models is that the train and test sets are independent and identically distributed (i.i.d). In other words, the test data $\mathcal{X}^{\text{te}}$ are considered statistically unchanged in comparision with the train data $\mathcal{X}^{\text{tr}}$, and this assumption has vastly contributed to the development of ML/DL models in the past decades [31].

Definition 3. No shifts - In–Distribution (ID) [8]
The test data $\mathcal{X}^{\text{te}}$ is considered to be In–Distribution (ID), or not shifted, when the following holds:

- $\mathbf{P}^{\text{te}}(\mathbf{X}) = \mathbf{P}^{\text{tr}}(\mathbf{X})$
- $\mathbf{P}^{\text{te}}(\mathbf{y} \mid \mathbf{X}) = \mathbf{P}^{\text{tr}}(\mathbf{y} \mid \mathbf{X})$

As a result, the models $\hat{f}$ show excellence performance on validation samples, but are vulnerable to real-world data. Therefore, it is imperative to detect and address these issues, so that the models can perform better on real data. In practice, there are [8] two primary types of distributional shifts, which are shifts in feature space $\mathcal{X}$, known as **covariate shifts** and shifts in label space $\mathcal{Y}$, known as **concept or semantic shift**. Both of them contribute to the fact that $\mathbf{P}^{\text{tr}}(\mathbf{X}, \mathbf{y}) \neq \mathbf{P}^{\text{te}}(\mathbf{X}, \mathbf{y})$, such test data are said to be **Out–of–Distribution (OOD)**, and thus it downgrades the performance of models $\hat{f}$. Recall our initial objective is to induce an approximation function $\hat{f} : \mathcal{X} \mapsto \mathcal{Y}$, the mapping direction is essential to construct the upcoming definitions.

Definition 4. Covariate shifts [32]

Covariate shift is defined when the following holds:

- $\mathbf{P}^{\text{te}}(\mathbf{X}) \neq \mathbf{P}^{\text{tr}}(\mathbf{X})$
- $\mathbf{P}^{\text{te}}(\mathbf{y} \mid \mathbf{X}) = \mathbf{P}^{\text{tr}}(\mathbf{y} \mid \mathbf{X})$

Definition 5. Concept/Semantic shifts [32]

Concept shift is defined when the following holds:

- $\mathbf{P}^{\text{te}}(\mathbf{X}) = \mathbf{P}^{\text{tr}}(\mathbf{X})$
- $\mathbf{P}^{\text{te}}(\mathbf{y} \mid \mathbf{X}) \neq \mathbf{P}^{\text{tr}}(\mathbf{y} \mid \mathbf{X})$

Besides these dataset shifts, [32] theoretically there is another situation that possibly occur in real-world scenario, i.e.

- $\mathbf{P}^{\text{te}}(\mathbf{X}) \neq \mathbf{P}^{\text{tr}}(\mathbf{X})$
- $\mathbf{P}^{\text{te}}(\mathbf{y} \mid \mathbf{X}) \neq \mathbf{P}^{\text{tr}}(\mathbf{y} \mid \mathbf{X})$

3.3 Convex hull of data

Another definition, though rarely used, originated from a geometric view that leverages the definition of **Convex Hull**.

Definition 6. Convex Hull [33] [34]

Given a set of m points lying in n–dimension space, $\mathcal{S} \triangleq \{\mathbf{x}_1, \mathbf{x}_2, \ldots, \mathbf{x}_m\} \subset \mathbb{R}^n$. The Convex Hull induced by $\mathcal{S}$ is the smallest convex set that completely contains $\forall \mathbf{x} \in \mathcal{S}$. Formally:

$$\text{Conv}\{\mathcal{S}\} \stackrel{\text{def}}{=} \left\{ \mathbf{v} = \sum_{i=1}^{m} \lambda_i \mathbf{x}_i \mid \forall \lambda_i \geq 0 \wedge \sum_{i=1}^{m} \lambda_i = 1 \right\}$$

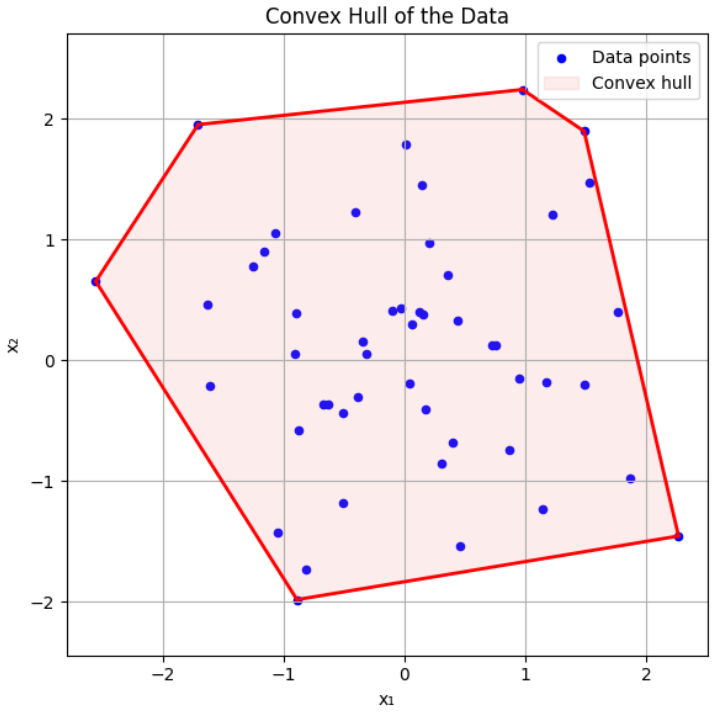

Figure 1: Convex Hull of randomly generated points in 2D visualization

Following the above definition, the OOD generalization can be interpreted as extrapolation.

Definition 7. Interpolation vs. Extrapolation with Convex Hull [25] [28]

Given a model $\hat{f}$ trained on a set of points $\mathcal{X}^{\text{tr}}$, the convex hull of $\mathcal{X}^{\text{tr}}$, denoted as $\text{Conv}(\mathcal{X}^{\text{tr}})$. For any point $\mathbf{x} \sim \mathcal{X}^{\text{te}}$ and $\mathcal{X}^{\text{te}} \cap \mathcal{X}^{\text{tr}} = \emptyset$:

- Interpolation occurs when $\mathbf{x} \in \text{Conv}(\mathcal{X}^{\text{tr}})$.
- Extrapolation occurs when $\mathbf{x} \notin \text{Conv}(\mathcal{X}^{\text{tr}})$.

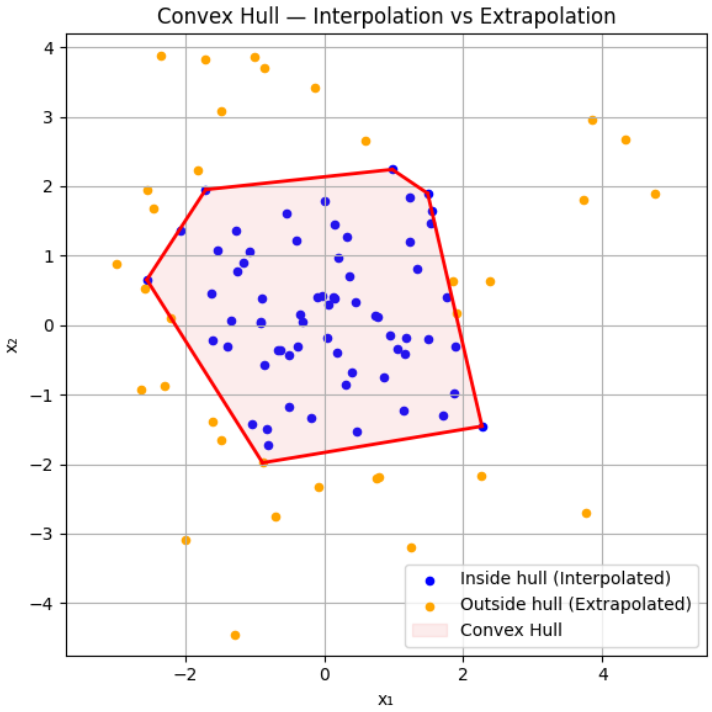

Figure 2: Interpolation vs. Extrapolation with Convex Hull in 2D visualization

In 2D space, the boundary between interpolation and extrapolation can be clearly distinguished. Nonetheless, in higher dimensional space, it is highly vulnerable to the effect of the Curse of Dimensionality [35]. More specifically, [36] the distances among the data points become more scattered, and the commonly used distance metrics, e.g. Euclidean-based, may become invalid, owing to the phenomenon of distance concentration. A (mis)conception emerged that higher precision on training set implies better models' generalization/extrapolation on unseen data regions, which was proven to be completely false, ([25], [36]), this issue is known as overfitting, which may be lessened by the expansion of number of training samples ([25], [36]). However, the datasets used within this report is low-dimensional data, at most 20 features, and the training size ranging from tens to hundreds of thoundsands samples, which may mitigate the impact of the Curse of Dimensionality.

4 Benchmark

4.1 Dataset

This section offers an overview of the dataset used and the sources of them. In addition, some of the datasets below are different from the original datasets due to sample and features removals to suit the need of this report. Moreover, the datasets were modified so that the target column is the last column, from left to right, in tabular format.

Table 1: Dataset overview

#	Original name	Dataset name	#samples	#features
1	218_house_8L	218_house_8L	22,784	8
2	House-16H	house	22,784	16
3	California Housing	california	20,640	8
4	Elevators	elevators	16,599	18
5	Physicochemical Properties of Protein Tertiary Structure	CASP	45,730	9
6	Gas Turbine CO and NOx Emission Data Set	gas_turbine_co_and _nox_emission	36,733	10
7	Condition Based Maintenance of Naval Propulsion Plants	CBM_1	11,934	16
8	Condition Based Maintenance of Naval Propulsion Plants	CBM_2	11,934	16
9	Zurich public transport delay data	delays_zurich_ transport	800,000	8
10	diamonds	diamonds	53,940	6
11	nyc-taxi-green-dec-2016	nyc_taxi_green_ dec_2016	581,835	9
12	house_sales	house_sales	21,613	15
13	medical_charges	medical_charges	163,065	3
14	MiamiHousing2016	MiamiHousing2016	13,932	13
15	sulfur	sulfur_1	10,081	5
16	sulfur	sulfur_2	10,081	5
17	Goodreads-books	books	10,348	3
18	FIFA 22 complete player dataset	players_22	17,041	10
19	Taxi Duration for Domain Generalization	taxi	800,000	13
20	UCI Bike Sharing for Domain Generalization	bike	17,379	5
21	Vessel Power Estimation	power_consumption	567,442	9
22	3droad	3droad	434,874	3
23	keggdirected	keggdirected	48,827	20
24	kin40k	kin40k	40,000	8
25	protein	protein	45,730	9
26	tamielectric	tamielectric	45,781	3

4.2 Train/Test splitting strategy

Due to length limits, through out this report, the dataset `CASP` is utilized as an example to visualize how the splitting approaches influence the data distributions. The remaining statistics summary of both raw and splitted datasets are given in the GitHub repo of this report.

In addition, for extensive datasets, e.g. `taxi` and `delays_zurich_transport`, which is more than 1,000,000 samples, they are randomly selected 800,000 samples to guarantee the distributions of the original datasets. Besides, the train/test proportion used within this report is 70% and 30%, respectively. To guarantee reproducability of this work, we chose a list of 10 random seeds, i.e. `SEEDS = [0, 42, 19, 2, 23, 11, 37, 29, 57, 5]`.

4.2.1 Random strategy

Throughout this report, to highlight the distinction among the common splitting strategy and the proposed strategies, and to show the difference between the models' performance on ID and OOD data, we leverage a `sklearn` built–in function, i.e. `train_test_split()`, to randomly sample the train and test sets, so as to construct iid train/test sets. More specifically, the distributions of train and test sets are identical, or $\mathbf{P}^{\text{tr}}(\mathbf{X}, \mathbf{y}) = \mathbf{P}^{\text{te}}(\mathbf{X}, \mathbf{y})$.

Figure 3: Pairplot of Train vs. Test

4.2.2 Distribution–based strategy

In this approach, we look at the distributions of each feature and the conditional distributions between the features and outputs to construct the OOD data. Recall **Definition 4. Covariate shifts** and **Definition 5. Concept shifts**, we would like to simulate the behaviour of these distributional shifts.

Regarding **Covariate shifts**, we proposed a fairly naive approach. More specifically, we rely on the distribution of each feature to construct the train/test sets, thereby, the number of such train/test set pairs depending how many features each dataset has.

Algorithm 1: **MarginalDistributionSplit: Covariate Shift Splitting**

Require: Dataset $\mathcal{D} = (\mathbf{X}, \mathbf{y})$ with features $\{\text{feat}_1, \ldots, \text{feat}_m\}$ and target $\mathbf{y}$; test ratio $\tau \in (0, 1)$

Ensure: Multiple train/test splits saved as `train_idx.parquet` and `test_idx.parquet`

1: $idx \leftarrow 0$

2: **for all** feat $\in \{\text{feat}_1, \ldots, \text{feat}_m\}$ **do**

$mask \leftarrow \text{DistributionBasedSelection}(\mathcal{D}, \text{feat}, \tau)$
if $\frac{\sum mask}{|\mathcal{D}|} < \tau$ **then**
  Print "Skip feature {feat} due to insufficient test samples"
  continue
end if
$X_{train} \leftarrow X[\neg mask]$; $y_{train} \leftarrow y[\neg mask]$
$X_{test} \leftarrow X[mask]$; $y_{test} \leftarrow y[mask]$
$\mathcal{D}_{train} \leftarrow \text{ConcatColumns}(X_{train}, y_{train})$
$\mathcal{D}_{test} \leftarrow \text{ConcatColumns}(X_{test}, y_{test})$
Save $\mathcal{D}_{train}$ as `train_`idx`.parquet`
Save $\mathcal{D}_{test}$ as `test_`idx`.parquet`
$idx \leftarrow idx + 1$
end for

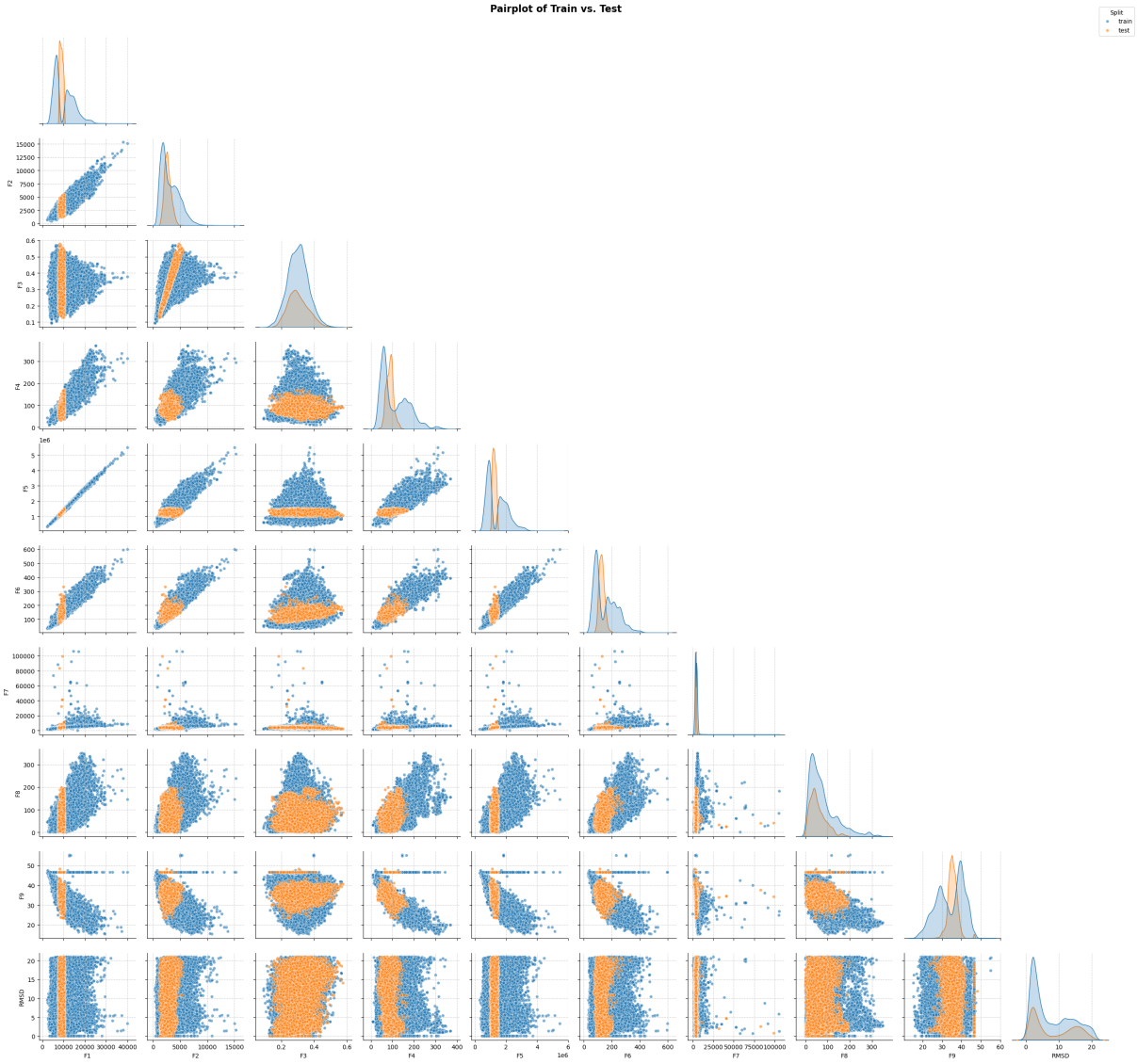

Figure 4: Pairplot of Covariate_Shift train_0/test_0

In contrast, to mimic the behaviour of **Concept shifts**, we utilized the Metafeature–based

approach from [12]. Let us briefly summarize how this technique works.

Now, supposing a given dataset $\mathcal{D} = \{(\mathbf{x}_i, y_i) \mid \mathbf{x}_i \in \mathcal{X}, y_i \in \mathcal{Y}\}$. [37] A **meta–feature** $\mathcal{M} \ni M : \mathcal{D} \mapsto \mathbb{R}^k$, is defined as:

$$M(\mathcal{D}) = \sigma\left(m\left(\mathcal{D}, h_m\right), h_s\right)$$

where $\sigma(\cdot)$ is a characterization measure, $m(\cdot)$ is a summarization function, h_m and h_s are associated hyperparameters for $m(\cdot)$ and $\sigma(\cdot)$, respectively. We do not delve deeper into this definition as it is not out main goal in this report. However, it is imperative to know that the proposed **meta–feature** $M(\cdot)$ distils the dataset $\mathcal{D}$ into the set of k values that can characterize and predict the performance of the algorithms applied to $\mathcal{D}$.

Next, we are now given a labeled dataset $D = \{(\mathbf{x}_i, y_i) \mid \mathbf{x}_i \in \mathbf{X}, y_i \in \mathbf{y}\}$, and D is not representative of the whole population. The objective of [12] is to partition D into $D^{\text{tr}} \cup D^{\text{te}} = D$, where $D^{\text{tr}} \cap D^{\text{te}} = \emptyset$ and their meta–charactistics are different. Let $d \sim D$, [12] defines the following notions:

- Meta–feature extractor $M : d \mapsto \mathbb{R}^k$, that distils d into a vector of k meta–features values, defined as:
$$M(d) = [M_1(d), M_2(d), \cdots, M_k(d)]$$
- For each meta–feature M_j, the directed distance is defined as:
$$dist_j\left(D^{\text{tr}}, D^{\text{te}}\right) = \begin{cases} \dfrac{M_j(D^{\text{tr}})}{M_j(D^{\text{te}})} & \text{, if } M_j(D^{\text{tr}}) \cdot M_j(D^{\text{te}}) \neq 0 \\ 0 & \text{, otherwise} \end{cases}$$
- A scalar imbalance measure is defined as:
$$imb\left(d\right) = \frac{\min_c n_c(d)}{\max_c n_c(d)}$$
where $n_c(\cdot)$ counts the label c in d.
- The imbalance objective is defined as:
$$o_{\text{imb}}\left(D^{\text{tr}}, D^{\text{te}}\right) = \left|imb\left(D^{\text{tr}}\right) - imb\left(D^{\text{te}}\right) + \lambda\left(1 - \min\left[imb\left(D^{\text{tr}}\right), imb\left(D^{\text{te}}\right)\right]\right)\right|$$
where λ is the penalizing hyperparameter.

Eventually, [12] subjects to optimizing the multi–objective vectors:

$$obj\left(D^{\text{tr}}, D^{\text{te}}\right) = \left[dist_1\left(D^{\text{tr}}, D^{\text{te}}\right), dist_1\left(D^{\text{tr}}, D^{\text{te}}\right), \cdots, dist_k\left(D^{\text{tr}}, D^{\text{te}}\right), o_{\text{imb}}\left(D^{\text{tr}}, D^{\text{te}}\right)\right]$$

that minimizes the directed distance for each meta–feature while maximizing the balance between classes, by the utilization of the Evolutionary Algorithm.

In this report, we chose **mutual information** as our information-theoretic meta-feature. In brief, [12] **mutual information** measures the statistical correlation between covariates and target, and is computed as [38]:

$$\mathbf{I} = \left[I_1\left(\mathbf{X}_1; \mathbf{y}\right), I_2\left(\mathbf{X}_2; \mathbf{y}\right), \cdots, I_m\left(\mathbf{X}_m; \mathbf{y}\right)\right]$$

where:

$$I_i\left(\mathbf{X}_i;\mathbf{y}\right) = H\left(\mathbf{X}_i\right) + H\left(\mathbf{y}\right) - H\left(\mathbf{X}_i,\mathbf{y}\right)$$

the Shanon's Entropy $H\left(\mathbf{X}_i\right), H\left(\mathbf{y}\right)$ are defined as:

$$H\left(\mathbf{X}_i\right) = -\sum_{\mathbf{val}\in\phi_{\mathbf{X}_i}} \left(P(\mathbf{X}_i = \mathbf{val}) * \log_2\left(P(\mathbf{X}_i = \mathbf{val})\right)\right)$$

$$H\left(\mathbf{y}\right) = -\sum_{\mathbf{val}\in\phi_{\mathbf{y}}} \left(P(\mathbf{y} = \mathbf{val}) * \log_2\left(P(\mathbf{y} = \mathbf{val})\right)\right)$$

and the Joint Entropy $H\left(\mathbf{X}_i,\mathbf{y}\right)$ is defined by:

$$H\left(\mathbf{X}_i,\mathbf{y}\right) = -\sum_{\phi_{\mathbf{X}_i}} \left(\sum_{\phi_{\mathbf{y}}} \left(p_{ij} * \log_2(p_{ij})\right)\right), \quad p_{ij} = P\left(\mathbf{X}_i = \left\{\phi_{\mathbf{X}_i}\right\}_i, \mathbf{y} = \left\{\phi_{\mathbf{y}}\right\}_j\right)$$

where $\phi_{\mathbf{X}_i}, \phi_{\mathbf{y}}$ are sets of possible distinct values for feature $\mathbf{X}_i$ and target variable $\mathbf{y}$. Note that, the subscript i in $\mathbf{X}_i$ refers to the i^{th} feature of $\mathbf{X}$, whereas i, below the curly brackets, in $\left\{\phi_{\mathbf{X}_i}\right\}_i$ refers to the i^{th} element of the set $\phi_{\mathbf{X}_i}$.

Nonetheless, we made some modifications to suit the resource constraints and align with the objectives of this report. The primary changes were made to the hyperparameters for the Evolutionary Algorithm, i.e. `population = 10` and `generation = 25`, and we excluded the imbalance objective $o_{\text{imb}}(\cdot,\cdot)$ as it is impractical for tabular regression tasks. The modified version can be seen in `modified_mfs_split.py` in the GitHub repo provided.

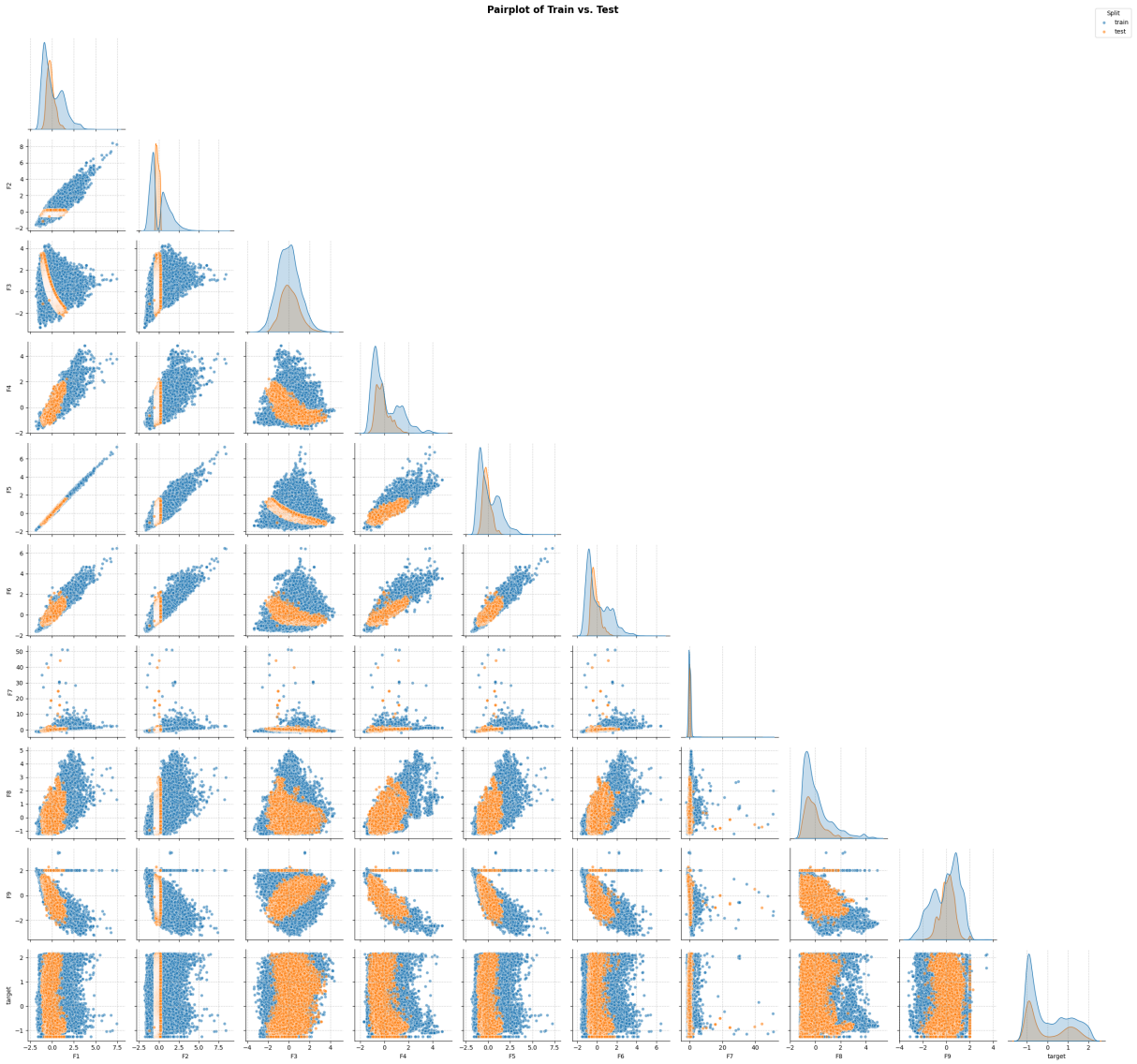

Figure 5: Pairplot of Metafeature_based_Split train_1/test_1

4.2.3 Geometry–based strategy

In this group of strategies, we utilise the **Definition 7. Interpolation vs. Extrapolation with Convex Hull** to construct train/test pairs. However, we do not explicitly apply the idea of Convex Hull. We are inspired from the geometric perspective of this definition, and thereby, proposed 5 different geometry–based approaches, i.e. Single Hyperball, Multiple Hyperballs, K–Means Hyperballs, Single Slab and Semi–infinite Slab. In the context of this report, we will be working on Euclidean space.

1. Single Hyperball

In this approach, we utilized the **Euclidean distance** of data points in the feature space $\mathcal{X}$ to split the given dataset $\mathbf{X}$. Let us define the metric used to measure the distance between two points residing in the Euclidean space.

Definition 8. Euclidean distance/metric [39]

$$\forall \mathbf{x}, \mathbf{y} \in \mathbb{R}^n : d(\mathbf{x}, \mathbf{y}) = \|\mathbf{x} - \mathbf{y}\|_2 = \sqrt{\sum_{i=1}^{n} (\mathbf{x}_i - \mathbf{y}_i)^2}$$

Now, our approach relies on the idea of the **r–Ball**. To generalize this concept, we call it **Hyperball**, similar to plane and hyperplane, which can be called in high dimensional space.

Definition 9. A Hyperball [40]

A Hyperball, or ball $\mathbf{B}_r(\mathbf{w})$ of radius r about $\mathbf{w} \in \mathbb{R}^n$ is defined as:

$$\mathbf{B}_r(\mathbf{w}) \stackrel{\text{def}}{=} \{\mathbf{x} \in \mathbb{R}^n \mid d(\mathbf{x}, \mathbf{w}) \leq r\}$$

where $d(\mathbf{x}, \mathbf{w})$ is defined by Definition 8. Indeed, it is a convex set [28].

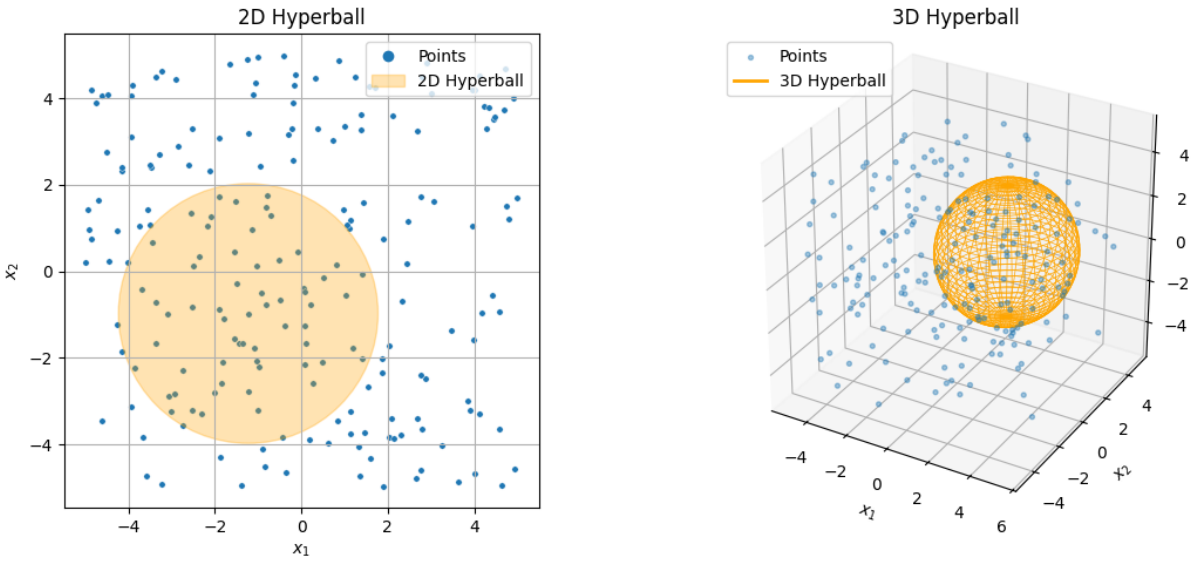

Figure 6: Hyperballs Visualization

Next, we introduce the algorithm that relies on this definition to split the datasets as following. The code below does not resemble the source code; still, it covers the core idea of the algorithm.

Algorithm 2: **Single Hyperball Split (Single_Hyperball)**

Require: $\mathbf{X} \in \mathbb{R}^{n \times d}$ (features), $\mathbf{y} \in \mathbb{R}^n$ (targets), train ratio $\alpha \in (0, 1)$
Ensure: A train/test split saved as `train.parquet` and `test.parquet`

$u \leftarrow \texttt{RandInt}(1, n)$
$\mathbf{c} \leftarrow \mathbf{x}_u \in \mathbf{X}$ ▷ Pick a random center among data points
$I_{\text{train}} \leftarrow \textsc{BallSelection}(\mathbf{X}, \mathbf{c}, \alpha)$ ▷ Select indices inside a hyperball of size $\approx \alpha \times n$
$I_{\text{test}} \leftarrow \{1, \ldots, n\} \setminus I_{\text{train}}$
$\mathbf{X}_{\text{train}} \leftarrow \mathbf{X}[I_{\text{train}}]$; $\mathbf{y}_{\text{train}} \leftarrow \mathbf{y}[I_{\text{train}}]$
$\mathbf{X}_{\text{test}} \leftarrow \mathbf{X}[I_{\text{test}}]$; $\mathbf{y}_{\text{test}} \leftarrow \mathbf{y}[I_{\text{test}}]$
$\mathcal{D}_{train} \leftarrow \textsc{ConcatColumns}(\mathbf{X}_{train}, \mathbf{y}_{train})$
$\mathcal{D}_{test} \leftarrow \textsc{ConcatColumns}(\mathbf{X}_{test}, \mathbf{y}_{test})$
Save $\mathcal{D}_{train}$ as `train.parquet`
Save $\mathcal{D}_{test}$ as `test.parquet`

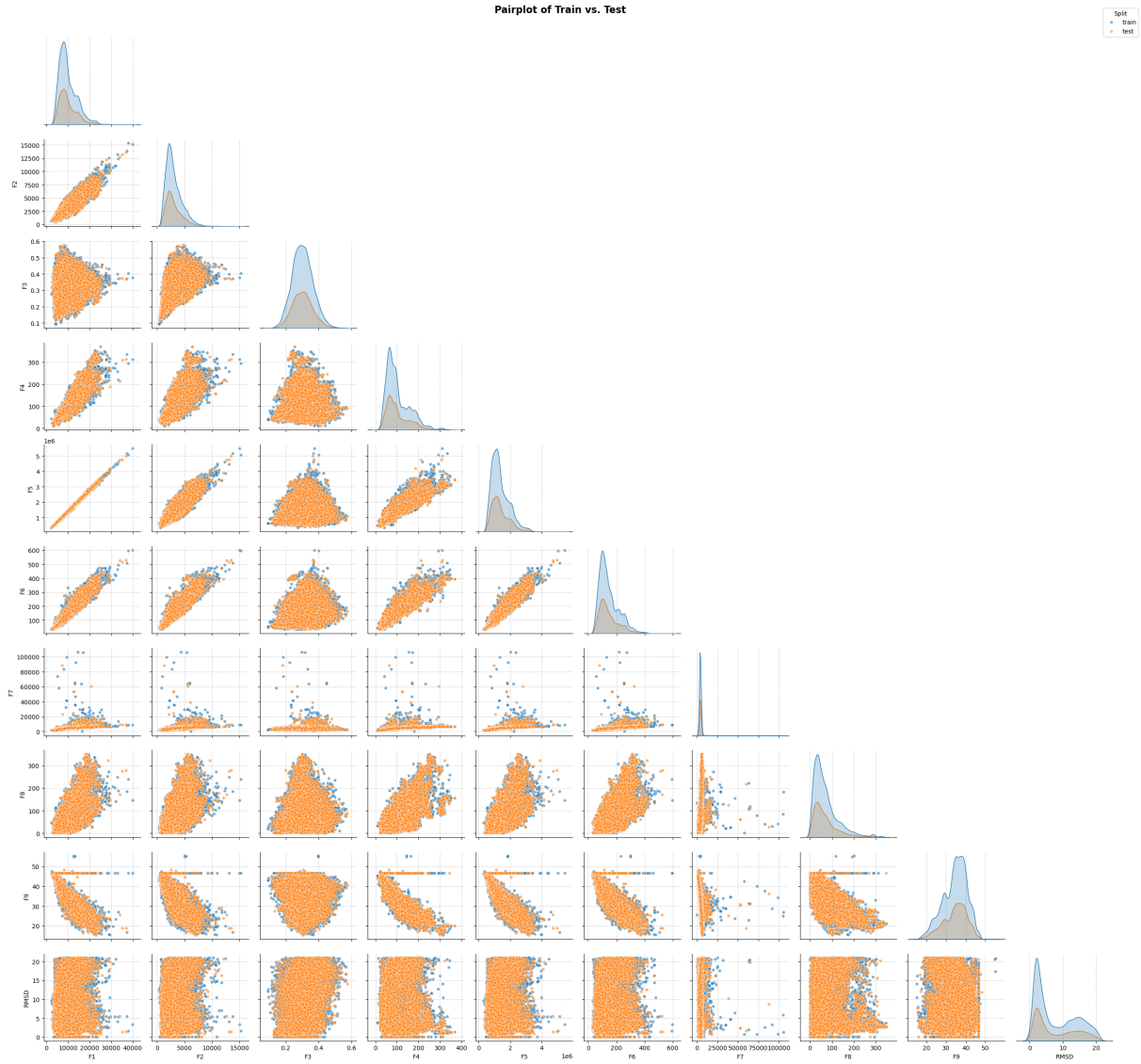

Figure 7: Pair plot of Single_Hyperball Split train_0/test_0

2. Multiple Hyperballs

In this approach, we attempt to split the feature space of the given dataset by using multiple Hyperballs in lieu of a single Hyperball. The technique can be demonstrated as follows.

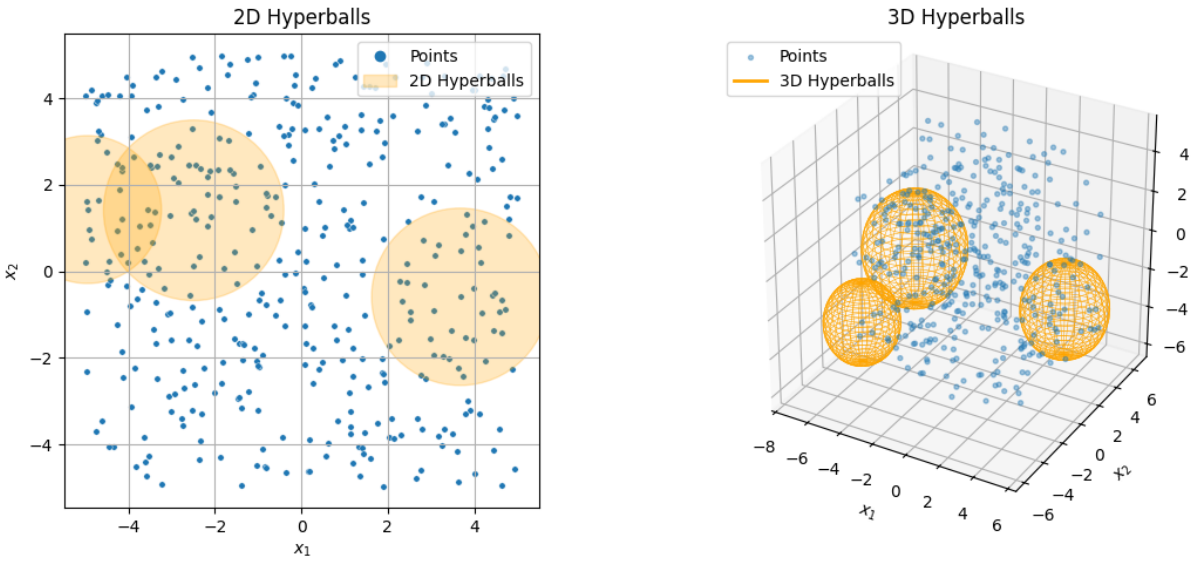

Figure 8: Multiple Hyperballs Visualization

The pseudocode is provided below, though it does not resemble the source code, it covers the primary idea of the implementation. In this report, we chose the number of Hyperballs to be 5 as default.

Algorithm 3: **Multiple Hyperball Selection (Multiple_Hyperballs)**

Require: $\mathbf{X} \in \mathbb{R}^{n \times d}$, $\mathbf{y} \in \mathbb{R}^n$, test ratio $\tau \in (0, 1)$, number of balls B
Ensure: A train/test split saved as `train.parquet` and `test.parquet`

$\mathbf{X}^{(avail)} \leftarrow \mathbf{X}$; $I_{\text{test}} \leftarrow \emptyset$ ▷ Work on a copy to avoid modifying the original set
$(\tau_1, \tau_2, \ldots, \tau_B) \leftarrow \textsc{RandomSums}(\tau, B)$ ▷ Split total test ratio into random parts
for all $\tau_b \in (\tau_1, \ldots, \tau_B)$ **do**
    $\mathbf{c} \leftarrow \textsc{SampleOne}(\mathbf{X}^{(avail)})$ ▷ Pick a random center from available points
    $S \leftarrow \textsc{BallSelection}(\mathbf{X}^{(avail)}, \mathbf{c}, \tau_b)$ ▷ Select points within a ball occupying about τ_b of the dataset
    $S \leftarrow S \setminus I_{\text{test}}$ ▷ Avoid overlap with previously selected points
    $\mathbf{X}^{(avail)} \leftarrow \mathbf{X}^{(avail)} \setminus \mathbf{X}^{(avail)}[S]$ ▷ Remove selected points to diversify later centers
    $I_{\text{test}} \leftarrow I_{\text{test}} \cup S$ ▷ Accumulate selected test indices
end for
$I_{\text{train}} \leftarrow \{1, \ldots, n\} \setminus I_{\text{test}}$
$\mathbf{X}_{\text{test}} \leftarrow \mathbf{X}[I_{\text{test}}]$; $\mathbf{y}_{\text{test}} \leftarrow \mathbf{y}[I_{\text{test}}]$
$\mathbf{X}_{\text{train}} \leftarrow \mathbf{X}[I_{\text{train}}]$; $\mathbf{y}_{\text{train}} \leftarrow \mathbf{y}[I_{\text{train}}]$
$\mathcal{D}_{train} \leftarrow \textsc{ConcatColumns}(\mathbf{X}_{train}, \mathbf{y}_{train})$
$\mathcal{D}_{test} \leftarrow \textsc{ConcatColumns}(\mathbf{X}_{test}, \mathbf{y}_{test})$
Save $\mathcal{D}_{train}$ as `train.parquet`
Save $\mathcal{D}_{test}$ as `test.parquet`

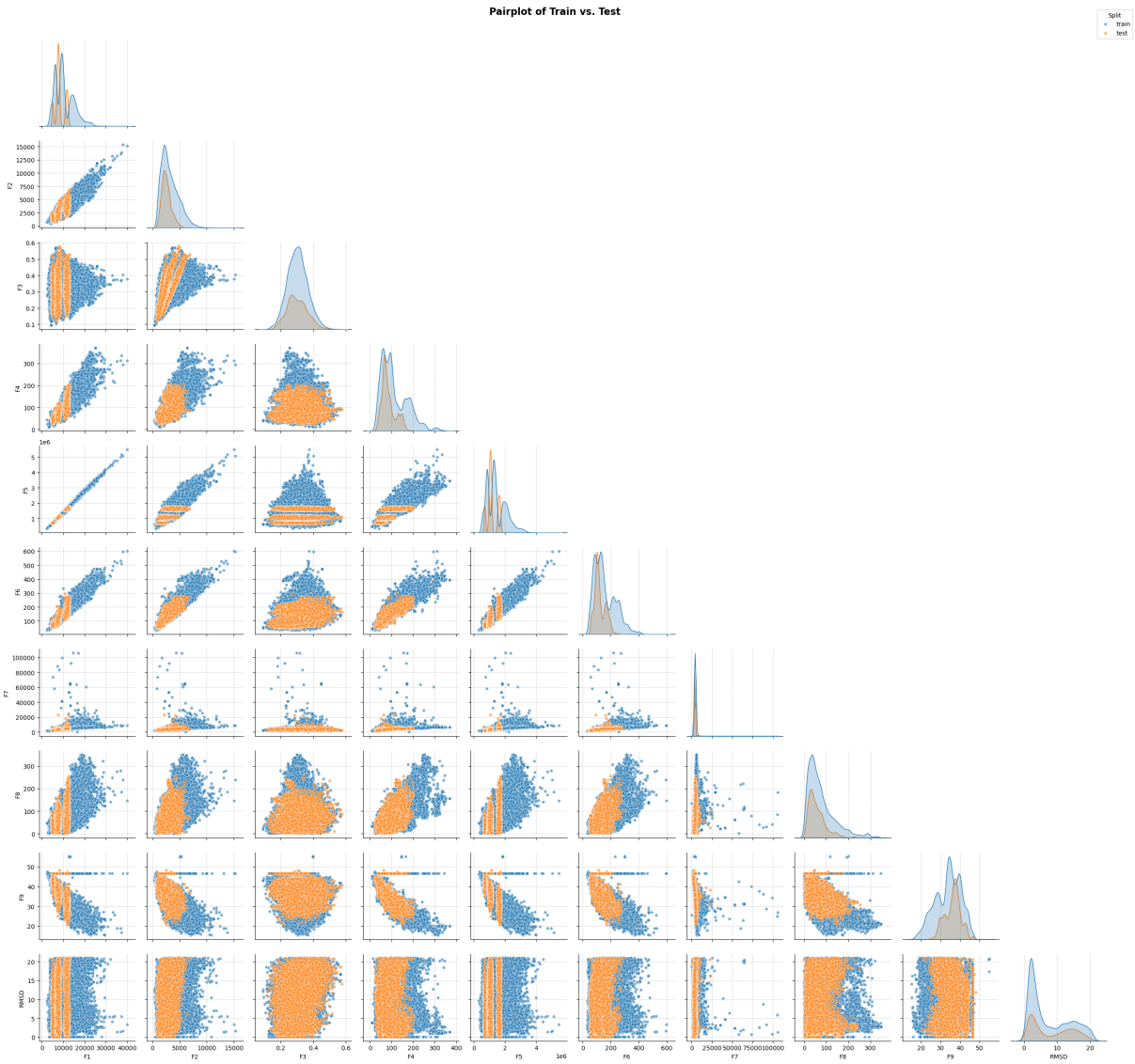

Figure 9: Pair plot of Multiple_Hyperballs Split train_0/test_0

3. K–Means Hyperballs

Developing from the previous approach, i.e. Multiple Hyperballs, we now select the centroids of the hyperballs more wisely by the utilization of the **K–Means Clustering method** ([41], [42]) implemented by Python scikit-learn library [43]. We collect the points assigned to each centroid until it reaches the expected train/test ratio. Nonetheless, this does not guarantee the selected points to be in the Hyperball about the centroids, one can either choose to follow this, or to apply the previous algorithm, i.e. Algorithm 3 with appropriate ratios for each Hyperball to construct the train/test pair, they are not significantly different. In this report, we chose the number of clusters to be 5 and the default algorithm is **Lloyd's algorithm** [42]. Since we leverage the implementation of scikit–learn library [43], we do not introduce to pseudocode of this method. Alternatively, we propose the problem formulation and objective of the solution.

Problem statement: Given a set of n data points in the feature space $\mathbf{X}$ and no label space $\mathbf{y}$, which is known as an **unsupervised problem** [42]. Group the data points into different clusters so that each cluster shares similar charateristics among the data points

residing in the same cluster. The problem can be mathmematically formulized as below.

Problem formulation [41]: Given a dataset $\mathbf{X} = \{\mathbf{x}_1, \mathbf{x}_2, \ldots, \mathbf{x}_n \mid \mathbf{x}_i \in \mathbb{R}^m\}$ of n points. Partition $\mathbf{X}$ into K clusters, denoted as $\mathcal{C} = \{C_1, C_2, \ldots, C_K\}$, subject to:

$$C_i \cap C_j = \varnothing, \forall C_i, C_j \in \mathcal{C}$$
$$\bigcup_{i=1}^{K} C_i = \mathbf{X}$$
$$\arg\min_{\mathcal{C}} \sum_{k=1}^{K} \sum_{\mathbf{x}_i \in C_k} \|\mathbf{x}_i - \boldsymbol{\mu}_k\|_2^2$$

where:

- $\|\cdot\|_2^2$ is the squared Euclidean distance.
- $\boldsymbol{\mu}_k = \frac{\sum_{\mathbf{x}_i \in C_k} \mathbf{x}_i}{|C_k|}$ is the centroid of the cluster C_k.

Lloyd's algorithm ([42], [41]):

Step 1. Clusters' centroids random initialization, i.e. $\boldsymbol{\mu} = \{\boldsymbol{\mu}_1, \boldsymbol{\mu}_2, \ldots, \boldsymbol{\mu}_K\}$

Step 2. Assign the points $\mathbf{x}_i \in \mathbf{X}$ to their nearest centroid. More specifically,

$$z(\mathbf{x}_i) = \arg\min_{k \in \overline{1,K}} \|\mathbf{x}_i - \boldsymbol{\mu}_k\|_2^2 \quad \Rightarrow C_z = C_z \cup \mathbf{x}_i$$

Step 3. Centroids (re–)calculation

$$k \in \overline{1,K}: \quad \boldsymbol{\mu}_k = \frac{\sum_{\mathbf{x}_i \in C_k} \mathbf{x}_i}{|C_k|}$$

then empty the sets $C_1, \cdots, C_K = \emptyset$

Repeat Step 2 and Step 3 until convergence, i.e. the clusters' centroids remain unchanged.

Algorithm 4: **K-Means Hyperballs (KMeans_Hyperballs)**

Require: $\mathbf{X} \in \mathbb{R}^{n \times d}$, $\mathbf{y} \in \mathbb{R}^n$, test ratio $\tau \in (0, 1)$, number of clusters K
Ensure: A train/test split saved as `train.parquet` and `test.parquet`

$kmeans \leftarrow$ KMeans($n\_clusters = K$) ▷ KMeans($\cdot$) is adopted from sklearn library [43]
Fit($kmeans, \mathbf{X}$)
$centroids \leftarrow kmeans.cluster\_centers\_$
$labels \leftarrow kmeans.labels\_$
$total \leftarrow n$
$clusters \leftarrow \{0, 1, \ldots, K-1\}$
$I_{\text{test}} \leftarrow \emptyset$ ▷ indices selected for test set
while $\frac{|I_{\text{test}}|}{total} < \tau$ **and** $clusters \neq \emptyset$ **do**
    $cl \leftarrow$ RandomChoice($clusters$)
    $C \leftarrow \{\, i \in \overline{1,n} : labels[i] = cl \,\}$ ▷ indices of chosen cluster
    if $\frac{|I_{\text{test}}|+|C|}{total} < \tau$ **then**
        $I_{\text{test}} \leftarrow I_{\text{test}} \cup C$

else
    $num \leftarrow \lfloor \tau \cdot total - |I_{\text{test}}| \rfloor$
    $I_{\text{test}} \leftarrow I_{\text{test}} \cup \textsc{FirstK}(C, num)$ ▷ take first num indices
end if
$clusters \leftarrow clusters \setminus \{cl\}$ ▷ prevent re-selecting the same cluster
end while
$I_{\text{train}} \leftarrow \{1, \ldots, n\} \setminus I_{\text{test}}$
$\mathbf{X}_{\text{train}} \leftarrow \mathbf{X}[I_{\text{train}}]$; $\mathbf{y}_{\text{train}} \leftarrow \mathbf{y}[I_{\text{train}}]$
$\mathbf{X}_{\text{test}} \leftarrow \mathbf{X}[I_{\text{test}}]$; $\mathbf{y}_{\text{test}} \leftarrow \mathbf{y}[I_{\text{test}}]$
$\mathcal{D}_{train} \leftarrow \textsc{ConcatColumns}(\mathbf{X}_{\text{train}}, \mathbf{y}_{\text{train}})$
$\mathcal{D}_{test} \leftarrow \textsc{ConcatColumns}(\mathbf{X}_{\text{test}}, \mathbf{y}_{\text{test}})$
Save $\mathcal{D}_{train}$ as `train.parquet`
Save $\mathcal{D}_{test}$ as `test.parquet`

Figure 10: Pair plot of KMeans_Hyperballs Split train_0/test_0

In the last two splitting techniques, we are inspired by the geometric perspectives of **single slab** and **semi–infinite slab** from Partial Differential Equations [44], not their

calculus meaning. Thus, we utilize the notions of planes, vectors and distance to construct these two kinds of slab.

4. Single Slab

Firstly, let us formally review certain useful notions.

Definition 10. A Hyperplane [28]

A Hyperplane P is a set given by:

$$P \stackrel{\text{def}}{=} \{\mathbf{x} \in \mathbb{R}^n \mid \mathbf{n}^\intercal \mathbf{x} = b\}$$

where $\mathbf{n} \in \mathbb{R}^n \setminus \{\mathbf{0}\}$ is called the normal vector, which is normal/orthogonal to P, i.e. $\mathbf{n} \perp P$, and $b \in \mathbb{R}$.
Said differently, P is defined by a normal vector $\mathbf{n}$ and a point $\mathbf{x}_0 \in \mathbb{R}^n$. The above set turns out to be:

$$P \stackrel{\text{def}}{=} \{\mathbf{x} \in \mathbb{R}^n \mid \mathbf{n}^\intercal (\mathbf{x} - \mathbf{x}_0) = 0\}$$

then $b = \mathbf{n}^\intercal \mathbf{x}_0$.

It is imperative to know how to measure the distance of a point $\mathbf{x}$ to a given hyperplane P, as it determines whether the points belong to train or test sets.

Definition 11. (Shorteast) Distance from a point to a Hyperplane [40]

Given a point $\mathbf{w} \in \mathbb{R}^n$ and a hyperplane $P = \{\mathbf{x} \mid \mathbf{n}^\intercal \mathbf{x} = b\} \subset \mathbb{R}^n$. The shorteast distance from $\mathbf{w}$ to P is defined as:

$$d(\mathbf{x}, P) \stackrel{\text{def}}{=} \frac{|\mathbf{n}^\intercal \mathbf{x} - b|}{\|\mathbf{n}\|_2}$$

From the above definitions, we propose a useful notation of δ–**slab**, which is leveraged to partition the datasets.

Definition 12. A δ–slab

Given a hyperplane $P = \{\mathbf{x} \mid \mathbf{n}^\intercal \mathbf{x} = b\} \subset \mathbb{R}^n$. **A δ–slab** defined by hyperplane P, is given by:

$$Slab_\delta(P) \stackrel{\text{def}}{=} \{\mathbf{x} \in \mathbb{R}^n \mid d(\mathbf{x}, P) < \delta\} \subset \mathbb{R}^n, \quad \text{for } \delta \in \mathbb{R}$$

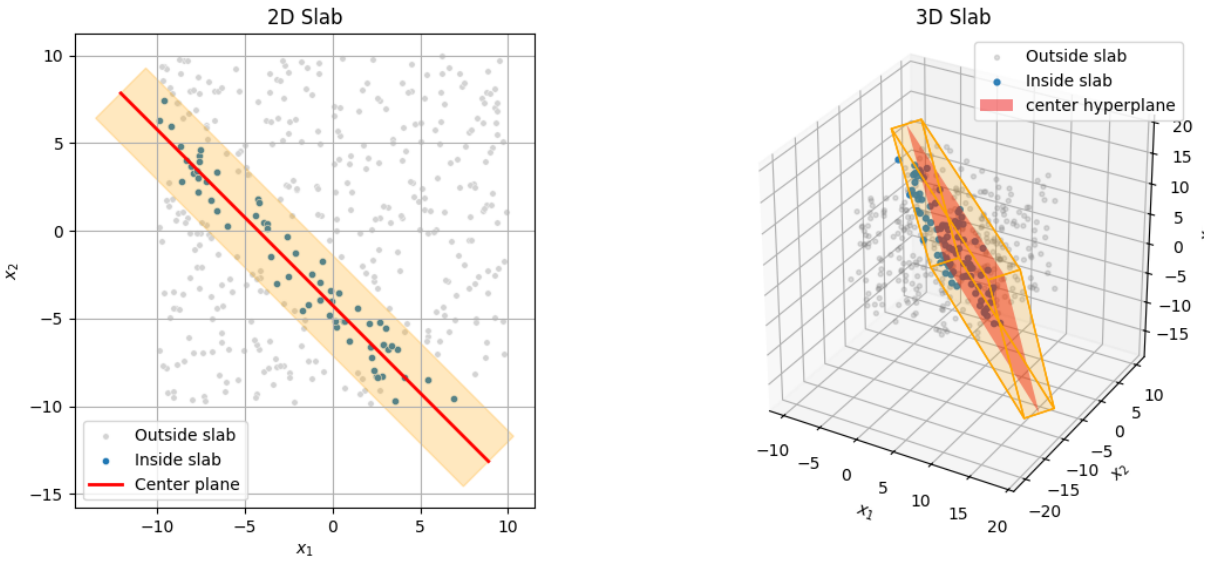

Figure 11: δ–slab Visualization

Next, we propose the following algorithm that follows the concept of δ–slab discussed above to split the given datasets into train/test pairs.

Algorithm 5: **Single Slab (Single_Slab)**

Require: $\mathbf{X} \in \mathbb{R}^{n \times d}$, $\mathbf{y} \in \mathbb{R}^n$, train ratio $\alpha \in (0, 1)$
Ensure: A train/test split saved as `train.parquet` and `test.parquet`

$\varepsilon \leftarrow 10^{-2}$
$(\mathbf{n}, b, \mathbf{p}_0) \leftarrow \textsc{ConstructHyperplane}(\mathbf{X})$
▷ **Binary search** for δ so that $\approx \alpha \times n$ points lie within the δ–slab
$lo \leftarrow 0$
$high \leftarrow \max_{i \in \overline{1,n}} \|\mathbf{X}_i - \mathbf{p}\|_2$
while $lo < high$ **do**

$\delta \leftarrow lo + \dfrac{high - lo}{2}$

$I_{\text{train}} \leftarrow \left\{ i \in \overline{1,n} : \dfrac{|\mathbf{n}^\intercal \mathbf{X}_i - b|}{\|\mathbf{n}\|_2} < \delta \right\}$ ▷ indices within the δ–slab
if $\dfrac{|I_{\text{train}}|}{n} < \alpha$ **then**
$lo \leftarrow \delta + \varepsilon$
else
$high \leftarrow \delta - \varepsilon$
end if
end while
$I_{\text{test}} \leftarrow \{1, \ldots, n\} \setminus I_{\text{train}}$
$\mathbf{X}_{\text{train}} \leftarrow \mathbf{X}[I_{\text{train}}]$; $\mathbf{y}_{\text{train}} \leftarrow \mathbf{y}[I_{\text{train}}]$
$\mathbf{X}_{\text{test}} \leftarrow \mathbf{X}[I_{\text{test}}]$; $\mathbf{y}_{\text{test}} \leftarrow \mathbf{y}[I_{\text{test}}]$
$\mathcal{D}_{train} \leftarrow \textsc{ConcatColumns}(\mathbf{X}_{\text{train}}, \mathbf{y}_{\text{train}})$
$\mathcal{D}_{test} \leftarrow \textsc{ConcatColumns}(\mathbf{X}_{\text{test}}, \mathbf{y}_{\text{test}})$
Save $\mathcal{D}_{train}$ as `train.parquet`
Save $\mathcal{D}_{test}$ as `test.parquet`

Figure 12: Pair plot of Single_Slab Split train_0/test_0

5. Semi–infinite Slab

In this final approach, we utilise the hyperplane P defined in **Definition 10. A Hyperplane** to split the given datasets. [28] A hyperplane $P \subset \mathbb{R}^n$ partitions the space into two halves, i.e. a (closed) halfspace $\mathcal{S}_1 = \{\mathbf{x} \mid \mathbf{n}^\intercal \mathbf{x} \leq b\}$ and an (open) halfspace $\mathcal{S}_2 = \{\mathbf{x} \mid \mathbf{n}^\intercal \mathbf{x} > b\}$, which hold $\mathbb{R}^n = \mathcal{S}_1 \cup \mathcal{S}_2$ and $\mathcal{S}_1 \cap \mathcal{S}_2 = \varnothing$. In our testbench, we use the (closed) halfspace to construct the test set and the remaining will be allocated to the train set. More formally:

Definition 13. A semi–infinite slab

Given a hyperplane $P = \{\mathbf{x} \mid \mathbf{n}^\intercal \mathbf{x} = b\} \subset \mathbb{R}^n$. **A semi–infinite slab** defined by hyperplane P, is given by:

$$Slab_\infty(P) \stackrel{\text{def}}{=} \{\mathbf{x} \in \mathbb{R}^n \mid \mathbf{n}^\intercal \mathbf{x} < b\}$$

Indeed, it is a convex set [28].

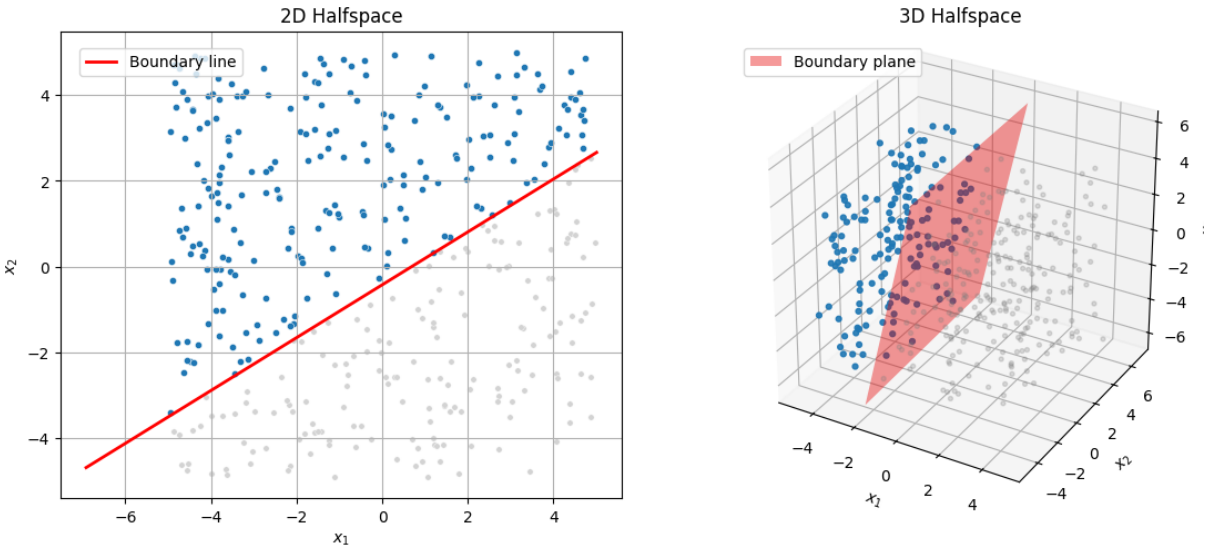

Figure 13: Semi–infinite slab Visualization

Next, we propose the following algorithm that follows the concept of semi–infinite slab discussed above to split the given datasets into train/test pairs.

Algorithm 6: **Semi-Infinite Slab (Semi_Infinite_Slab)**

Require: $\mathbf{X} \in \mathbb{R}^{n \times d}$, $\mathbf{y} \in \mathbb{R}^n$, test ratio $\tau \in (0, 1)$
Ensure: A train/test split saved as `train.parquet` and `test.parquet`

$\varepsilon \leftarrow 0.01$; $cur \leftarrow 0$
$(\mathbf{n}, \mathbf{lo}, \mathbf{hi}) \leftarrow \textsc{FindBounds}(\mathbf{X}, \tau)$ ▷ $\mathbf{n}$ is a random normal; $\mathbf{lo}, \mathbf{hi}$ are bounding points for binary search for appropriate points to construct the hyperplane
while $cur < \tau$ **or** $cur > \tau + \varepsilon$ **do**
    $\mathbf{p} \leftarrow \mathbf{lo} + \dfrac{\mathbf{hi} - \mathbf{lo}}{2}$
    $b \leftarrow \mathbf{n}^\top \mathbf{p}$
    $I_{\text{test}} \leftarrow \textsc{DataOneSide}(\mathbf{X}, \mathbf{n}, b)$
    $cur \leftarrow \dfrac{|I_{\text{test}}|}{n}$
    if $cur < \tau$ **then**
        $\mathbf{lo} \leftarrow \mathbf{p} + \varepsilon$ ▷ shift lower bound upward
    else
        $\mathbf{hi} \leftarrow \mathbf{p} - \varepsilon$ ▷ shift upper bound downward
    end if
end while
$I_{\text{train}} \leftarrow \{1, \ldots, n\} \setminus I_{\text{test}}$
$\mathbf{X}_{\text{train}} \leftarrow \mathbf{X}[I_{\text{train}}]$; $\mathbf{y}_{\text{train}} \leftarrow \mathbf{y}[I_{\text{train}}]$
$\mathbf{X}_{\text{test}} \leftarrow \mathbf{X}[I_{\text{test}}]$; $\mathbf{y}_{\text{test}} \leftarrow \mathbf{y}[I_{\text{test}}]$
$\mathcal{D}_{train} \leftarrow \textsc{ConcatColumns}(\mathbf{X}_{\text{train}}, \mathbf{y}_{\text{train}})$
$\mathcal{D}_{test} \leftarrow \textsc{ConcatColumns}(\mathbf{X}_{\text{test}}, \mathbf{y}_{\text{test}})$
Save $\mathcal{D}_{train}$ as `train.parquet`
Save $\mathcal{D}_{test}$ as `test.parquet`

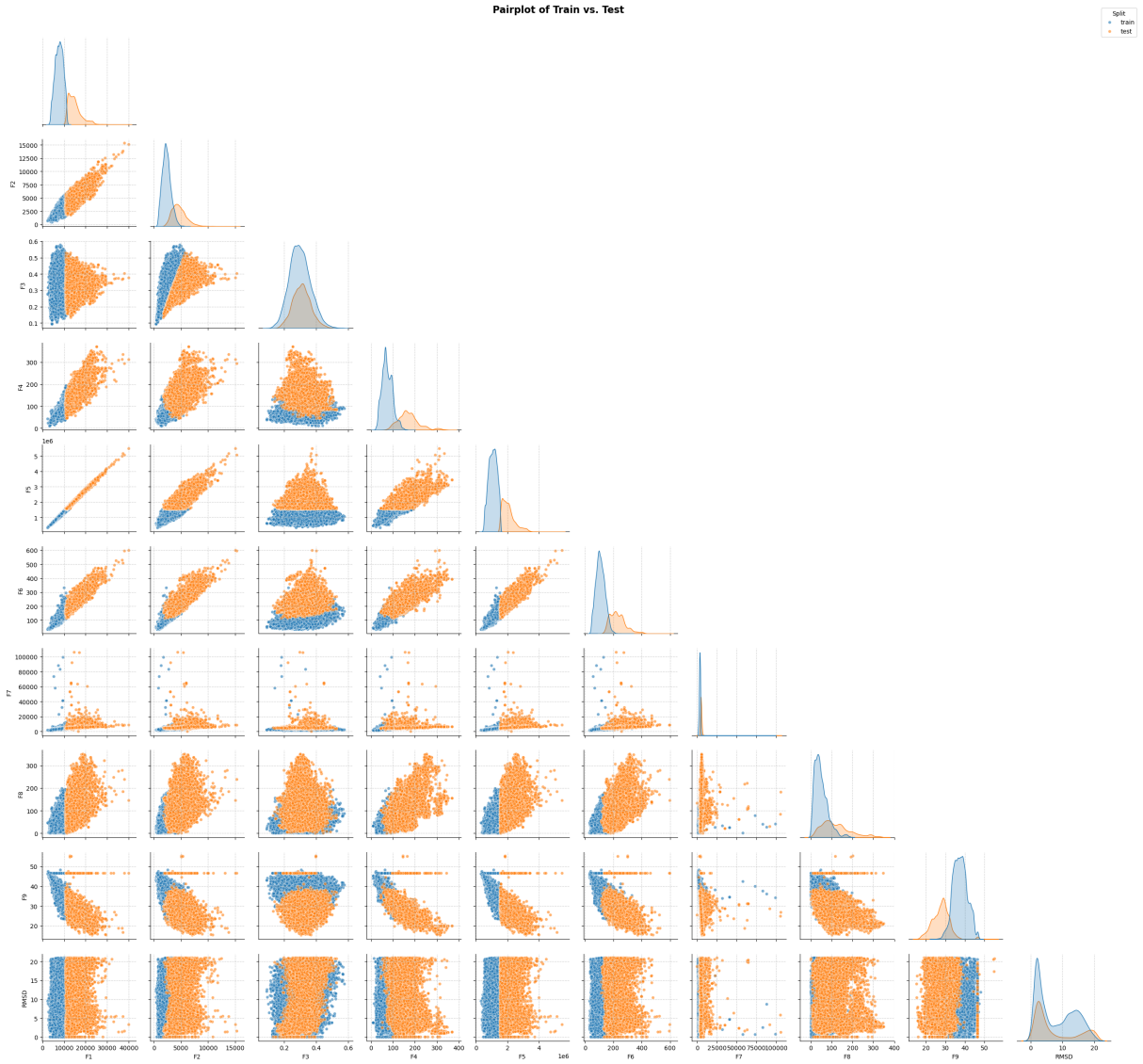

Figure 14: Pair plot of Semi–infinite_Slab Split train_0/test_0

4.3 Models

Next, we provide the baseline models, those are usually preferred when discussing about regression problems ([45], [46]). In this work, we do not designate any proposed models as baseline to compare with others. Alternative, we compare all models together to find out the "best" model for OOD generalization on low–dimensional, tabular regression data. Additionally, the models' configurations were selected heuristically so as to balance the models' complexity and precision trade–off on the train sets. Nonetheless, the codebase also contains the readily used hyperparameter tuning framework, i.e. Optuna [47], if practicioners prefer. Throughout this work, we leveraged the Python sklearn library [43] to construct and evaluate models, except for XGBoost, LightGBM, RealMLP, ResNet, and FT–Transformer models, which were implemented or adapted from external sources. Also, we provide several figures to depict feature–wise models' performance using 95% confidence interval. Again, we utilise the the dataset `CASP` and standardize it using StandardScaler() from sklearn library [43] before training and subsequently plotting these figures.

4.3.1 Linear Regression

Definition 14. Linear Regression [48] [49]

Given a dataset $\mathcal{D}^{\text{tr}} = (\mathbf{X}^{n\times d}, \mathbf{y})$. The linear regression is expected to find an approximation function:

$$\hat{f}(\mathbf{x}) = \boldsymbol{\theta}^{\intercal}\mathbf{x} + \mathbf{b}$$

subject to minimizing the **Empirical Risk** $\mathcal{R}_{\text{emp}}(\hat{f})$, where $\mathbf{x} = [x_1, x_2, \cdots, x_d]$, $\boldsymbol{\theta} = [\theta_1, \theta_2, \cdots, \theta_d]$ and $\mathbf{b} = [b_1, b_2, \cdots, b_d]$ are called trainable/learnable parameters.

Since now, the use of $\boldsymbol{\theta}, \mathbf{b}$ as trainable parameters of the model being considered, is applied in the context of this report.

Typically, the most widely used loss functions for Linear Regression are Mean Squared Error (MSE) and Mean Absolute Error (MAE). Nonetheless, [50] each of them has both advantages and critical drawbacks. In particular, MSE is differentiable at all points, but bound to outliers sensitivity, which is not recommended in the domain of data generalization. Meanwhile, MAE is more robust to outliers, compared to MSE, but is not differentiable. Therefore, we leveraged the Huber Loss [51] as our loss functions to train this simple model. The loss function is defined as follows.

Definition 15. Huber loss [50]

Given a pair of input/output $(\mathbf{x}_i, y_i)$, where $y \equiv f : \mathcal{X} \mapsto \mathcal{Y}$, and an approximation function $\hat{y} \equiv \hat{f} : \mathcal{X} \mapsto \mathcal{Y}$. The Huber loss is given by:

$$\ell_\epsilon(y, \hat{y}) = \begin{cases} \dfrac{1}{2}(y - \hat{y})^2, & \text{if } |y - \hat{y}| \leq \epsilon \\ \epsilon|y - \hat{y}| - \dfrac{1}{2}\epsilon^2, & \text{otherwise} \end{cases}$$

where $\epsilon \in \mathbb{R}_{>0}$, which is a hyperparameter required tuning.

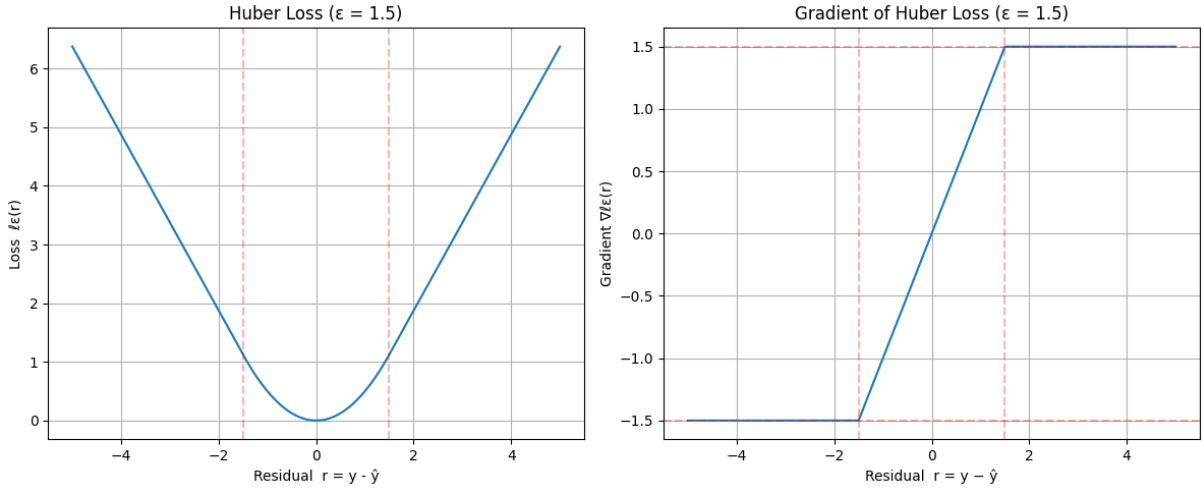

Figure 15: Huber Loss and its Gradient Visualization

Aside from being [50] robust to outliers and differentiable on $\mathbb{R}$, the convergence speed is much faster as the gradients beyond the threshold ϵ stay constant, i.e. $\nabla_r \ell_\epsilon = \epsilon = 1.5$.

To [52] improve the models' generalization to unseen data, we added a new term to the initial loss function, i.e. regularization term. More formally, the models are trained to minimize the **regularized** empirical risk.

Definition 16. Regularized Empirical Risk [43]

$$\textbf{reg}\text{–}\mathcal{R}_{\text{emp}}\left(\hat{f}\right) \equiv \textbf{reg}\text{–}\mathcal{R}_{\mathbf{x}\in\mathcal{X}^{\text{tr}}}\left(\hat{f}\right) \stackrel{\text{def}}{=} \mathbb{E}_{\mathbf{x}\in\mathcal{X}^{\text{tr}}}\Big[\ell\Big(\hat{f}(\mathbf{x}), f(\mathbf{x})\Big)\Big] + \lambda\mathbf{R}(\boldsymbol{\theta})$$

where

- $\lambda \in \mathbb{R}_{\geq 0}$ is a hyperparameter used to control the impact of the regularization term.
- $\mathbf{R}$ is the regularization term.

Typically, there are two well–known regularization terms, i.e. [49] L_1 norm (used in Lasso Regression) and [48] L_2 norm (used in Ridge Regression). However, [53] each of which has its limitations, and hence, we utilize the ElasticNet [53] to learn the appropriate regularization term for the models presented in this report.

Definition 17. Penalty/Regularization terms $\mathbf{R}(\boldsymbol{\theta})$ [43]

- $L_1\text{–}\mathbf{R}(\boldsymbol{\theta}) \stackrel{\text{def}}{=} \|\boldsymbol{\theta}\|_1$
- $L_2\text{–}\mathbf{R}(\boldsymbol{\theta}) \stackrel{\text{def}}{=} \frac{1}{2}\|\boldsymbol{\theta}\|_2^2$
- ElasticNet–$\mathbf{R}(\boldsymbol{\theta}) \stackrel{\text{def}}{=} \rho \times L_2\text{–}\mathbf{R}(\boldsymbol{\theta}) + (1-\rho) \times L_1\text{–}\mathbf{R}(\boldsymbol{\theta})$

where $\rho \in [0, 1]$.

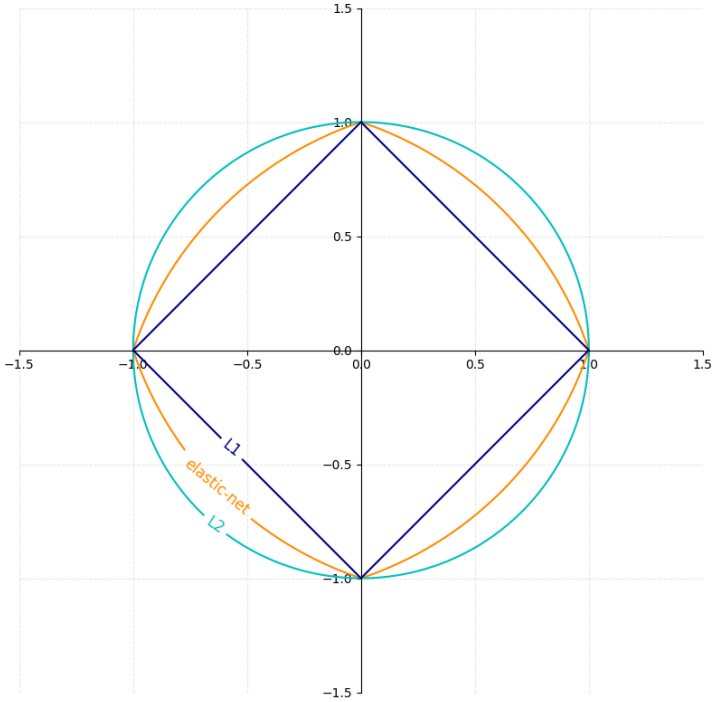

Figure 16: Regularization terms Visualization [43]

We utilise the Stochastic Gradient technique SGDRegressor() [43] to train the model. See **Table 6** for further details about the hyperparameter settings. The subplots provided below show how Linear Regression with Huber Loss performs on each feature of the example dataset.

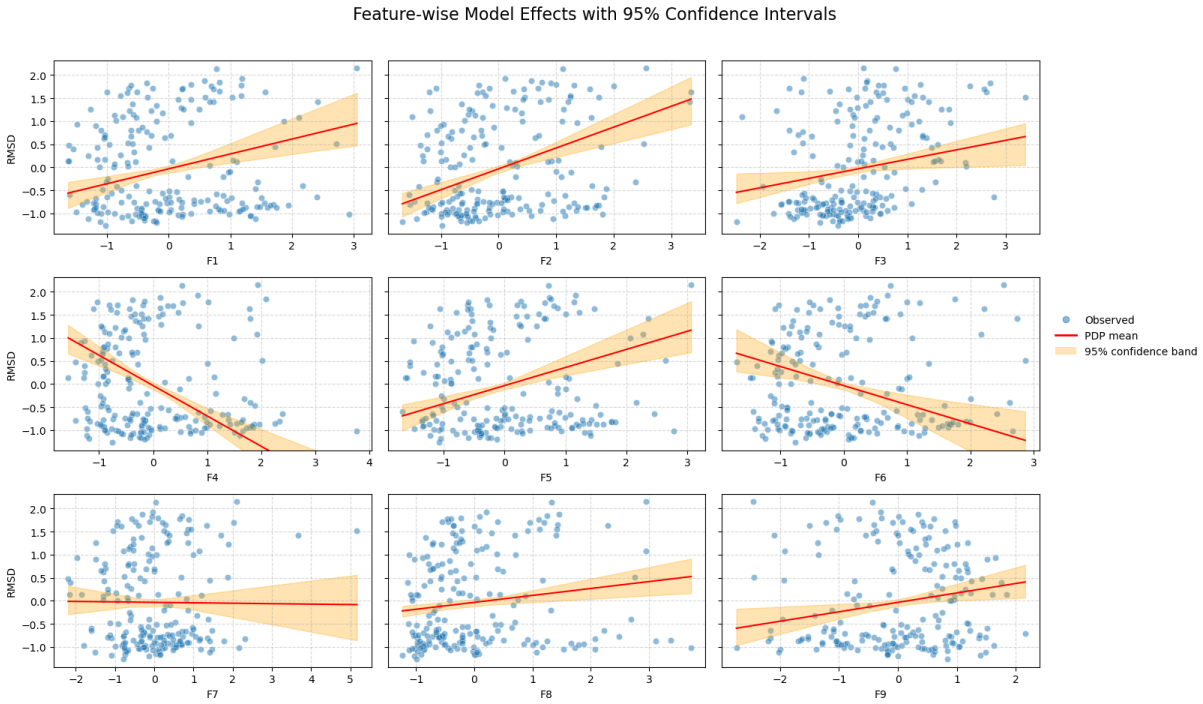

Figure 17: Feature–wise Linear Regression Effect with 95% confidence interval

4.3.2 Polynomial Regression

In most real–world data, the relationships between the covariates and the target are non–linear. Thereby, to better approximate the non–linearity phenomena in real data, we utilize the simpleast form of function approximation, i.e. Polynomial Regression.

Definition 18. Multi–index notation α [54]

Define the multi–index α as:

$$\alpha(d; deg) = (\alpha_1, \alpha_2, \cdots, \alpha_d), \quad \alpha_i \in \mathbb{N} \quad \text{and} \quad |\alpha| = \sum_i \alpha_i = deg$$

where $deg \in \mathbb{N}$ is the degree being considered. Given a vector $\mathbf{x} = [x_1, x_2, \cdots, x_d]$. Then:

- $x_i^{\alpha_j} = x_i \times x_i \times \cdots \times x_i, \quad \alpha_j$ times.
- $\mathbf{x}^\alpha = x_1^{\alpha_1} \times x_2^{\alpha_2} \times \cdots \times x_d^{\alpha_d}$

Definition 19. Polynomial Regression [54]

Given a dataset $\mathcal{D}^{\text{tr}} = (\mathbf{X}^{n\times d}, \mathbf{y})$. The polynomial regression of degree "deg", is expected to find an approximation function:

$$\hat{f}_{deg}(\mathbf{x}) = \boldsymbol{\theta}^\intercal \tilde{\mathbf{x}} + \mathbf{b}$$

subject to minimizing the **Empirical Risk** $\mathcal{R}_{\text{emp}}(\hat{f})$, where

$$\tilde{\mathbf{x}} = \begin{bmatrix} \left[(\mathbf{x}^\alpha)_{\alpha\sim S(d;1)}\right] \\ \left[(\mathbf{x}^\alpha)_{\alpha\sim S(d;2)}\right] \\ \vdots \\ \left[(\mathbf{x}^\alpha)_{\alpha\sim S(d;deg)}\right] \end{bmatrix}, \quad S(d;k) \triangleq \{\alpha(d;k)\} \text{ pairwise distinct and } |S| = \binom{d+k-1}{k}$$

and $|\boldsymbol{\theta}| = |\mathbf{b}| = \sum_{k=1}^{deg} \binom{d+k-1}{k}$.

We leverage the PolynomialFeatures() from sklearn library [43] to complement features that are the interaction between the existing features, and omit the terms whose degree is higher than 2. We believe this setting would make the model less likely to overfit some individual features and encourage it to explore the interaction between the features. For example:

$$\begin{aligned} &\text{PolynomialFeatures}([X_1, X_2, X_3]; \text{deg} = 3) \\ &= [X_1, X_2, X_3, X_1X_2, X_1X_3, X_2X_3, X_1X_2X_3] \end{aligned}$$

Then, we train the model using SGDRegressor() [43] on this new set of features, objective to minimizing the **Regularized Empirical Risk** using **Huber loss** and **ElasticNet** as the penalty coefficient. See **Table 8** for further details about the hyperparameter settings. The subplots provided below show how Polynomial Regression performs on each feature of the example dataset.

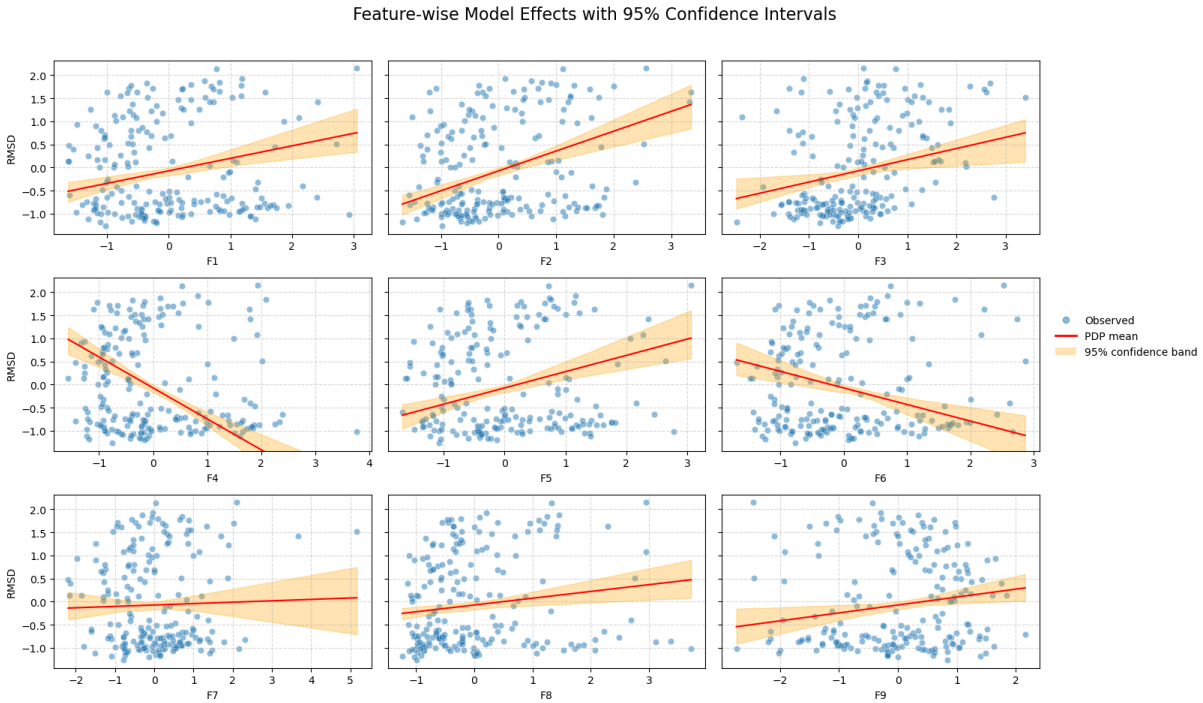

Figure 18: Feature–wise Polynomial Regression Effect with 95% confidence interval

4.3.3 K–Neareast Neighbours–based Regression

In our next regression model, we rely on the assumption that neighbouring points may have underlying patterns that determine the value of the target value, which is known as the [55] locality in data space. From this assumption, the model simply makes prediction by taking the average of the nearest points of the given datum. Recall our initial assumption that we are working on Euclidean space, thereby the distances between points are measured using **Euclidean distance**.

Definition 20. KNN Regression [55] [56]

Given a dataset $\mathcal{D}^{\text{tr}} = (\mathbf{X}^{n\times d}, \mathbf{y})$. The KNN regression is expected to find an approximation function:

$$\hat{f}_K(\mathbf{x}) = \frac{1}{K} \sum_{i \in I_K(\mathbf{x})} \omega(\mathbf{x}, \mathbf{x}_i) \times y_i$$

where $I_K(\mathbf{x})$ is the set of K nearest samples, in $\mathcal{D}^{\text{tr}}$, to $\mathbf{x}$, and $\omega(\mathbf{x}, \mathbf{x}_i)$ is the weight imposed on the neighbouring points based on their distances.

The weight function $\omega(\cdot)$ can be, but not limited to [56]:

- Uniform distance:
$$\omega(\mathbf{x}, \mathbf{x}_i) = 1$$
- Inverse distance
$$\omega(\mathbf{x}, \mathbf{x}_i) = \frac{1}{d(\mathbf{x}, \mathbf{x}_i)}$$
- Exponential distance
$$\omega(\mathbf{x}, \mathbf{x}_i) = e^{-d(\mathbf{x}, \mathbf{x}_i)}$$

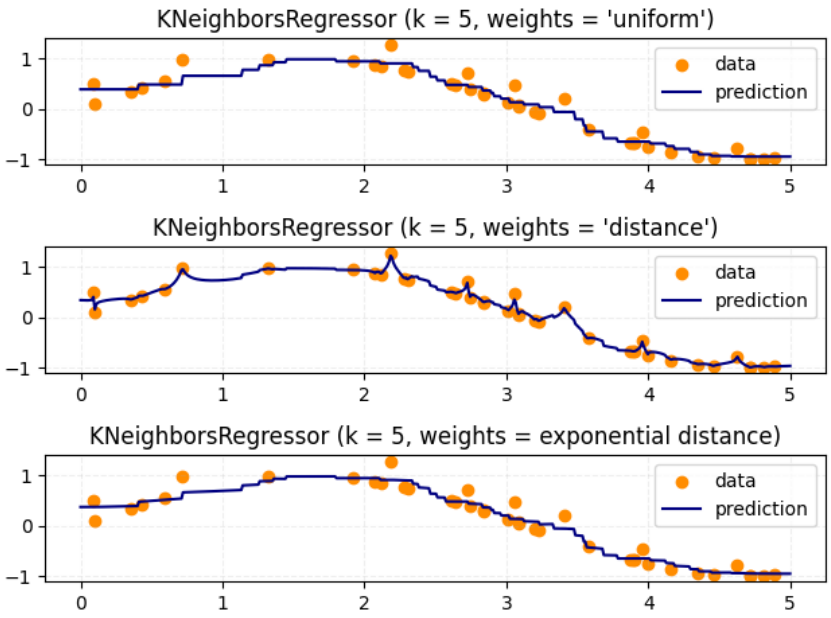

Figure 19: Weighted Distance functions Visualization [43]

In this work, we chose the **Inverse distance** to compute the prediction for any given data as we believe that the distance has strong correlation with the target value. Besides, in the regard of finding the nearest neighbours, we let the sklearn [43] decide the algorithm. To train this model, we utilize KNeighborsRegressor() [43]. See **Table 10** for further details about the hyperparameter settings. The subplots provided below show how KNN Regression performs on each feature of the example dataset.

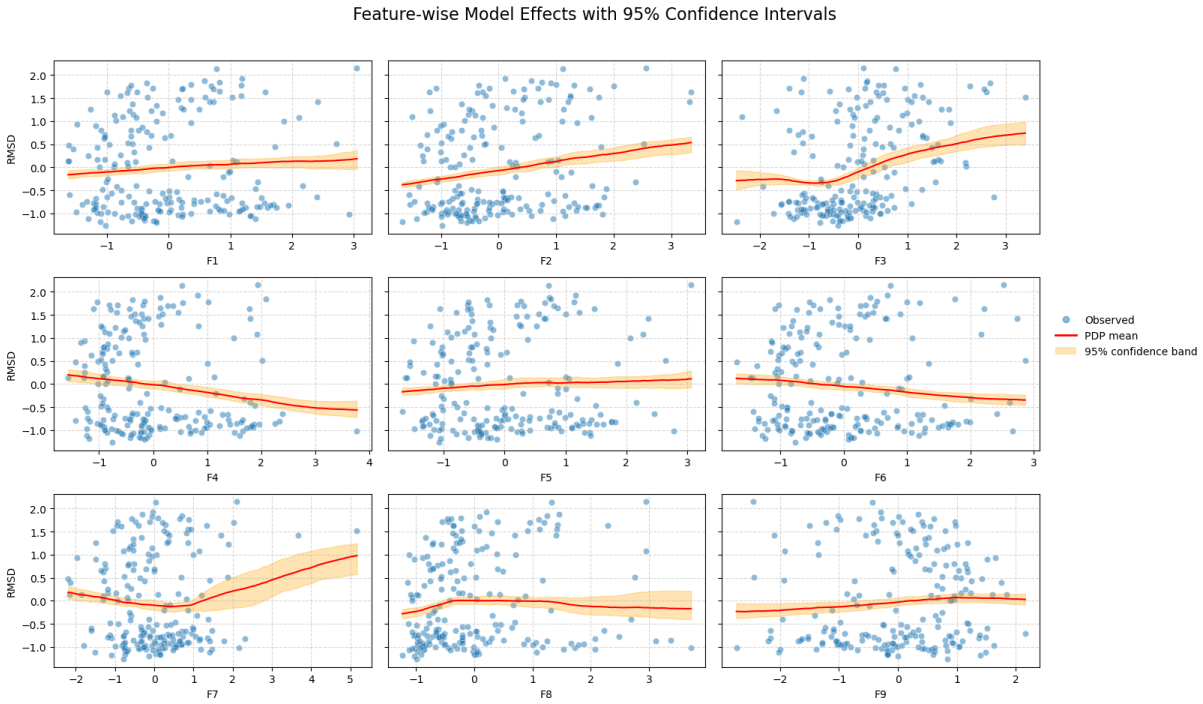

Figure 20: Feature–wise KNN Regression Effect with 95% confidence interval

4.3.4 Support Vector Machine–based Regression

Next, we utilize Support Vector Machine (SVM) theory to approach the problem of function estimation, known as Support Vector Regression (SVR), it has both linear and non–linear versions of SVR [57]. For the sake of simplicity, we will train and evaluate the linear SVR, which, in fact, a **Linear Regression** model trained by a different loss function.

Definition 21. ϵ**–insensitive loss** [58] [57]

Given a pair of input/output $(\mathbf{x}_i, y_i)$, where $y \equiv f : \mathcal{X} \mapsto \mathcal{Y}$, and an approximation function $\hat{y} \equiv \hat{f} : \mathcal{X} \mapsto \mathcal{Y}$. The ϵ–insensitive loss is given by:

$$\ell_\epsilon(y, \hat{y}) = \begin{cases} 0, & \text{if } |\hat{y} - y| < \epsilon \\ |\hat{y} - y| - \epsilon, & \text{otherwise} \end{cases}$$

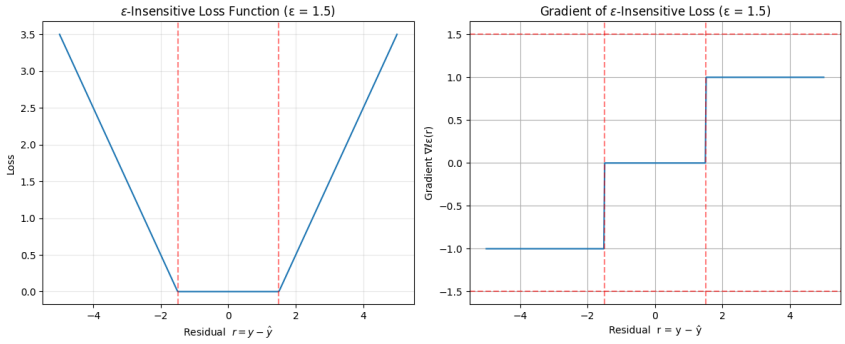

Figure 21: ϵ–Insensitive Loss and its Gradient Visualization

Definition 22. Linear SVR [57] [58]

Given a dataset $\mathcal{D}^{\text{tr}} = (\mathbf{X}^{n \times d}, \mathbf{y})$. The Linear SVR is expected to find an approximation function:

$$\hat{f}_\epsilon(\mathbf{x}) = \boldsymbol{\theta}^\intercal \mathbf{x} + \mathbf{b}$$

subject to minimizing the **regularized empirical risk reg**–$\mathcal{R}_{\text{emp}}(\hat{f})$ of ϵ–**insensitive loss**.

Note that, in some literature ([57] [58]), they introduced a similar **reg**–$\mathcal{R}_{\text{emp}}(\hat{f})$ as:

$$\textbf{reg}\text{–}\mathcal{R}_{\text{emp}}(\hat{f}) \overset{\text{def}}{=} C \times \mathbb{E}_{\mathbf{x} \in \mathcal{X}^{\text{tr}}} \Big[\ell_\epsilon \Big(\hat{f}(\mathbf{x}), f(\mathbf{x}) \Big) \Big] + \mathbf{R}(\boldsymbol{\theta})$$

where $C \in \mathbb{R}_{\geq 0}$ is a hyperparameter used to [57] balance between the flatness of $\hat{f}$ and the amount of tolerance to deviations from ϵ. It turns out to be the [58] inverse regularization term, i.e. $C = \dfrac{1}{\lambda}$.

To build this ML model, we, again, utilise the SGDRegressor() using ϵ–**insensitive loss** supported by sklearn library [43]. However, it does not accept the hyperparameter C, thereby, our assumption objective function is the proposed **regularized empirical risk reg**–$\mathcal{R}_{\text{emp}}(\hat{f})$ within this report, not the ones introduced in [57] [58]. In addition, we use the L_2–$\mathbf{R}(\boldsymbol{\theta})$ to penalize model's overfitting parameters, this regularization term also aligns with the regularized objective function in [57] [58]. See **Table 12** for further details about the hyperparameter settings. The subplots provided below show how SVR performs on each feature of the example dataset.

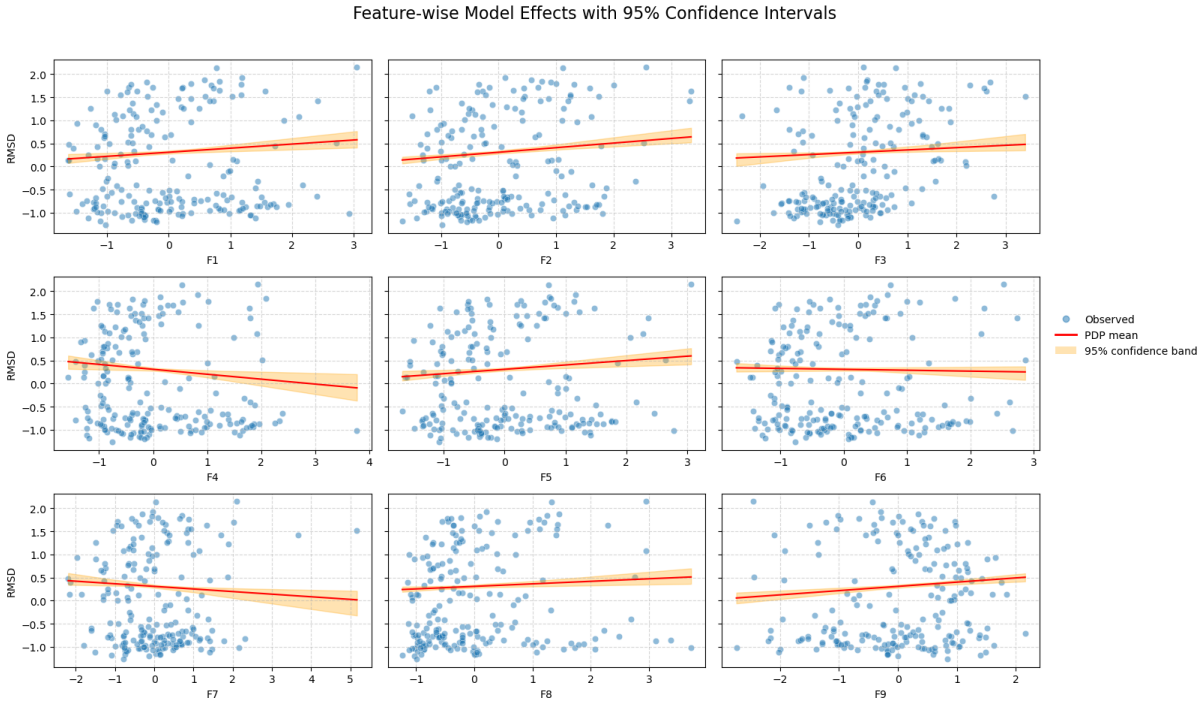

Figure 22: Feature–wise SVR Effect with 95% confidence interval

4.3.5 Decision Tree–based Regression

So far, we have been working on gradient–based models. Next, we would like to move to a completely new class of models, i.e. tree–based models. At the simplest form of tree–based model, we will review and evaluate performance of Decision Tree (DT) regression.

Definition 23. Decision Tree Regression [59] [60] [61]

Given a dataset $\mathcal{D}^{\text{tr}} = (\mathbf{X}^{n\times d}, \mathbf{y})$. The DT regression is expected to find an approximation function:

$$\hat{f}_{\mathcal{C}}(\mathbf{x}) = \sum_{C\in\mathcal{C}} \left(\frac{1}{|C|} \sum_{y\in C} y \right) \cdot \mathbb{1}_C(\mathbf{x})$$

subject to minimizing the **Empirical Risk** $\mathcal{R}_{\text{emp}}(\hat{f})$, where:

- $\mathcal{C} = \{C_1, C_2, \ldots, C_k\}$ is a set of k regions/clusters partitioned by the DT.
- $\mathbb{1}_C(\mathbf{x})$ is the indicator function defined as

$$\mathbb{1}_C(\mathbf{x}) = \begin{cases} 1, & \text{if } \mathbf{x} \in C \\ 0, & \text{otherwise} \end{cases}$$

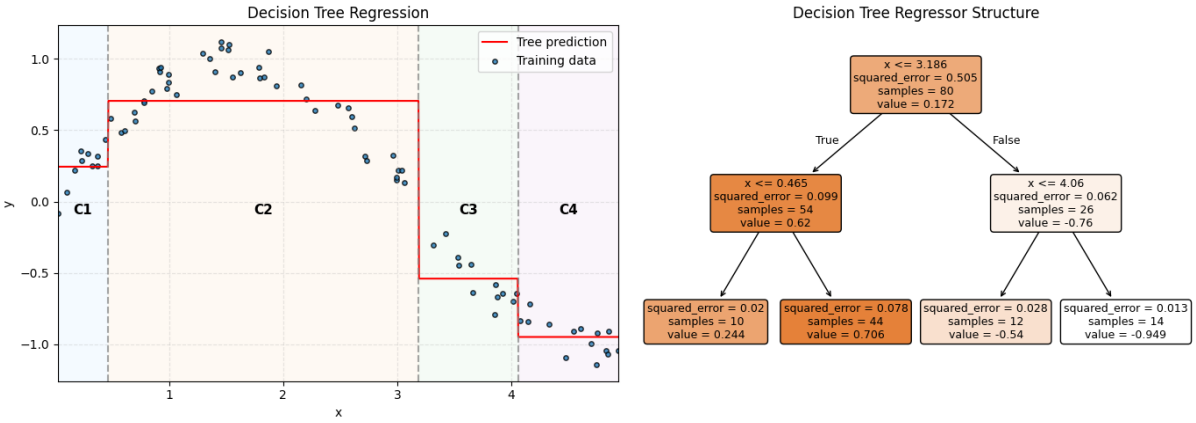

Figure 23: Decision Tree Regression and Structure Visualization

Although it is not our main goal to show how the algorithm works, as we leverage the DecisionTreeRegressor() from sklearn [43], it is imperative to know what the splitting criteria used in this report, i.e. SSE. More specifically, [59] [61] supposing our current region $C_k = (\mathbf{X}', \mathbf{y}') \subset (\mathbf{X}, \mathbf{y})$, we would then split it into two sub–regions.

For the j^{th} feature of $\mathbf{X}$ and a threshold $\delta \in \mathbb{R}$:

$$C_{2k} = \{\mathbf{x} \in C_k \mid \mathbf{x}_j < \delta\} \quad \text{and} \quad C_{2k+1} = \{\mathbf{x} \in C_k \mid \mathbf{x}_j \geq \delta\}$$

subject to:

$$\min_{j,\delta} \left\{ \sum_{y\in C_{2k}} (y - \mu_{C_{2k}})^2 + \sum_{y\in C_{2k+1}} \left(y - \mu_{C_{2k+1}}\right)^2 \right\}, \quad \text{where } \mu_i = \frac{1}{|C_i|} \sum_{y\in C_i} y$$

Theoretically, Decision Tree can partition the training dataset fully, such that it achieves 100% precision, or zero loss, on the training set, such tree is known as the fully grown DT. Indeed, this is overfitting, and thereby we would have to avoid this to aim for OOD generalization. Many literature proposed a strategy [59] [61] [60], namely Tree Pruning, somewhat analogous to the introduced **regularized empirical risk reg**–$\mathcal{R}(\hat{f})$. However,

DT is a non–parametric model [62], as there are no learnable parameters. Therefore, [61] offered a quite different objective function:

$$\textbf{reg–}\mathcal{R}_{\text{emp}}(\hat{f}) = \mathcal{R}_{\text{emp}}(\hat{f}) + \lambda|\mathcal{C}|$$

Nonetheless, in our work, we do not train towards this objective. Alternatively, [62] we limit the size of regions, or the number of terminal nodes, $\mathcal{C}$, DT's depth and the minimum number required at one node to perform further splitting at that node. See **Table 14** for further details about the hyperparameter settings. The subplots provided below show how DT Regression performs on each feature of the example dataset.

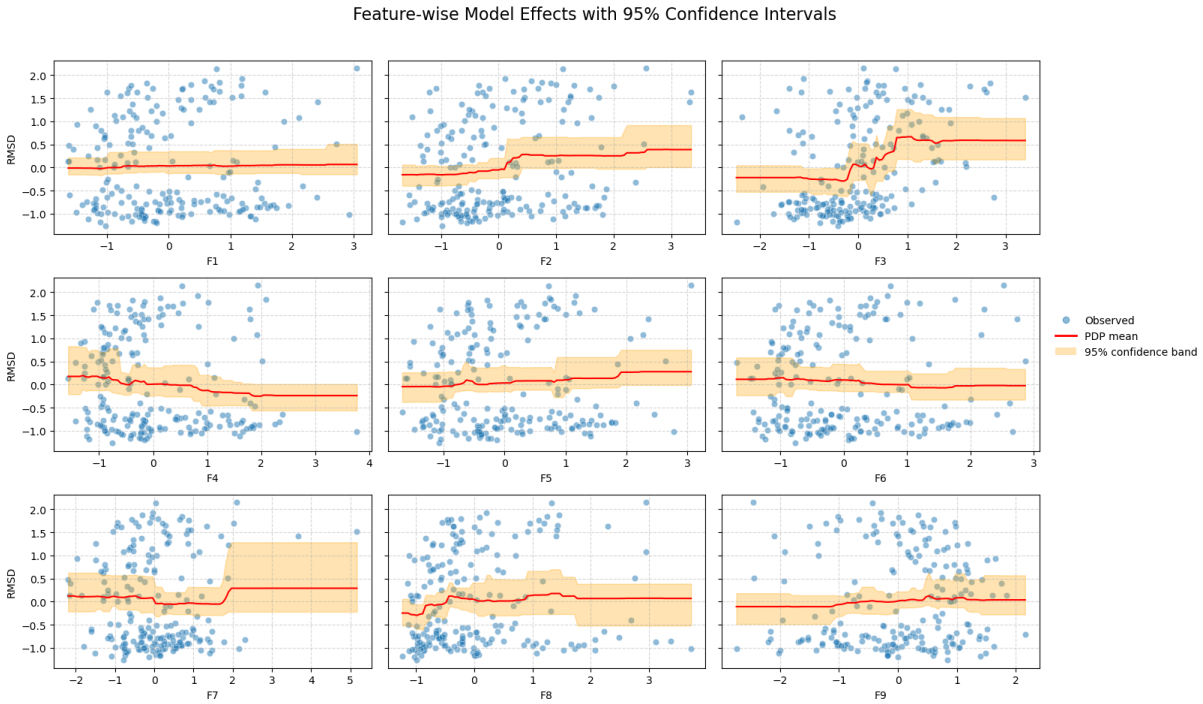

Figure 24: Feature–wise DT Regression Effect with 95% confidence interval

4.3.6 Random Forest–based Regression

Next, we will leverage the idea of tree–based predictors and working on another class of ML, i.e. ensemble learning [63], consisting of two main approaches, i.e. parallel and sequential ensemble. More specifically, it combines multiple models, called base/weak learners, to yield prediction rather than using a single model. It was proven to be superior to implementing a single model [63], [61] [59]. We first evaluate a parallel ensemble method, known as bagging or bootstrap aggregation, via Random Forest.

Definition 24. Random Forest Regression [59] [61] [62]

Given a dataset $\mathcal{D}^{\text{tr}} = (\mathbf{X}^{n\times d}, \mathbf{y})$. The RF regression is expected to find an approximation function:

$$\hat{f}_{\mathcal{G}}(\mathbf{x}) = \frac{1}{|\mathcal{G}|}\sum_{i=1}^{|\mathcal{G}|}\hat{g}_i(\mathbf{x}) \times \omega_i(\hat{g}_i)$$

where $\omega_i(\hat{g}_i)$ is the weight function assigned to each learner and $\mathcal{G} = \{\hat{g}_1, \ldots, \hat{g}_k\}$ are the ensemble of base learners trained on different subsets of $\mathcal{D}^{\text{tr}}$.

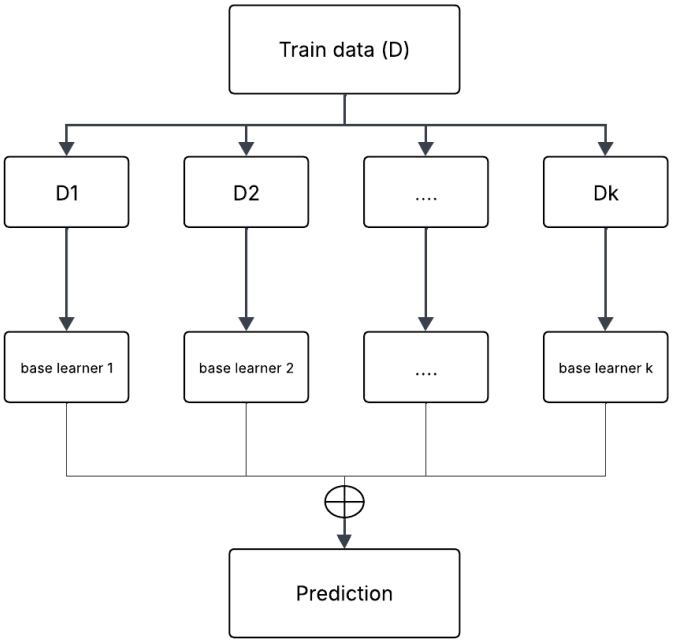

Figure 25: Parallel Ensemble Learning Visualization [63]

Note that, the base learners in this model are the shallow **DT Regressors**. In particular, they are DTs with limited depths and terminal nodes. The RF Regressor computes the prediction by taking average of all base learners [43], i.e. $\omega_i(\hat{g}_i) = 1, \text{for } i = \overline{1,k}$. See **Table 16** for further details about the hyperparameter settings. The subplots provided below show how RF Regression performs on each feature of the example dataset.

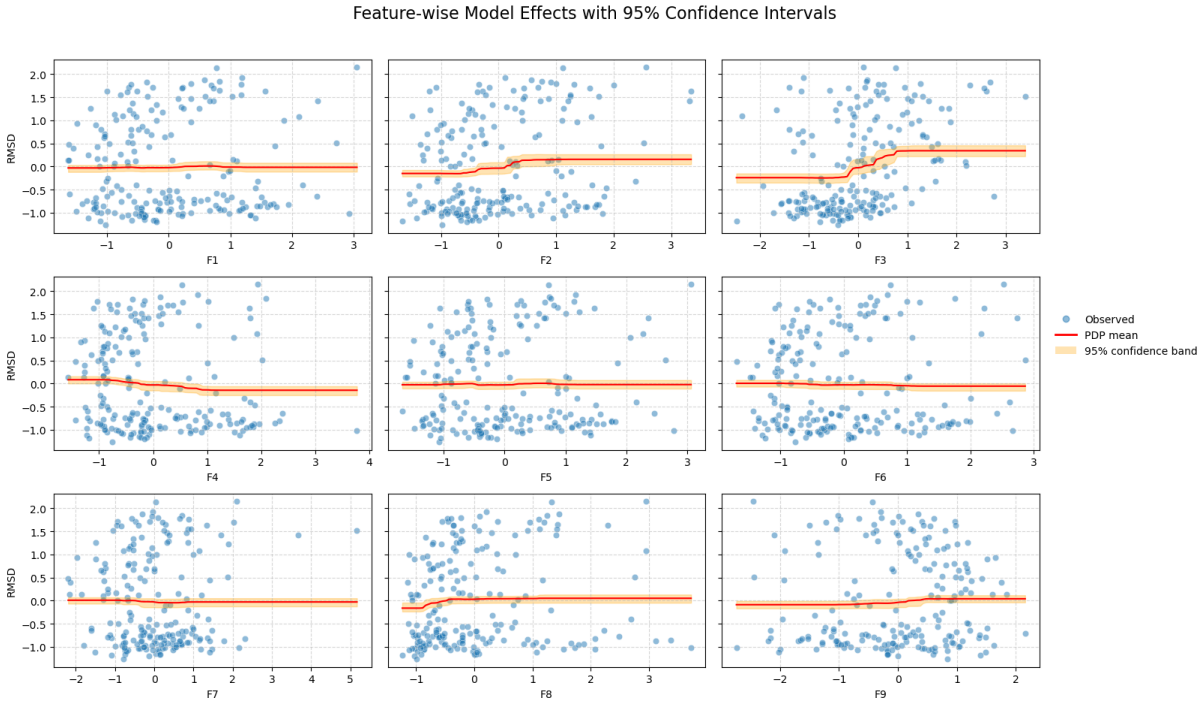

Figure 26: Feature–wise RF Regression Effect with 95% confidence interval

4.3.7 AdaBoost–based Regression

The next four models follow the idea of sequential ensemble learning, i.e. Boosting. [63] Simply put, the algorithm iteratively train a weak learner, update the dataset using the predefined weights and predictions of that weak learner, and save the learners' configuration on that modified version of data. It terminates when reaching the predefined number of weak learners. Now, let us introduce the very first successful Boosting solution, i.e. AdaBoost, short for Adaptive Boosting. Since we are utilizing the implementation of AdaBoost by sklearn library [43], the following notion follows the algorithmic design of the library.

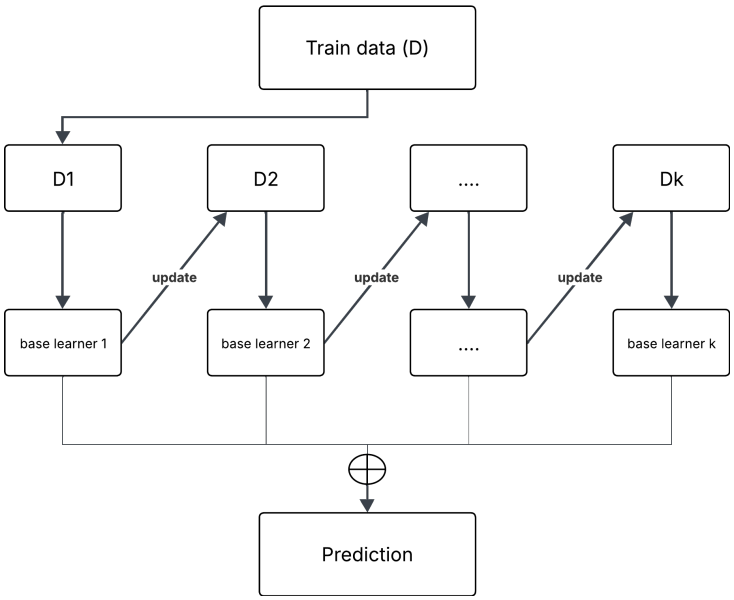

Figure 27: Sequential Ensemble Learning Visualization [63]

It is noteworthy that, in AdaBoost, the original dataset is aggregated with a weight column and hence, base learners are trained using the mutiplication of the true output and its corresponding weight.

Definition 25. AdaBoost Regression [63] [64]

Given a dataset $\mathcal{D}^{\text{tr}} = (\mathbf{X}^{n\times d}, \mathbf{y})$. The AdaBoost regression is expected to find an approximation function:

$$\hat{f}(\mathbf{x}) = \inf\left\{ \sum_{i:\hat{g}_i(\mathbf{x})\leq y} \ln\left(\frac{1}{\beta_i}\right) \geq \frac{1}{2}\sum_{i=1}^{|\mathcal{G}|} \ln\left(\frac{1}{\beta_i}\right) \right\}$$

where $\mathcal{G} = \{\hat{g}_1, \ldots, \hat{g}_k\}$ are the ensemble of weak learners trained on different weighted $\mathcal{D}^{\text{tr}}$ and β_i is the confidence of base model $\hat{g}_i$.

This formulation is also known as the weighted median, which is more robust to outliers than weighted mean (shown in **RF Regression** and **KNN Regression**). Likewise, the weak learners used in AdaBoost are shallow DTs as described in **RF Regression**. See **Table 18** for further details about the hyperparameter settings. The subplots provided below show how AdaBoost Regression performs on each feature of the example dataset.

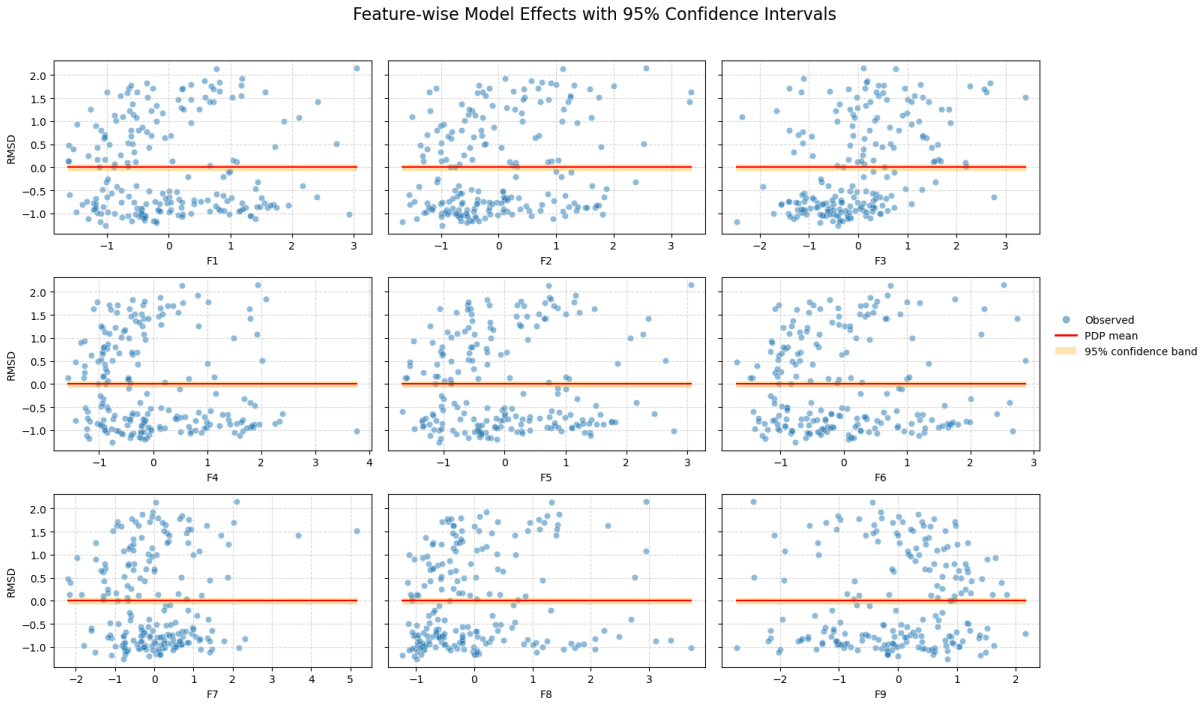

Figure 28: Feature–wise AdaBoost Regression Effect with 95% confidence interval

4.3.8 Gradient Boosting–based Regression

Next, we move on to another application of Boosting method, i.e. Gradient Boosting. In the previous model, AdaBoost, it iteratively updates the weights assigned to the outputs of the dataset. In contrast, Gradient Boosting proposes a new term, i.e. "pseudo–residuals" [65], which are computed using the squared error (possibly other metrics) of the true outputs and the prediction of the currently trained weak learner. Similarly, the weak learner is then trained using this updated dataset.

Unlike AdaBoost, which leverage the weighted median, said differently, [64] yields predictions using one weak learner whose cumulative weight is more than half of the total weights. Gradient Boosting computes prediction based on all base learners considering the learning rates of each, some sort of a Generalized Additive Model (GAM) [61].

$$\hat{f}(\mathbf{x} \in \mathbb{R}^d) = \sum_{i=1}^{d} \hat{g}_i(\mathbf{x}_i) + \beta$$

However, $\hat{g}_i$ are feature–wise models, while, the base learners are DTs, which predict based on several features. More precisely, Gradient Boosting is a [65] forward stage–wise additive model.

Definition 26. Gradient Boosting Regression [65] [63]

Given a dataset $\mathcal{D}^{\text{tr}} = (\mathbf{X}^{n\times d}, \mathbf{y})$. The GB regression is expected to find an approximation function:

$$\hat{f}(\mathbf{x}) = \sum_{i=0}^{k} \eta \hat{g}_i(\mathbf{x})$$

subject to greedily minimizing the **empirical risk** at each stage i^{th}:

$$\hat{g}_i = \arg\min_{\hat{g}} \mathcal{R}_{\text{emp}} \left(\hat{f}_{i-1} + \hat{g} \right)$$

where η is the fixed learning rate (a hyperparameter), $\hat{g}_i$ are obtained through line search optimization, and k is the total number of base learners. Note that, $\hat{g}_0$ is the initial guess, typically mean for squared error and median for absolute error. Additionally, in [65] suggests using line search optimization to find the learning rate η for each $\hat{g}_i$. Nonetheless, due to its complexity, sklearn [43] applies the same fixed learning rate for all base learners.

See **Table 20** for further details about the hyperparameter settings. The subplots provided below show how GB Regression performs on each feature of the example dataset.

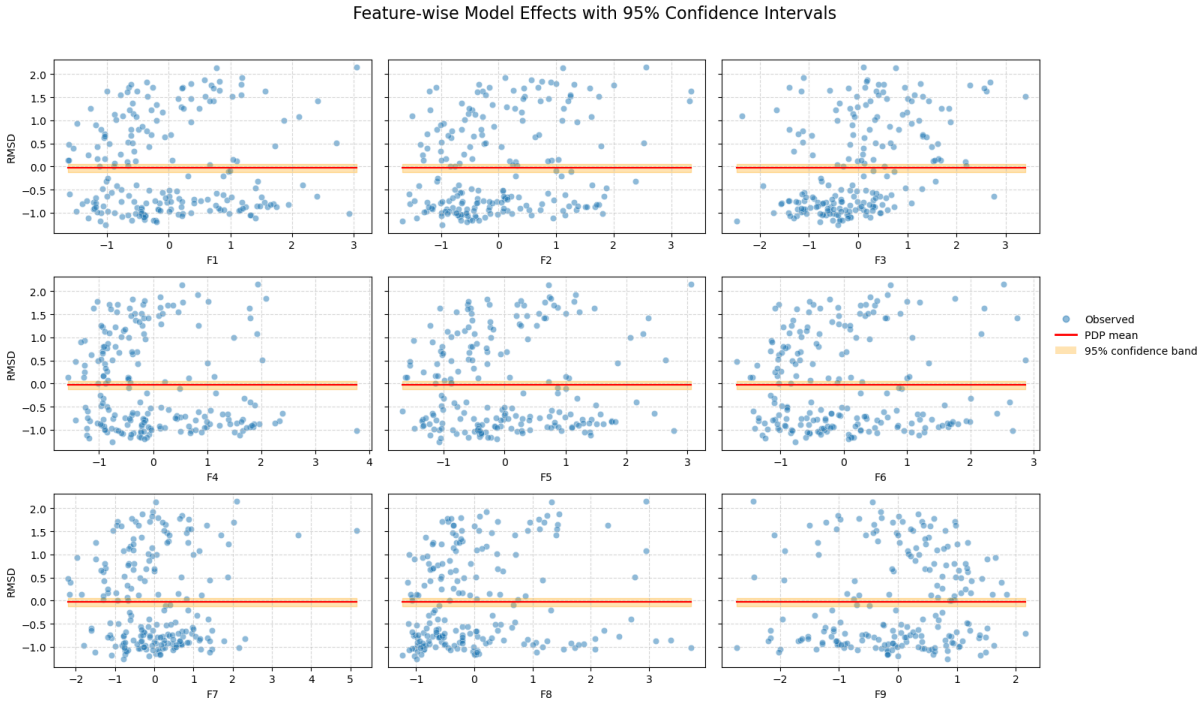

Figure 29: Feature–wise GB Regression Effect with 95% confidence interval

4.3.9 XGBoost–based Regression

The eXtreme Gradient Boosting, short as XGBoost, model is, indeed, an [66] optimization of priorly reviewed Gradient Boosting model. Aside from [67] system design improvements that support parallel training and feature engineering, it contributes towards the theoretical perspectives of the Gradient Boosting by the utilization of Hessian of loss functions to accelerate the convergence speed and stronger regularization to penalize overfitting attempts.

Definition 27. XGBoost Regression [67] [63]

Given a dataset $\mathcal{D}^{\text{tr}} = (\mathbf{X}^{n \times d}, \mathbf{y})$. The GB regression is expected to find an approximation function:

$$\hat{f}(\mathbf{x}) = \sum_{i=0}^{k} \eta \hat{g}_i(\mathbf{x})$$

subject to greedily minimizing the **regularized empirical risk** at each stage i^{th}:

$$\begin{aligned}\hat{g}_i &= \arg\min_{\hat{g}} \textbf{reg}\text{–}\mathcal{R}_{\text{emp}}\left(\hat{f}_{i-1} + \hat{g}\right) \\ &\backsimeq \mathbb{E}_{(\mathbf{x},y)\in\mathcal{D}^{\text{tr}}}\left[\ell\left(\hat{f}_{i-1}(\mathbf{x}), y\right) + \partial_{\hat{f}_{i-1}}\ell \times \hat{g}_i(\mathbf{x}) + \frac{1}{2}\partial_{\hat{f}_{i-1}}\ell \times \hat{g}_i^2(\mathbf{x})\right] + \mathbf{R}(\boldsymbol{\theta})\end{aligned}$$

wherein the penalty is given by:

$$\mathbf{R}(\boldsymbol{\theta}) = \gamma \times T + \frac{1}{2}\lambda\|\boldsymbol{\theta}\|_2^2$$

where η is the fixed predefined learning rate, λ and γ are regularization hyperparameters, T is the number of terminal nodes or leaves of the current tree, and $\boldsymbol{\theta}$ specifies the prediction of the leaf nodes.

See **Table 22** for further details about the hyperparameter settings. The subplots provided below show how XGBoost Regression performs on each feature of the example dataset.

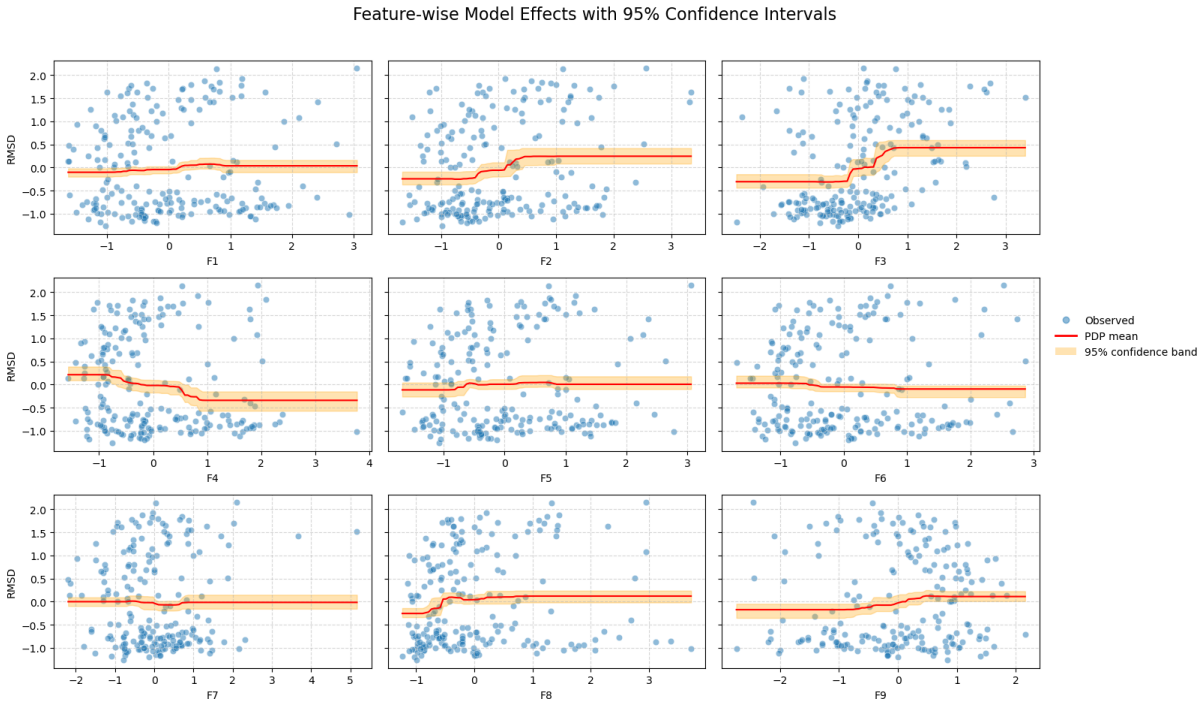

Figure 30: Feature–wise XGBoost Regression Effect with 95% confidence interval

4.3.10 LightGBM–based Regression

So far, we have reviewed three variants of GBM, i.e. AdaBoost, Gradient Boosting, and XGBoost. Although XGBoost shows more promising performance in comparision with other two models, it faces several setbacks. Still, it does not satisfy the scalability of today's data, which involves milions of samples and excessively large numbe of features. [66] To overcome this, Light Gradient Boosting Machine, short as LightGBM, was proposed, which is similar to the implementation of **XGBoost Regression** [63] [68]. However, it is superior to XGBoost, mostly due to Gradient–based One-Side Sampling (GOSS)

and Exclusive Feature Bundling (EFB), in addition with leaf–wise tree growth and histogram–based splitting algorithm. Let us briefly go through the conceptual idea of these techniques.

Typically, a DT is grown level–wise, specifically, it only grows deeper if the current level is full of terminal nodes.

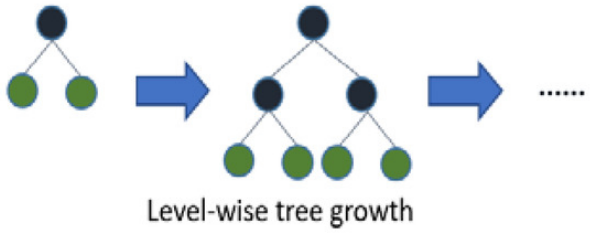

Figure 31: Level–wise Tree Growth Visualization [69]

On the other hand, leaf–wise DT [66] grows a quite abnormal tree, which possibly has great depth but small number of leaf nodes. As a result, [66] [69] it achives higher accuracy while accelerate training time but is vulnerable to overfitting.

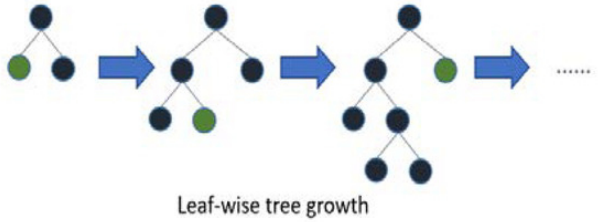

Figure 32: Leaf–wise Tree Growth Visualization [69]

The histogram–based splitting algorithm [66] is the discretization of each feature into several bins, which mitigates the overfitting phenomena but downgrades the performance.

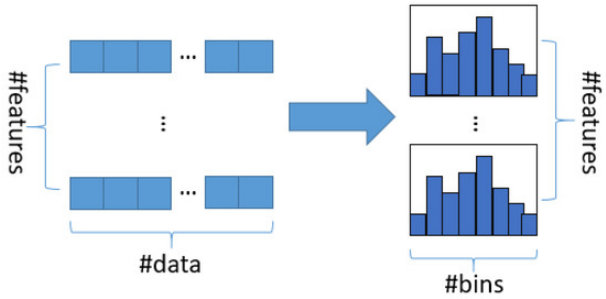

Figure 33: Histogram–based Splitting Visualization [69]

GOSS [68] was inspired from **AdaBoost Regression**, specifically the weighted output. However, GBMs do not implement weights, hence it leverages the gradients as an alternative. Simply put, samples that have small gradients imply well–trained, whereas larger gradients imply under–trained. Therefore, this technique suggests keeping samples that have large gradients and randomly select samples with smaller gradients. It is not recommended to completely exclude samples of small gradients due to major changes to data distribution, which amounts to model's precision reduction.

EFB [68] is some sort of a Dimensionality Reduction technique, it assumes that in high dimensional feature space, some dimensions are mutually exclusive. More particularly, some features are less likely to have non–zero zeros concurrently, and hence, this technique suggests aggregating/bundling them into a single feature. This is expected to significantly reduce the training time while maintaining the accuracy.

See **Table 24** for further details about the hyperparameter settings. The subplots provided below show how LightGBM Regression performs on each feature of the example dataset.

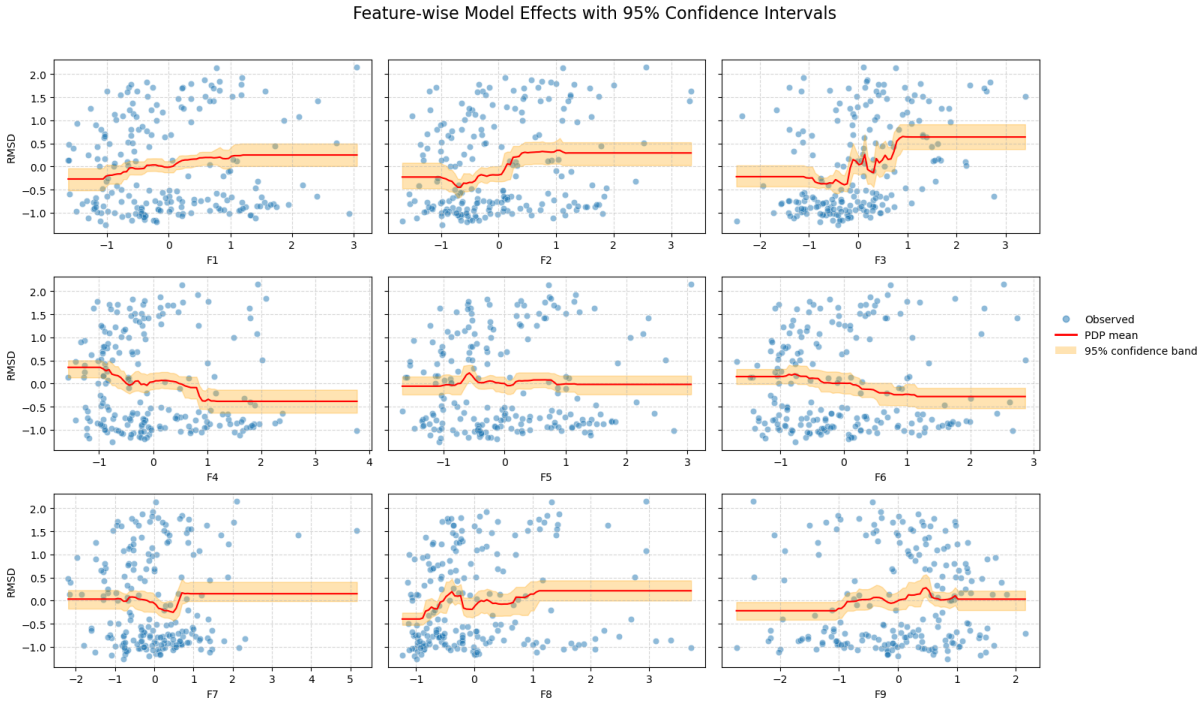

Figure 34: Feature–wise LightGBM Regression Effect with 95% confidence interval

4.3.11 MLP-like Regression model (RealMLP)

So far, we have reviewed the SOTA ML models that are commonly used in competitions and productions [45]. Now, we would like to move to axn engaged research area of AI, i.e. Deep Learning, specifically Artificial Neural Networks (ANNs) or shortly Neural Networks (NN), by reviewing three of the foundational NN–based models, i.e. Multilayer Perceptron (MLP), ResNet, and an emerging prospective architecture, Transformer. While working on this, we noted that there is an existing paper [24] shown that the tree–based models outperform the NN–based models. However, we also noticed that they benchmarked on iid datasets, while our focus is more about OOD generalization. Therefore, our experiment expands this empirical experiment, to find if it still holds for OOD data.

Let we revisit the very first implementation of ANNs, i.e. MLP models, but in a better tuned default hyperparameters version. More specifically, we utilise a NN–based regressor offered by [45], i.e. RealMLP. Since we leverage their implementation, we do not deeply study how these models are built. However, we still provide the computation graph for each architecture.

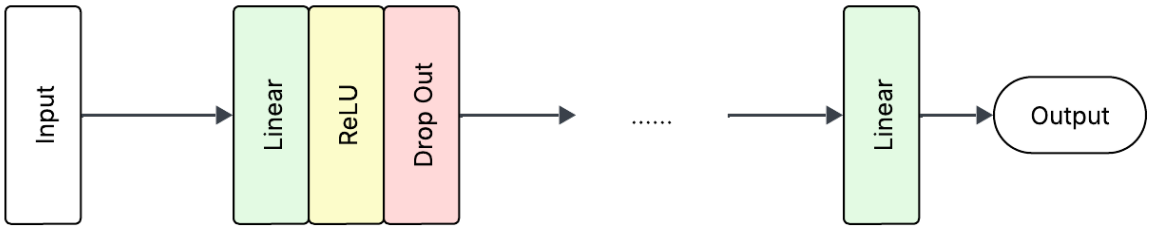

Figure 35: RealMLP architecture Visualization [45]

Let us summarize what information the graph is conveying. Firstly, the Linear layer is a set of neurons, wherein each neuron is a linear mathematical operator akin to the **Linear Regression** and trained independently. Secondly, the ReLU layer is an activation layer, while it does not consist any learnable parameters, it adds non–linearity to the predictor $\hat{f}(\cdot)$. Finally, the DropOut layer is one of regularization methods, which deactivates some of the neurons during the training process, to prevent overfitting attempts of the model.

Although we leverage the implementation of [45], we made some minor changes to hyperparameters to speed up the training and inference processes. See **Table 26** for further details about the hyperparameter settings. The subplots provided below show how RealMLP Regression performs on each feature of the example dataset.

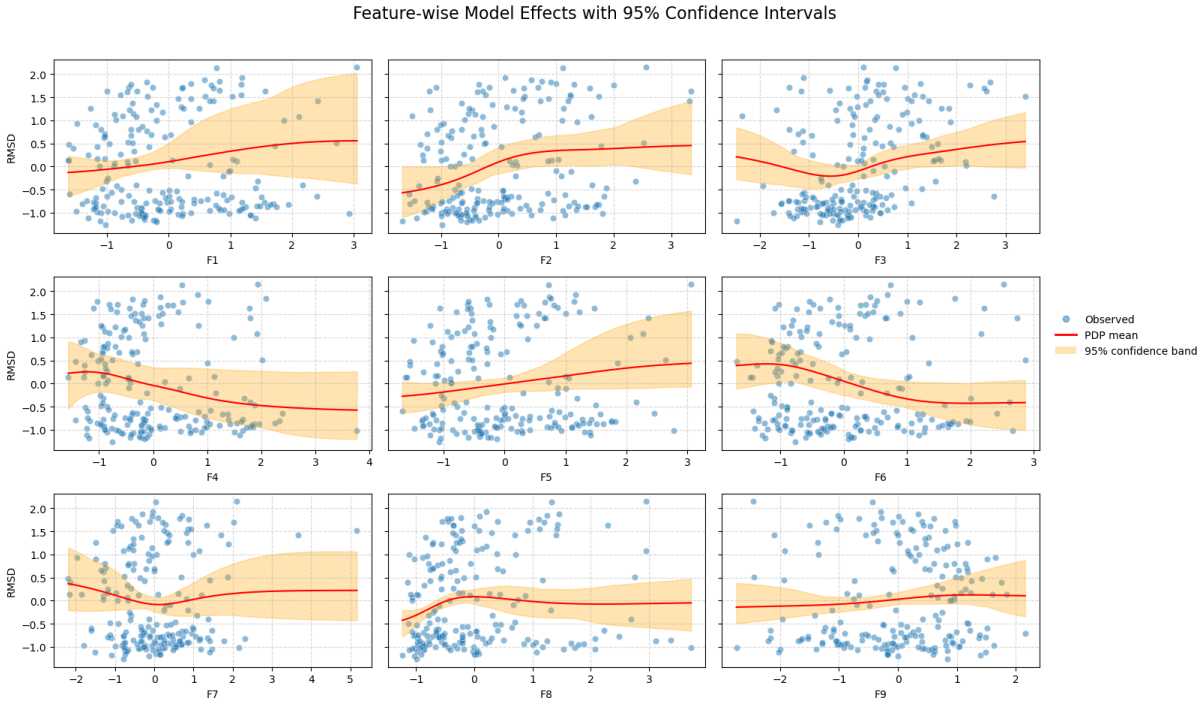

Figure 36: Feature–wise RealMLP Regression Effect with 95% confidence interval

4.3.12 ResNet Regression model

ResNet is an improvement of MLP–like models, which leveraged a normalization technique, i.e. Batch Norm [70], and the heart of the model, residual learning or shortcut connection [71]. The residual learning allows the model to forward and back–propagate using additional paths, which mitigates the chance of gradient exploding/vanishing issues

and accuracy degradation phenomena. For the implementation of ResNet, we were inspired by the architecture introduced by [45] along with certain modifications to suit the resouces constraints.

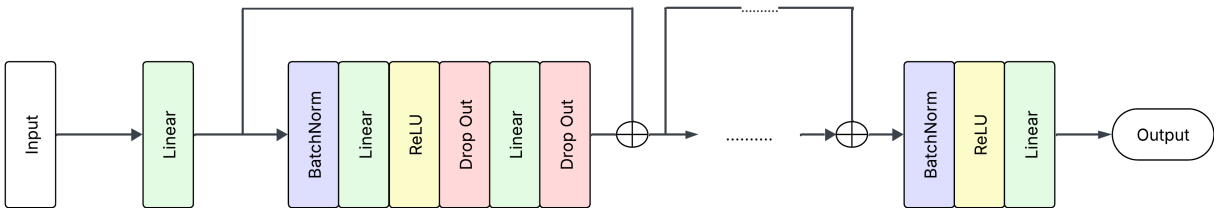

Figure 37: ResNet architecture Visualization [45]

See **Table 28** for further details about the hyperparameter settings. The subplots provided below show how ResNet Regression performs on each feature of the example dataset.

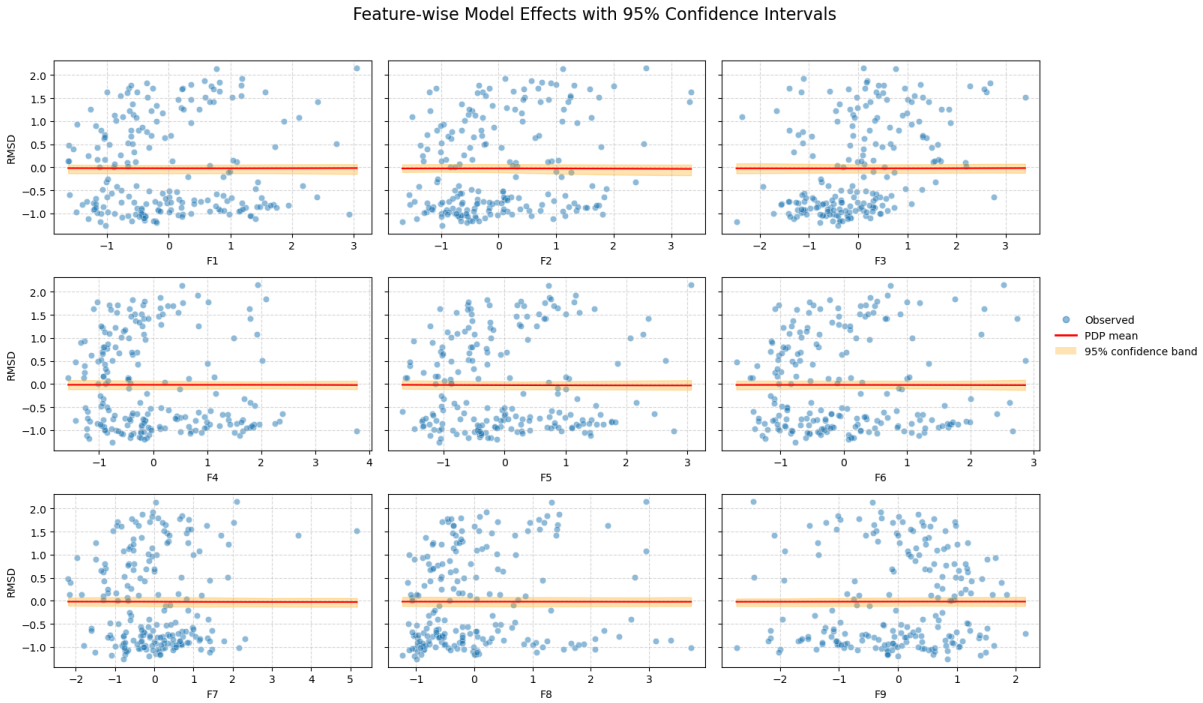

Figure 38: Feature–wise ResNet Regression Effect with 95% confidence interval

4.3.13 FT–Transformer Regression model

Finally, we adopted a Transformer–based model from [45], i.e. FT–Transformer. In short, this model firstly tokenizes the inputs into continuous embeddings, and subsequently use a simple Transformer, particularly the encoder block instead of the full encoder–decoder architecture [72]. We leverage the default hyperparameter configurations and architecture proposed by [45], except that in the context of our experiment, we do not have categorical features, thereby, no categorical embeddings involved and minor modifications to the number of layers, training epochs and batch size. Additionally, [45] provided with a well–illustrated computation graph of the model, hence, we do not include it in our report.

See **Table 30** for further details about the hyperparameter settings. The subplots provided below show how FT–Transformer Regression performs on each feature of the example dataset.

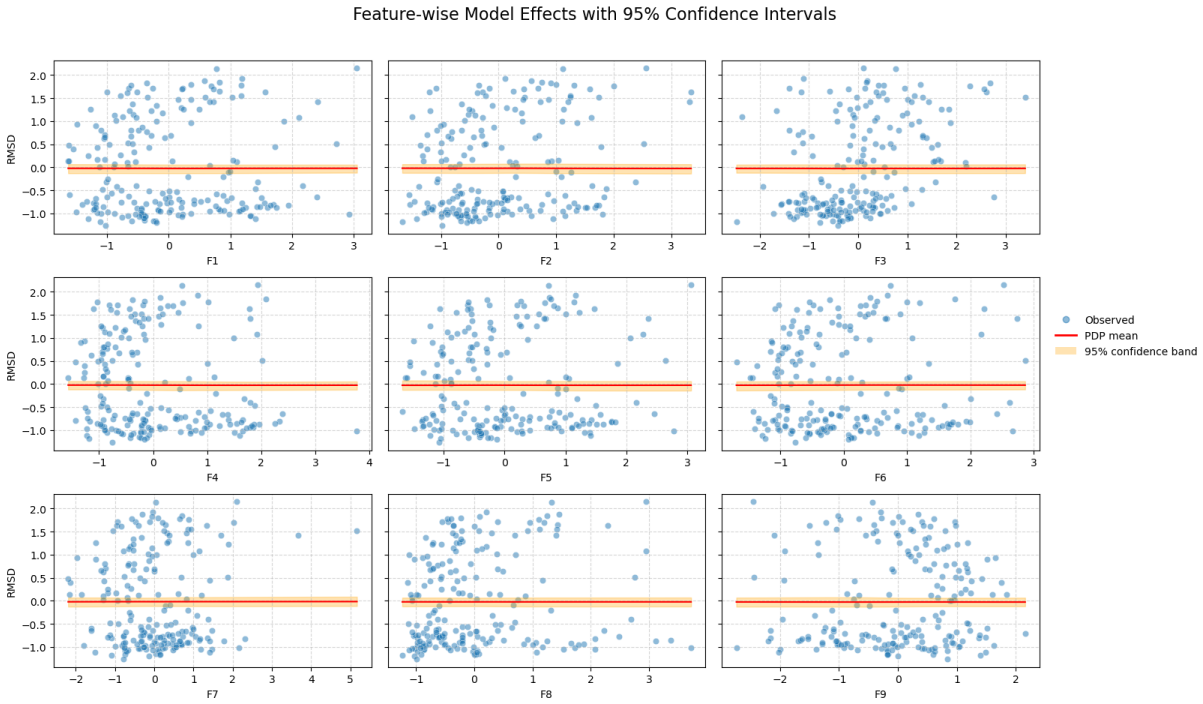

Figure 39: Feature–wise FT–Transformer Regression Effect with 95% confidence interval

5 Experimental setup

5.1 Experiment Procedure

This section is devoted to briefly convey how the experiment was conducted to obtain the results presented within this report. This testbench is some sort of semi–automation which requires practicioners manually downloading datasets and working with the Jupyter Notebooks. We believe by allowing practicioners partly involved in the procedure can improve the flexibility. More thorough guides to using this testbench is available in the README.md file in the GitHub repository provided. The procedure consists of three main phases.

1. Data Loading and Partitioning

We require practicioners downloading csv/tsv files, which are the common type of files for tabular data, these files are then converted to parquet files, a more convenient file for saving and reading actions. Then, practicioners can perform certain basic statistics summary that we already implemented, if prefer. Also, we provide tools to split the datasets and summarize some basic statistics of the partitioned data.

2. Model Training and Evaluation

Having splitted the uploaded datasets, practicioners can now train and evaluate the models for analysis. We also provide a notebook allowing plotting the training process and evaluation outcomes of the available models on any splitted dataset. For comprehensive

analysis, we provide a fully automated notebook, practicioners can modify some configurations that suit their computational resources. It is noteworthy that we apply StandardScaler() [43] to the splitted datasets before training and evaluating the models.

3. Result Summary and Analysis

To rank the models' performance across datasets and under different splitting approaches, we utilize the statistical tools recommended by [73]. In this report, we select nRMSE as our primary metric to rank the robustness of models to OOD generalization, while nMAE and sMAPE as our support metrics. More specifically, we first leverage the Friedman's test to find statistical evidence if there are model(s) that outperform others. Next, we apply Wilcoxon signed–rank test to aggregate the results (nRMSE) of the models across datasets and split regimes. If the Friedman's test shows significant statistical evidence, we subsequently perform Holm–Bonferroni post–hoc tests to compare models pairwise to find which model(s) yield better performance. Throughout this work, we are 95% confident about our results, said differently, we use significance level $\alpha = 0.05$ to conduct hypotheses tests and relevant statistics tools.

To answer our research question, i.e. which is the "best", or most robust model when generalizing to unseen data regions, we propose a **split–agnostic framework** that combines all datasets despite of splitting strategies, then follow the procedure described above. In contrast, to study if the splitting approaches impact models' performance, we propose a **split–wise framework** to aggregate the datasets with regards to the split regimes, and perform the same procedure to rank the models' performance. For simplicity, we present the outcomes of both **split–agnostic framework** and **split–wise framework** in **Section 6. Results**. Meanwhile, detailed nRMSE, nMAE, and sMAPE of each model under different datasets and split regimes are presented in the **Appendix D. Benchmark results**

Due to space limitations, the splitting techniques are summarized as indices, from #1 to #8, indicating "Random Split", "Covariates Shift", "Mfs–based", "Single Hyperball", "Multiple Hyperballs", "KMeans Hyperballs", "Single Slab", "Semi–infinite Slab", respectively. Noting that, "Random Split" specifies how the model performs on iid data, whereas, the rest indicates models' OOD extrapolation capability. The order of the datasets is preserved, as shown in **Table 1. Dataset overview**.

5.2 Evaluation metrics

Let us present our considerations about the evaluation metrics and explain why we encourage practicioners to use nRMSE as the primary performance metric, and nMAE and sMAPE as support metrics.

Given a test set of N samples as $\mathcal{D}^{\text{te}} = \left\{(\mathbf{x}_i \in \mathbb{R}^d, y_i)\right\}_1^N$ and a predictor $\hat{f}$ trained on $\mathcal{D}^{\text{tr}} = \left\{(\mathbf{x}_i \in \mathbb{R}^d, y_i)\right\}_1^M$. We consider the following evaluation metrics [50]:

1. Mean Squared Error (MSE)

$$\text{MSE} \triangleq \frac{1}{N}\sum_{i=1}^{N}\left(y_i - \hat{f}(\mathbf{x}_i)\right)^2$$

2. Root Mean Squared Error (RMSE)

$$\text{RMSE} \triangleq \sqrt{\text{MSE}}$$

3. normalized Root Mean Squared Error (nRMSE)

$$\text{nRMSE} \triangleq \frac{\text{RMSE}}{\sigma(\mathcal{D}^{\text{tr}})}$$

4. Mean Absolute Error (MAE)

$$\text{MAE} \triangleq \frac{1}{N}\sum_{i=1}^{N}\left|y_i - \hat{f}(\mathbf{x}_i)\right|$$

5. normalized Mean Absolute Error (nMAE)

$$\text{nMAE} \triangleq \frac{\text{MAE}}{\sigma(\mathcal{D}^{\text{tr}})}$$

6. Coefficient of Determination ($\mathbf{R}^2$)

$$\mathbf{R}^2 \triangleq 1 - \frac{\sum_{i=1}^{N}\left(y_i - \hat{f}(\mathbf{x}_i)\right)^2}{\sum_{i=1}^{N}\left(y_i - \mu(\mathcal{D}^{\text{te}})\right)^2}$$

7. Adjusted $\mathbf{R}^2$

$$\text{Adjusted } \mathbf{R}^2 \triangleq 1 - (1 - \mathbf{R}^2) \times \frac{N-1}{N-d-1}$$

8. Mean Absolute Percentage Error (MAPE) [43]

$$\text{MAPE} \triangleq \frac{1}{N}\sum_{i=1}^{N}\left(\frac{\left|y_i - \hat{f}(\mathbf{x}_i)\right|}{\max\{y_i, \epsilon\}}\right)$$

   where ϵ is a extremely small number to avoid "division by zero" error.

9. Symmetric Mean Absolute Percentage Error (sMAPE)

$$\text{sMAPE} \triangleq \frac{1}{N}\sum_{i=1}^{N}\frac{\left|y_i - \hat{f}(\mathbf{x}_i)\right|}{\left(|y_i| + \left|\hat{f}(\mathbf{x}_i)\right|\right)/2} \times 100$$

where $\sigma(\cdot)$ indicates the standard deviation and $\mu(\cdot)$ indicates the mean.

To begin with, **MSE** is not a proper metric due to its sensitivity to outliers, which are inevitable in the domain of OOD generalization. **RMSE** is better as its high interpretability and tolerance to outliers at some extent; however, it is still a scale–dependent metric, which turns out to be inappropriate to compare across datasets of different sizes and units of measurement. A better choice is **nRMSE**, which normalizes the RMSE priorly introduced, this preserves the penalty to outliers while evaluating if a model performs better than using the mean, which is more proper to compare across different datasets. Another metric is **MAE**, more robust to outliers than MSE/RMSE, but still scale–dependent. A

similar solution is the utilization of **nMAE**, which is also our support metric. Both $\mathbf{R}^2$ and **Adjusted** $\mathbf{R}^2$ are inappropriate due to their assumptions about the data, i.e. linear relationship between covariates and targets. Thereby, it cannot capture the performance of models on non–linear datasets. Next, **MAPE** is relatively good metric owing to its scale–independence and interpretability. Nevertheless, it is vulnerable to outliers and near–zero values, additionally, favours over–forecasting, i.e. penalizes $\hat{f}(\mathbf{x}_i) > y_i$ less than $\hat{f}(\mathbf{x}_i) < y_i$, which is unfair. A better scale–independent metric is **sMAPE**, which gets rid of this asymmetry property of MAPE, and diminishes the vulnerability to outliers and near–zero values.

In brief, we solely report three performance metrics, i.e. nRMSE, nMAE, and sMAPE. More specifically, we will rank the models based on nRMSE (main indicator) and use other two as support metrics.

5.3 Sided experiment

Aside from the main experiment, we are curious about the geometric partitioning approaches. Currently, we divide the original datasets, such that it follows the notions of **Definition 7. Interpolation vs. Extrapolation with Convex Hull** proposed earlier. In this sided experiment, we conduct a similar work flow, despite that, we do not follow the notions above. For example, consider **Algorithm 2. Single Hyperball Split**, we use the inner points of the hyperball as the test data and the outer points as train data, similar modifications are made for other geometry–based algorithms.

6 Results

For tables within this section, for each splitting strategy, the ranks which are computed using nRMSE, are bolded in red if they are the lowest mean ranks, temporarily called "best candidates" under the context of this report. For those statistically, by Holm–corrected Wilcoxon signed–ranks test, tie with the "best candidates" are bolded in blue.

Table 2: Across-dataset split–agnostic performance of all models. Lower mean ranks indicate better robustness across datasets regardless of split regimes.

Models	**Mean–rank**
LinearRegressor	8.967033
PolynomialRegressor	9.752747
KNNRegressor	9.631868
SVMRegressor	10.329670
DTRegressor	9.857143
RFRegressor	6.543956
GBRegressor	4.956044
ABRegressor	6.565934
XGBRegressor	5.219780
LightGBMRegressor	4.675824
RealMLPRegressor	4.351648
ResnetRegressor	5.203297

Models	Mean–rank
FTTransformerRegressor	4.945055
Friedman χ^2	782.8004
Significance	*
Significance codes: * $p \lll 0.001$	

According to **Table 2**, the "best candidate" is **RealMLP** [46], which is an ANN regressor. However, it is statistical insignificant that it outperforms all models presented in our experiment. In particular, **GBRegressor**, **ResnetRegressor**, and **FTTransformerRegressor** are competitively good. A remarkable point is that of the top four models, only **GBRegressor** is a tree–based model. This possibly implies optimistic signs about the future of NN–based models in OOD generalization. Indeed, the experiment is yet reliable due to various factors, e.g. hyperparameters and datasets' characteristics ignorance. Hence, we can only present the clear performance gap between ANNs and some simpler models, not between complex NN– and tree–based models.

Next, to explore if the performance of models depends on split regimes, we computed the mean ranks of models across datasets with regards to split regimes, as shown in **Table 3**.

Table 3: Across-dataset split–wise performance of all models. Lower mean ranks indicate better robustness across datasets w.r.t split regimes.

Models	#1	#2	#3	#4	#5	#6	#7	#8
LinearRegressor	11.08	8.96	9.38	8.81	8.92	9.00	9.00	8.69
PolynomialRegressor	10.15	8.65	9.62	10.46	8.35	9.42	11.00	10.77
KNNRegressor	7.42	10.12	9.42	9.08	10.31	9.73	9.38	9.38
SVMRegressor	12.31	10.04	10.92	10.23	9.96	10.31	10.69	10.15
DTRegressor	9.73	10.69	10.31	8.81	10.69	10.00	9.23	9.27
RFRegressor	6.73	6.92	6.88	6.08	6.88	6.81	6.19	6.04
GBRegressor	5.04	5.12	4.81	4.69	5.35	5.38	4.31	5.04
ABRegressor	6.19	7.12	6.73	6.23	6.85	6.42	6.08	6.54
XGBRegressor	5.00	5.73	5.00	5.27	5.69	5.27	4.58	5.00
LightGBMRegressor	2.92	4.85	4.54	5.00	4.23	5.04	4.35	4.73
RealMLPRegressor	2.81	3.96	3.46	5.23	4.08	4.04	4.88	4.81
ResnetRegressor	4.85	3.35	4.81	6.58	4.00	4.96	6.50	6.23
FTTransformerRegressor	6.77	5.50	5.12	4.54	5.69	4.62	4.81	4.35
Friedman χ^2	188.3	126.5	138.1	96.7	120.5	112.6	129.4	110.5
Significance	*	*	*	*	*	*	*	*
Significance codes: * $p \lll 0.001$								

Intriguingly, the results show that, although **GBRegressor** is a competitive candidate in our experiment, it only shows superior performance under **Single Slab** splitting method. Meanwhile, other three NN–based models have demonstrated theirs excellence in OOD extrapolation under other split regimes. This may imply ANNs perform well on more diverse train/test data distributions in comparision with tree–based models.

7 Analysis

8 Conclusion

Conclusion. From the experiment, we conclude that ANNs yet to completely outperform tree–based models in OOD extrapolation. More specifically, it is statistically insignificant to have any presented models as the "best model" in the context of unseen data generalization. Nonetheless, due to statistical tests, the top models are GradientBoost, MLP, ResNet, and FT–Transformer. Apparently, this group contains one tree–based model and three NN–base models. This may imply that the prospective future of ANNs under different train/test distribution shifts. Noticeably, this result does not contradict with the results presented in [24], as [24] evaluates models under iid data, whereas in our experiment, we evaluate models under iid and distributionally shifted data.

Limitations. We have identified three major obstacles which highly impact the reliability of the experiment's results. Firstly, limited computational resources pose big issues to fitting models, and tuning hyperparameter settings, which leads to arbitrarily poor performance of models. Secondly, we lack of domain knowledge and feature engineering to the used datasets, which amount to poor performance of datasets [74]. Finally, we do not conduct optimal hyperparameter searching, alternatively, we heuristically "guess", use the default hyperparameters along with certain suggestions from ChatGPT [75]. Despite these constraints, it only affects the comparision of performance among the top models, e.g. **LightGBM Regressor**, **RealMLP Regressor**. It does not affect the difference between, e.g. **Linear Regressor** or **SVM Regressor** vs. **LightGBM Regressor**, due to clear performance gap.

References

[1] P. P. Shinde and S. Shah, "A review of machine learning and deep learning applications," in *2018 Fourth International Conference on Computing Communication Control and Automation (ICCUBEA)*, 2018.

[2] C. Janiesch, P. Zschech, and K. Heinrich, "Machine learning and deep learning," *Electronic Markets*, vol. 31, p. 685–695, Apr. 2021.

[3] Y. LeCun, Y. Bengio, and G. Hinton, "Deep learning," *Nature*, vol. 521, no. 7553, pp. 436–444, 2015.

[4] N. Rane, S. K. Mallick, O. Kaya, and J. Rane, "Applications of deep learning in healthcare, finance, agriculture, retail, energy, manufacturing, and transportation: A review," in *Applied Machine Learning and Deep Learning: Architectures and Techniques*, ch. 7, pp. 132–152, Deep Science Publishing, October 2024.

[5] M. I. Jordan and T. M. Mitchell, "Machine learning: Trends, perspectives, and prospects," *Science*, vol. 349, no. 6245, pp. 255–260, 2015.

[6] S. J. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*. Hoboken, NJ: Pearson, 4 ed., 2021.

[7] A. Malinin, N. Band, G. Chesnokov, Y. Gal, M. Gales, A. Noskov, A. Ploskonosov, L. Ostroumova Prokhorenkova, I. Provilkov, V. Raina, V. Raina, M. Shmatova, P. Tigas, and B. Yangel, "Shifts: A dataset of real distributional shift across multiple large-scale tasks," *Science*, 07 2021.

[8] L. Tamang, M. R. Bouadjenek, R. Dazeley, and S. Aryal, "Handling out-of-distribution data: A survey," 2025.

[9] J. Quiñonero-Candela, M. Sugiyama, A. Schwaighofer, and N. D. Lawrence, *Dataset Shift in Machine Learning*. Cambridge, MA: The MIT Press, 2009.

[10] P. W. Koh, S. Sagawa, H. Marklund, S. M. Xie, M. Zhang, A. Balsubramani, W. Hu, M. Yasunaga, R. Lanas Phillips, S. Beery, J. Leskovec, A. Kundaje, E. Pierson, S. Levine, C. Finn, and P. Liang, "Wilds: A benchmark of in-the-wild distribution shifts," *arXiv preprint arXiv:2012.07421*, 2020.

[11] I. Gulrajani and D. Lopez-Paz, "In search of lost domain generalization," 2020.

[12] I. Deeva, N. Amerkhanova, and A. Kropacheva, "Evaluating robustness of tabular models under meta-features based shifts," *arXiv preprint*, 2023. AI Institute, ITMO University.

[13] A. Malinin, A. Athanasopoulos, M. Barakovic, M. B. Cuadra, M. J. F. Gales, C. Granziera, M. Graziani, N. Kartashev, K. Kyriakopoulos, P.-J. Lu, N. Molchanova, A. Nikitakis, V. Raina, F. L. Rosa, E. Sivena, V. Tsarsitalidis, E. Tsompopoulou, and E. Volf, "Shifts 2.0: Extending the dataset of real distributional shifts," 2022.

[14] R. Taori, A. Dave, V. Shankar, N. Carlini, B. Recht, and L. Schmidt, "Measuring robustness to natural distribution shifts in image classification," 2020.

[15] A. Barbu, D. Mayo, J. Alverio, W. Luo, C. Wang, D. Gutfreund, J. Tenenbaum, and B. Katz, "Objectnet: A large-scale bias-controlled dataset for pushing the limits of object recognition models," in *Advances in Neural Information Processing Systems* (H. Wallach, H. Larochelle, A. Beygelzimer, F. d'Alché-Buc, E. Fox, and R. Garnett, eds.), vol. 32, Curran Associates, Inc., 2019.

[16] M. T. Ribeiro, T. Wu, C. Guestrin, and S. Singh, "Beyond accuracy: Behavioral testing of NLP models with CheckList," in *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics* (D. Jurafsky, J. Chai, N. Schluter, and J. Tetreault, eds.), (Online), pp. 4902–4912, Association for Computational Linguistics, July 2020.

[17] S. Kolesnikov, "Wild-tab: A benchmark for out-of-distribution generalization in tabular regression," 2023.

[18] V. Borisov, T. Leemann, K. Seßler, J. Haug, M. Pawelczyk, and G. Kasneci, "Deep neural networks and tabular data: A survey," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 35, p. 7499–7519, June 2024.

[19] J. B. Heaton, N. G. Polson, and J. H. Witte, "Deep learning in finance," *arXiv preprint arXiv:1602.06561*, 2018.

[20] A. M. Ozbayoglu, M. U. Gudelek, and O. B. Sezer, "Deep learning for financial applications : A survey," 2020.

[21] A. Nayyar, L. Gadhavi, and N. Zaman, "Chapter 2 - machine learning in healthcare: review, opportunities and challenges," in *Machine Learning and the Internet of Medical Things in Healthcare* (K. K. Singh, M. Elhoseny, A. Singh, and A. A. Elngar, eds.), pp. 23–45, Academic Press, 2021.

[22] A. Rajkomar, E. Oren, K. Chen, A. M. Dai, N. Hajaj, M. Hardt, P. J. Liu, X. Liu, J. Marcus, M. Sun, P. Sundberg, H. Yee, K. Zhang, Y. Zhang, G. Flores, G. E. Duggan, J. Irvine, Q. V. Le, K. Litsch, A. Mossin, J. Tansuwan, D. Wang, J. Wexler, J. Wilson, D. Ludwig, S. L. Volchenboum, K. Chou, M. Pearson, S. Madabushi, N. H. Shah, A. J. Butte, M. D. Howell, C. Cui, G. S. Corrado, and J. Dean, "Scalable and accurate deep learning with electronic health records," *NPJ Digital Medicine*, vol. 1, 2018.

[23] J. Leukel, J. González, and M. Riekert, "Adoption of machine learning technology for failure prediction in industrial maintenance: A systematic review," *Journal of Manufacturing Systems*, vol. 61, pp. 87–96, 2021.

[24] L. Grinsztajn, E. Oyallon, and G. Varoquaux, "Why do tree-based models still outperform deep learning on tabular data?," 2022.

[25] R. Balestriero, J. Pesenti, and Y. LeCun, "Learning in high dimension always amounts to extrapolation," 2021.

[26] S. Zhang, Y. Luo, Q. Wang, H. Chi, W. Li, B. Han, and J. Li, "Are all unseen data out-of-distribution?," *CoRR*, vol. abs/2312.16243, 2023.

[27] H. Hwang and J. Shin, "Uncertainty measurement of deep learning system based on the convex hull of training sets," 2024.

[28] S. Boyd and L. Vandenberghe, *Convex Optimization*. Cambridge University Press, 2004.

[29] J. Vanschoren, J. N. van Rijn, B. Bischl, and L. Torgo, "Openml: networked science in machine learning," *ACM SIGKDD Explorations Newsletter*, vol. 15, p. 49–60, June 2014.

[30] J. Mendes-Moreira, C. Soares, A. M. Jorge, and J. F. de Sousa, "Ensemble approaches for regression: A survey," *ACM Computing Surveys*, vol. 45, pp. 10:1–10:40, 11 2012.

[31] J. Liu, Z. Shen, Y. He, X. Zhang, R. Xu, H. Yu, and P. Cui, "Towards out-of-distribution generalization: A survey," 2023.

[32] J. G. Moreno-Torres, T. Raeder, R. Alaiz-Rodríguez, N. V. Chawla, and F. Herrera, "A unifying view on dataset shift in classification," *Pattern Recognition*, vol. 45, no. 1, pp. 521–530, 2012.

[33] A. Nemirko and J. Dulá, "Machine learning algorithm based on convex hull analysis," *Procedia Computer Science*, vol. 186, pp. 381–386, 2021. 14th International Symposium "Intelligent Systems.

[34] U. Itai, A. B. Ilan, and T. Lazebnik, "Tighten the lasso: A convex hull volume-based anomaly detection method," 2025.

[35] N. Altman and M. Krzywinski, "The curse(s) of dimensionality," *Nature Methods*, vol. 15, pp. 399–400, 2018.

[36] D. Peng, Z. Gui, and H. Wu, "Interpreting the curse of dimensionality from distance concentration and manifold effect," 2025.

[37] A. Rivolli, L. P. Garcia, C. Soares, J. Vanschoren, and A. C. de Carvalho, "Meta-features for meta-learning," *Knowledge-Based Systems*, vol. 240, p. 108101, 2022.

[38] E. Alcobaça, F. Siqueira, A. Rivolli, L. P. F. Garcia, J. T. Oliva, and A. C. P. L. F. de Carvalho, "Mfe: Towards reproducible meta-feature extraction," *Journal of Machine Learning Research*, vol. 21, no. 111, pp. 1–5, 2020.

[39] R. Mussabayev, "Optimizing euclidean distance computation," *Mathematics*, vol. 12, no. 23, 2024.

[40] W. K. Nicholson, *Linear Algebra with Applications*. Calgary, Alberta, Canada: Lyryx Learning Inc., 2021a ed., Dec. 2020.

[41] A. M. Ikotun, A. E. Ezugwu, L. Abualigah, B. Abuhaija, and J. Heming, "K-means clustering algorithms: A comprehensive review, variants analysis, and advances in the era of big data," *Information Sciences*, vol. 622, pp. 178–210, 2023.

[42] A. Srivastava and M. Nawfal, "Parallelization of the k-means algorithm with applications to big data clustering," 2024.

[43] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and É. Duchesnay, "Scikit-learn: Machine learning in python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[44] J. B. Rawlings, "Diffusion and heat transfer," 2014. Copyright by James B. Rawlings.

[45] Y. Gorishniy, I. Rubachev, V. Khrulkov, and A. Babenko, "Revisiting deep learning models for tabular data," 2023.

[46] D. Holzmüller, L. Grinsztajn, and I. Steinwart, "Better by default: Strong pre-tuned mlps and boosted trees on tabular data," 2025.

[47] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, "Optuna: A next-generation hyperparameter optimization framework," in *The 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, pp. 2623–2631, 2019.

[48] A. E. Hoerl and R. W. Kennard, "Ridge regression: Biased estimation for nonorthogonal problems," *Technometrics*, vol. 42, no. 1, pp. 80–86, 2000.

[49] R. Tibshirani, "Regression shrinkage and selection via the lasso," *Journal of the Royal Statistical Society. Series B (Methodological)*, vol. 58, no. 1, pp. 267–288, 1996.

[50] J. Terven, D.-M. Cordova-Esparza, J.-A. Romero-González, A. Ramírez-Pedraza, and E. A. Chávez-Urbiola, "A comprehensive survey of loss functions and metrics in deep learning," *Artificial Intelligence Review*, vol. 58, Apr. 2025.

[51] P. J. Huber, "Robust Estimation of a Location Parameter," *The Annals of Mathematical Statistics*, vol. 35, no. 1, pp. 73 – 101, 1964.

[52] J. Kukačka, V. Golkov, and D. Cremers, "Regularization for deep learning: A taxonomy," 2017.

[53] H. Zou and T. Hastie, "Regularization and variable selection via the elastic net," *Journal of the Royal Statistical Society Series B: Statistical Methodology*, vol. 67, pp. 301–320, 03 2005.

[54] Y. D. Castro, F. Gamboa, D. Henrion, R. Hess, and J.-B. Lasserre, "Approximate optimal designs for multivariate polynomial regression," 2017.

[55] O. Kramer, "Unsupervised k-nearest neighbor regression," 2011.

[56] P. Cunningham and S. J. Delany, "k-nearest neighbour classifiers - a tutorial," *ACM Computing Surveys*, vol. 54, p. 1–25, July 2021.

[57] A. J. Smola and B. Schölkopf, "A tutorial on support vector regression," *Statistics and Computing*, vol. 14, no. 3, pp. 199–222, 2004.

[58] C. M. Bishop, *Pattern Recognition and Machine Learning*. New York: Springer, 2006.

[59] T. Hastie, R. Tibshirani, and J. Friedman, *The elements of statistical learning: data mining, inference and prediction*. Springer, 2 ed., 2009.

[60] J. KlemelÃ¤, S. Klinke, and H. Sofyan, *Classification and Regression Trees*. Humboldt-UniversitÃ¤t zu Berlin, Wirtschaftswissenschaftliche FakultÃ¤t, 2005.

[61] G. James, D. Witten, T. Hastie, and R. Tibshirani, *An Introduction to Statistical Learning: with Applications in R*. Springer, 2013.

[62] G. Louppe, "Understanding random forests: From theory to practice," 2015.

[63] I. D. Mienye and Y. Sun, "A survey of ensemble learning: Concepts, algorithms, applications, and prospects," *IEEE Access*, vol. 10, pp. 99129–99149, 2022.

[64] H. Drucker, "Improving regressors using boosting techniques," in *International Conference on Machine Learning*, 1997.

[65] J. Friedman, "Greedy function approximation: A gradient boosting machine," *The Annals of Statistics*, vol. 29, 11 2000.

[66] P. Florek and A. Zagdański, "Benchmarking state-of-the-art gradient boosting algorithms for classification," 2023.

[67] T. Chen and C. Guestrin, "Xgboost: A scalable tree boosting system," in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, KDD '16, p. 785–794, ACM, Aug. 2016.

[68] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu, "Lightgbm: A highly efficient gradient boosting decision tree," in *Neural Information Processing Systems*, 2017.

[69] Y. Wang and T. Wang, "Application of improved lightgbm model in blood glucose prediction," *Applied Sciences*, vol. 10, no. 9, 2020.

[70] S. Ioffe and C. Szegedy, "Batch normalization: Accelerating deep network training by reducing internal covariate shift," 2015.

[71] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," 2015.

[72] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. u. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems* (I. Guyon, U. V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, eds.), vol. 30, Curran Associates, Inc., 2017.

[73] J. Demšar, "Statistical comparisons of classifiers over multiple data sets," *Journal of Machine Learning Research*, vol. 7, no. 1, pp. 1–30, 2006.

[74] F. M. Rohrhofer, S. Saha, S. D. Cataldo, B. C. Geiger, W. von der Linden, and L. Boeri, "Importance of feature engineering and database selection in a machine learning model: A case study on carbon crystal structures," 2021.

[75] OpenAI, "Chatgpt (gpt-5.2 model)." https://chat.openai.com/, 2025.

[76] A. Malinin, A. Athanasopoulos, M. Barakovic, M. Bach Cuadra, M. Gales, C. Granziera, M. Graziani, N. Kartashev, K. Kyriakopoulos, P.-J. Lu, N. Molchanova, A. Nikitakis, V. Raina, F. La Rosa, E. Sivena, V. Tsarsitalidis, E. Tsompopoulou, and E. Volf, "Shifts marine cargo vessel power consumption prediction dataset," Sept. 2022.

[77] Z. Boros and A. Szaz, "Infimum and supremum completeness properties of ordered sets without axioms," *An. Ştiinţ. Univ. Ovidius Constanţa Ser. Mat.*, vol. 16, pp. 31–37, 01 2008.

Appendices

A Notation and Abbreviation Reference List

A.1 Abbreviation

Acronyms	Meaning
tr	train
te	test
emp	empirical
vs.	versus
iid	independent and identically distributed
w.r.t	with respect to
SOTA	State–Of–The–Art
deg	degree
ID	In-Distribution
OOD	Out-of-Distribution
PDP	Partial Dependence Plot
SSE	Sum of Squared Error
MSE	Mean Squared Error
MAE	Mean Absolute Error
nMAE	normalized Mean Absolute Error
RMSE	Root Mean Squared Error
nRMSE	normalized Root Mean Squared Error
MAPE	Mean Absolute Percentage Error
sMAPE	symmetric Mean Absolute Percentage Error
KNN	K–Neareast Neighbour
SVM	Support Vector Machine
SVR	Support Vector Regression
DT	Decision Tree
RF	Random Forest
GB	Gradient Boosting
GBM	Gradient Boosting Machine
GAM	Generalized Additive Model
GOSS	Gradient–based One–Side Sampling
EFB	Exclusive Feature Bundling
NN	Neural Network
MLP	Multi-Layer Perceptrons

A.2 Notation

Symbol	Meaning
$\mathcal{R}(\cdot)$	Risk
$\mathcal{X}$	Input/Feature space or Domain
$\mathcal{Y}$	Output/Label space or Codomain
$\lvert\mathcal{X}\rvert$	Size, or number of elements of set $\mathcal{X}$

Symbol	Meaning
$\mathbf{X}^{n\times m}$	Matrix of Covariates/Features of m features and n samples
$\mathbf{y}^{n\times 1}$	Target vector of n outputs
$\mathbf{R}^2$	Coefficient of Determination
$\mathbb{E}$	Expected Value
$\mathbf{P}(\mathbf{X},\mathbf{y})$	Joint Distribution
$\mathbf{P}(\mathbf{X}),\ \mathbf{P}(\mathbf{y})$	Marginal Distribution
$\mathbf{P}(\mathbf{y}\mid\mathbf{X})$	Conditional Distribution
$(\mathbf{x}_i, y_i)\sim\mathbf{P}(\cdot)$	A data point sampled from a distribution $\mathbf{P}(\cdot)$
$f(\mathbf{x}_i)\mapsto y_i$	True mapping from $\mathbf{x}_i\in\mathbf{X}$ onto $y_i\in\mathbf{y}$
$\hat{y}_i=\hat{f}(\cdot)\approx y_i$	An approximation mapping/function
$\ell(f(\mathbf{x}_i), y_i)\mapsto\mathbb{R}_{\geq 0}$	loss of a pair of input and true output $(\mathbf{x}_i, y_i)$
$\nabla_{\mathbf{x}} f$	gradient of f w.r.t $\mathbf{x}$
$\partial_x f$	partial derivative of f w.r.t x
$\mathrm{Conv}(\cdot)$	Convex hull
$\lVert\cdot\rVert_p\triangleq(\sum\lvert\cdot\rvert^p)^{\frac{1}{p}}$	ℓ_p–norm
$\overline{a,b}$	$\{a, a+1,\ldots,b-1,b\}$
$\lfloor\cdot\rfloor,\lceil\cdot\rceil$	approximate the nearest smaller/larger integer
$\mathbf{R}(\cdot)$	Regularization/Penalty term
$\alpha(\cdot;\cdot)$	multi–index
$\binom{n}{k}\triangleq\frac{n!}{k!(n-k)!}$	Combinations, choose k out of n without replacement and order not considered
$\inf\{\cdot\},\sup\{\cdot\}$	infimum and supremum [77]

B Supportive algorithms

This section aims to providing additional functions that support the core algorithms presented within this report.

Algorithm 7: **Covariate Distribution Based Selection**

Require: Dataset $\mathcal{D}=(\mathbf{X},\mathbf{y})$ with features $\{\mathbf{x}_1,\ldots,\mathbf{x}_m\}$ and target $\mathbf{y}$; test ratio $\tau\in(0,1)$; feat: selected feature to split

Ensure: *mask*: selected samples

$lo\leftarrow 0$; $hi\leftarrow 1$; $\varepsilon\leftarrow 10^{-3}$
while $lo<hi$ **do**
  $mid\leftarrow lo+\dfrac{hi-lo}{2}$
  $q_{low}\leftarrow Q_{\text{feat}}(mid)$; $q_{high}\leftarrow Q_{\text{feat}}(1-mid)$
  $mask\leftarrow\mathbb{1}\{\textsc{Digitize}(\mathbf{X}_{\text{feat}}; q_{low}, q_{high})=1\}$
  if $\dfrac{\sum mask}{|\mathcal{D}|}<\tau$ **then**
    $lo\leftarrow mid+\varepsilon$
  else
    $hi\leftarrow mid-\varepsilon$
  end if
end while

return $mask$

Algorithm 8: BallSelection **(binary search on radius)**

Require: $\mathbf{X} \in \mathbb{R}^{n \times d}$, center $\mathbf{c} \in \mathbb{R}^d$, train ratio α
Ensure: $I \subseteq \{1, \ldots, n\}$, indices of points inside the selected ball

$\varepsilon \leftarrow 10^{-3}$; $lo \leftarrow 0$
$high \leftarrow \max_{i \in \{1,\ldots,n\}} \|\mathbf{X}_i - \mathbf{c}\|_2$
while $lo < high$ **do**
    $mid \leftarrow lo + \dfrac{high - lo}{2}$
    $I \leftarrow \{\, i \in \overline{1,n} : \|\mathbf{X}_i - \mathbf{c}\|_2 \le mid \,\}$
    if $\dfrac{|I|}{n} < \alpha$ **then**
        $lo \leftarrow mid + \varepsilon$
    else
        $high \leftarrow mid - \varepsilon$
    end if
end while
return I

Algorithm 9: RandomSums **(random partition of a total fraction)**

Require: Total fraction τ, number of balls B
Ensure: $(\tau_1, \ldots, \tau_B)$ such that $\sum_{b=1}^{B} \tau_b = \tau$

if $B = 1$ **then**
    return (τ)
end if
Generate $B - 1$ i.i.d. cuts $u_1, \ldots, u_{B-1} \sim \mathsf{Uniform}(0, \tau)$
Sort cuts to get $u_{(1)} \le \cdots \le u_{(B-1)}$
$\tau_1 \leftarrow u_{(1)}$
for $b = 2$ to $B - 1$ **do**
    $\tau_b \leftarrow u_{(b)} - u_{(b-1)}$
end for
$\tau_B \leftarrow \tau - u_{(B-1)}$
return $(\tau_1, \ldots, \tau_B)$

Algorithm 10: **Hyperplane Construction** (ConstructHyperplane)

Require: $\mathbf{X} \in \mathbb{R}^{n \times d}$
Ensure: Normal vector $\mathbf{n}$, offset b, and a point $\mathbf{p}_0$ on the hyperplane

$u \leftarrow \textsc{RandInt}(1, n)$
$\mathbf{p}_0 \leftarrow \mathbf{X}_u$
$\mathbf{n} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}_d)$
$b \leftarrow \mathbf{n}^\intercal \mathbf{p}_0$
return $(\mathbf{n}, b, \mathbf{p}_0)$

Algorithm 11: **One-Side Selection** (DataOneSide)

Require: $\mathbf{X} \in \mathbb{R}^{n \times d}$, normal vector $\mathbf{n}$, offset $b \in \mathbb{R}$
Ensure: Index set $I = \{\, i \in \overline{1,n} : \mathbf{X}_i^\top \mathbf{n} < b \,\}$

$I \leftarrow \{\, i \in \overline{1,n} : \mathbf{X}_i^\top \mathbf{n} - b < 0 \,\}$
return I

Algorithm 12: **Find Bounds for Semi-Infinite Slab** (FindBounds)

Require: $\mathbf{X} \in \mathbb{R}^{n \times d}$, test ratio $\tau \in (0, 1)$
Ensure: Normal vector $\mathbf{n}$, lower/upper bounding points $(\mathbf{lo}, \mathbf{hi})$

$(\mathbf{n}, b, \mathbf{p}_0) \leftarrow$ ConstructHyperplane($\mathbf{X}$)
$dist \leftarrow \dfrac{|\mathbf{X}^\intercal \mathbf{n} - b|}{\|\mathbf{n}\|_2}$

$mask \leftarrow (\mathbf{X}^\intercal \mathbf{n} - b < 0)$
$L \leftarrow \{\, i \in \overline{1,n} : mask_i = \text{True} \,\}$; $H \leftarrow \{\, i \in \overline{1,n} : mask_i = \text{False} \,\}$
▷ Pick farthest points on each side; fallback if one side is empty
if $L \neq \emptyset$ **and** $H \neq \emptyset$ **then**
    $i_{lo} \leftarrow \arg\max_{i \in L} dist_i$; $i_{hi} \leftarrow \arg\max_{i \in H} dist_i$
else
    $i_{lo} \leftarrow \arg\min_{i \in \overline{1,n}} dist_i$; $i_{hi} \leftarrow \arg\max_{i \in \overline{1,n}} dist_i$
end if
$\mathbf{lo} \leftarrow \mathbf{X}_{i_{lo}}$; $\mathbf{hi} \leftarrow \mathbf{X}_{i_{hi}}$

$b_{hi} \leftarrow \mathbf{n}^\intercal \mathbf{hi}$
$I_{\text{test}} \leftarrow$ DataOneSide($\mathbf{X}, \mathbf{n}, b_{hi}$)
if $\dfrac{|I_{\text{test}}|}{n} < \tau$ **then**
    swap($\mathbf{lo}, \mathbf{hi}$) ▷ ensure upper bound can reach at least τ
end if
return $(\mathbf{n}, \mathbf{lo}, \mathbf{hi})$

C Hyperparameter setup & search space

This section is devoted to note the default hyperparameter settings used to train models and perform evaluation within this report. Additionally, we also provide search space for Optuna [47], a hyperparameter tuning framework, to search for more optimal settings. It is noteworthy that both default hyperparameter settings and search space were partly recommended by ChatGPT, a LLM model developed by OpenAI [75].

C.1 Linear Regression

Table 6: Default Hyperparameter Configuration

Hyperparameter	**Value**
loss	“huber”
penalty	“elasticnet”
l1_ratio	0.2
epsilon	1.35
alpha	10^{-4}
max_iter	5000
tol	10^{-4}

Table 7: Hyperparameter Search Space

Hyperparameter	**Search space**
l1_ratio	Uniform[1.05, 3.0]
epsilon	Uniform$[10^{-7}, 10^{-2}]$
alpha	LogUniform$[10^{-7}, 10^{-2}]$
max_iter	LogUniformInt[1000, 20000]
tol	LogUniform$[0, 1]$

C.2 Polynomial Regression

Table 8: Default Hyperparameter Configuration

Hyperparameter	**Value**
degree	2
include_bias	`False`
interaction_only	`True`
loss	“huber”
penalty	“elasticnet”
l1_ratio	0.25
epsilon	1.35
alpha	10^{-4}
max_iter	5000
tol	10^{-4}

Table 9: Hyperparameter Search Space

Hyperparameter	**Search space**
degree	UniformInt[1, 4]
l1_ratio	Uniform[0.0, 1.0]
epsilon	Uniform$[1.05, 3.0]$
alpha	LogUniform$[10^{-7}, 10^{-2}]$

Hyperparameter	Search space
max_iter	LogUniformInt[1000, 20000]
tol	LogUniform[$10^{-6}, 10^{-3}$]

C.3 K–Neareast Neighbours–based Regression

Table 10: Default Hyperparameter Configuration

Hyperparameter	Value
n_neighbors	5
weights	“distance”
algorithm	“auto”

Table 11: Hyperparameter Search Space

Hyperparameter	Search space
n_neighbors	UniformInt[1, 10]
weights	{“uniform”, “distance”}

C.4 Support Vector Machine–based Regression

Table 12: Default Hyperparameter Configuration

Hyperparameter	Value
loss	“epsilon–insensitive”
penalty	“l2”
epsilon	0.2
alpha	10^{-4}
max_iter	5000
tol	10^{-4}

Table 13: Hyperparameter Search Space

Hyperparameter	Search space
epsilon	Uniform[1.05, 3]
alpha	LogUniform[$10^{-7}, 10^{-2}$]
max_iter	LogUniformInt[1000, 20000]
tol	UniformInt[$10^{-6}, 10^{-3}$]

C.5 Decision Tree–based Regression

Table 14: Default Hyperparameter Configuration

Hyperparameter	**Value**
max_depth	12
min_samples_leaf	20
min_samples_split	40
max_features	“sqrt”

Table 15: Hyperparameter Search Space

Hyperparameter	**Search space**
max_depth	UniformInt$[2, 30]$
min_samples_leaf	LogUniformInt$[1, 200]$
min_samples_split	LogUniformInt$[2, 500]$
max_features	{“sqrt”, “log2”}

C.6 Random Forest–based Regression

Table 16: Default Hyperparameter Configuration

Hyperparameter	**Value**
n_estimators	400
max_depth	12
min_samples_leaf	20
min_samples_split	40
max_features	“sqrt”
max_samples	`None`

Table 17: Hyperparameter Search Space

Hyperparameter	**Search space**
n_estimators	UniformInt$[200, 800]$
max_depth	UniformInt$[2, 30]$
min_samples_leaf	LogUniformInt$[1, 200]$
min_samples_split	LogUniformInt$[2, 500]$
max_features	{“sqrt”, “log2”}
max_samples	Uniform$[0.4, 0.9]$

C.7 AdaBoost–based Regression

Table 18: Default Hyperparameter Configuration

Hyperparameter	**Value**
n_estimators	400
max_depth	12
min_samples_leaf	20
min_samples_split	40
max_features	“sqrt”
learning_rate	0.05

Table 19: Hyperparameter Search Space

Hyperparameter	**Search space**
n_estimators	UniformInt$[100, 1500]$
max_depth	UniformInt$[1, 6]$
min_samples_leaf	LogUniformInt$[5, 300]$
min_samples_split	LogUniformInt$[10, 800]$
max_features	{“sqrt”, “log2”, $0.5, 0.7, 1.0$}
learning_rate	LogUniform$[0.01, 0.5]$

C.8 Gradient Boosting–based Regression

Table 20: Default Hyperparameter Configuration

Hyperparameter	**Value**
n_estimators	600
max_depth	3
sub_sample	0.8
min_samples_leaf	20
min_samples_split	40
max_features	`None`
tol	10^{-4}
learning_rate	0.05

Table 21: Hyperparameter Search Space

Hyperparameter	**Search space**
n_estimators	UniformInt$[200, 2000]$
max_depth	UniformInt$[1, 6]$
sub_sample	Uniform$[0.5, 1.0]$
min_samples_leaf	LogUniformInt$[5, 300]$
min_samples_split	LogUniformInt$[10, 800]$
max_features	{“sqrt”, “log2”, $0.5, 0.7, 1.0$}

Hyperparameter	Search space
learning_rate	LogUniform$[0.01, 0.2]$

C.9 XGBoost–based Regression

Table 22: Default Hyperparameter Configuration

Hyperparameter	Value
max_depth	6
n_estimators	2000
learning_rate	0.03
subsample	0.8
colsample_bytree	0.8
reg_lambda	1.0
reg_alpha	1.0
gamma	1.0
max_bin	64

Table 23: Hyperparameter Search Space

Hyperparameter	Search space
max_depth	UniformInt$[3, 8]$
n_estimators	UniformInt$[100, 2000]$
learning_rate	LogUniform$[0.01, 0.15]$
subsample	Uniform$[0.6, 1.0]$
colsample_bytree	Uniform$[0.6, 1.0]$
reg_lambda	LogUniform$[10^{-2}, 1.0]$
reg_alpha	LogUniform$[10^{-2}, 1.0]$
gamma	Uniform$[0.0, 10.0]$
max_bin	$\{64, 152, 256\}$
min_child_weight	LogUniform$[1.0, 128.0]$

C.10 LightGBM–based Regression

Table 24: Default Hyperparameter Configuration

Hyperparameter	Value
max_depth	-1
n_estimators	2000
learning_rate	0.03
subsample	0.8
colsample_bytree	0.8

Hyperparameter	Value
reg_lambda	1.0
reg_alpha	0.0
num_leaves	63
min_child_samples	50

Table 25: Hyperparameter Search Space

Hyperparameter	Search space
max_depth	$\textsc{UniformInt}[3, 8]$
n_estimators	$\textsc{UniformInt}[1000, 2000]$
learning_rate	$\textsc{LogUniform}[0.01, 0.15]$
subsample	$\textsc{Uniform}[0.6, 1.0]$
colsample_bytree	$\textsc{Uniform}[0.6, 1.0]$
reg_lambda	$\textsc{LogUniform}[10^{-3}, 1.0]$
reg_alpha	$\textsc{LogUniform}[10^{-3}, 1.0]$
max_bin	$\{63, 127, 255\}$
num_leaves	$\min\left\{[16, 256], 2^{\text{max\_depth}}\right\}$
min_child_samples	$\textsc{LogUniformInt}[20, 5000]$
min_split_gain	$\textsc{LogUniform}[0.0, 1.0]$
subsample_freq	$\textsc{UniformInt}[1, 10]$

C.11 MLP-like Regression model (RealMLP)

Table 26: Default Hyperparameter Configuration

Hyperparameter	Value
n_epochs	100
batch_size	1024
predict_batch_size	4096
hidden_sizes	"rectangular"
n_hidden_layers	4
hidden_width	64
lr	0.05
wd	10^{-6}
p_drop	0.05
use_plr_embeddings	`False`
use_parametric_act	`False`

Table 27: Hyperparameter Search Space

Hyperparameter	Search space
hidden_sizes	"rectangular"
n_hidden_layers	UniformInt[2, 5]
hidden_width	{64, 128, 256}
lr	LogUniform[0.05, 0.3]
wd	LogUniform[10^{-6}, 0.05]
p_drop	Uniform[0.05, 0.35]
use_plr_embeddings	`False`
use_parametric_act	`False`

C.12 ResNet Regression model

Table 28: Default Hyperparameter Configuration

Hyperparameter	Value
d	512
n_res_blocks	4
d_hidden_factor	0.2
dropout_rate	0.2
act_fn	"relu"
norm	"batchnorm1d"
lr	0.001
weight_decay	10^{-4}
epochs	100
batch_size	1024

Table 29: Hyperparameter Search Space

Hyperparameter	Search space
d	UniformInt[64, 512]
n_res_blocks	UniformInt[1, 4]
d_hidden_factor	Uniform[0, 1]
dropout_rate	Uniform[0, 0.5]
act_fn	"relu"
norm	"batchnorm1d"
lr	Uniform[$10^{-5}, 10^{-2}$]
weight_decay	Uniform[$10^{-6}, 10^{-3}$]
batch_size	{128, 256, 512, 1024}

C.13 FT–Transformer Regression model

Table 30: Default Hyperparameter Configuration

Hyperparameter	Value
n_blocks	3
epochs	20
batch_size	1024

Table 31: Hyperparameter Search Space

Hyperparameter	Search space
n_blocks	UniformInt[1, 6]

D Benchmark results

The tables presented in this Appendix section are more detailed, which specify the averaging performance of all models on each dataset and split regime. As mentioned above, we report three metrics, i.e. nRMSE, nMAE, and sMAPE, and the lower the better. The top performance with regards to dataset is bolded in blue and to split type is bolded in red. Best performance figures in both regards are bolded in green. Additionally, for sMAPE, we divide the results by 100, i.e. scale from the interval [0, 200] to [0, 2], and we round the results to utilize the most space available for nice presentation. For example, a **0** score may imply an extremely small value, e.g. 10^{-5}.

D.1 Linear Regression

Table 32: Per-dataset performance of Linear Regression under different split regimes and datasets.

#	#1	#2	#3	#4	#5	#6	#7	#8
#1	0/0/0.94	0/0/0.92	0.57/0.38/0.88	0/0/1.19	0/0/0.92	0/0/0.94	0/0/1.19	0/0/0.9
#2	0/0/0.96	0/0/0.96	0.6/0.36/0.89	0/0/1.36	0/0/0.94	0/0/0.96	0/0/1.35	0/0/1.02
#3	0/0/0.84	0/0/0.85	0.67/0.48/0.86	0/0/0.89	0/0/0.85	0/0/0.85	0/0/0.86	0/0/0.85
#4	65.3/44.1/0.7	67.8/46/0.71	0.38/0.27/0.67	65.6/44.9/0.97	66/44.6/0.76	69.2/45.7/0.73	65.4/45.2/0.9	62/42.5/0.61
#5	0.14/0.12/1.2	0.15/0.13/1.29	0.88/0.74/1.11	0.16/0.12/1.05	0.14/0.12/1.23	0.15/0.12/1.23	0.16/0.12/1.06	0.18/0.14/0.93
#6	0/0/0.4	0/0/0.42	0.05/0.04/0.3	0.01/0.01/0.07	0/0/0.28	0/0/0.36	0.01/0/0.13	0.01/0/0.11
#7	36/29/0.76	63.7/54.6/1.27	0.64/0.52/0.71	136/111/1.37	40/32.6/0.87	49.4/43.2/0.99	145/124/1.37	137/115/1.35
#8	93.8/78.3/1.06	106/89.3/1.22	2.28/1.77/1.34	373/279/1.3	106/89.8/1.17	193/151/1.26	366/279/1.33	271/190/1.14
#9	0.32/0.26/1.57	0.32/0.26/1.64	1.01/0.82/1.68	0.33/0.27/1.73	0.32/0.26/1.54	0.33/0.26/1.54	0.33/0.27/1.78	0.33/0.25/1.37
#10	0.28/0.2/0.39	0.19/0.12/0.79	0.5/0.36/0.12	0.53/0.26/0.31	0.24/0.19/0.36	0.35/0.23/0.28	0.5/0.26/0.31	0.46/0.27/0.32
#11	1.44/0.97/1.15	1.38/0.96/1.14	0.78/0.53/1.05	2.18/1.27/1.09	1.13/0.85/1.24	1.72/1.18/1.12	2.06/1.21/1.08	1.71/1.13/1.08
#12	0.95/0.74/0.8	0.95/0.73/0.8	0.48/0.37/0.82	5.85/2.78/1.04	0.86/0.68/0.84	1.29/0.91/0.85	6.15/2.89/1.03	5.56/2.67/1
#13	0.76/0.55/0.64	0.39/0.31/0.77	0.52/0.34/0.63	7.07/3.67/0.41	0.62/0.52/0.68	0.79/0.63/0.59	4.67/2.31/0.42	1.16/1.03/0.5
#14	0.95/0.69/0.8	0.94/0.68/0.89	0.53/0.38/0.79	1.33/0.98/0.91	1.34/1.02/0.89	2.16/1.74/1.11	1.35/1.01/0.93	2.36/1.76/1.01
#15	18/8.71/1.38	13.8/7.73/1.35	0.94/0.38/1.39	29.1/13.1/1.31	13.5/6.89/1.39	21.2/10.1/1.29	28.5/14.2/1.4	26.1/13.6/1.45
#16	17/11.3/1.57	17.1/11.3/1.61	1.18/0.69/1.49	17.2/11.7/1.52	14.1/10/1.57	17.6/12.1/1.56	17.7/12.4/1.54	19.2/12.8/1.55
#17	2.65/1.77/1.65	2.24/1.58/1.76	0.64/0.51/1.6	41/9.78/1.43	3.09/1.95/1.69	3.3/1.98/1.61	39.3/9.44/1.43	7.35/4.33/1.62
#18	0/0/0.61	0/0/0.52	0.15/0.11/0.57	0/0/0.64	0/0/0.67	0/0/0.75	0/0/0.67	0/0/0.81
#19	11/8.42/1.71	11/8.46/1.71	0.93/0.72/1.71	55.9/21/1.66	10.6/8.19/1.75	10.7/8.19/1.72	11.1/8.47/1.65	11/8.46/1.66
#20	3.81/3.04/1.12	3.91/3.15/1.19	0.77/0.61/1.06	5.34/4.34/1.05	3.84/3.04/1.14	3.61/2.9/1.18	3.77/3.01/1.06	3.64/2.95/1.04
#21	0/0/0.36	0/0/0.37	0.29/0.21/0.33	0/0/0.36	0/0/0.4	0/0/0.35	0/0/0.36	0/0/0.33
#22	0.05/0.04/1.53	0.05/0.04/1.6	0.85/0.65/1.42	0.04/0.04/1.5	0.06/0.04/1.57	0.07/0.05/1.43	0.05/0.04/1.56	0.05/0.04/1.36
#23	0.2/0.15/0.46	0.41/0.32/0.48	0.29/0.22/0.5	13.2/2.92/0.7	0.43/0.31/0.43	2.17/0.84/0.58	6.01/1.44/0.74	12.2/3.13/0.8
#24	1.01/0.8/1.74	0.93/0.74/1.76	1.16/0.94/1.7	1.28/1.01/1.57	1/0.82/1.73	1.01/0.81/1.69	1.11/0.87/1.69	1.1/0.86/1.64
#25	1.1/0.92/1.21	1.15/1.01/1.27	0.94/0.77/0.98	1.29/1.02/1.04	1.13/0.96/1.21	1.13/0.95/1.3	1.26/1/1.03	1.31/1.03/1
#26	3.45/2.99/1.9	3.45/2.99/1.89	1.01/0.87/1.9	3.46/3/1.82	3.46/2.99/1.9	3.48/3.01/1.89	3.47/3.01/1.8	3.48/3.02/1.9

D.2 Polynomial Regressor

Table 33: Per-dataset performance of Polynomial Regression under different split regimes and datasets.

#	#1	#2	#3	#4	#5	#6	#7	#8
#1	0/0/0.82	0/0/0.84	0.57/0.36/0.8	0/0/1.17	0/0/0.83	0/0/0.86	0/0/1.16	0/0/0.86
#2	0/0/0.84	0/0/0.86	0.63/0.35/0.75	0/0/1.33	0/0/0.8	0/0/0.83	0/0/1.32	0/0/0.85
#3	0/0/0.79	0/0/0.8	0.69/0.45/0.77	0/0/0.9	0/0/0.79	0/0/0.78	0/0/0.88	0/0/0.78
#4	61.8/41.7/0.66	57.1/41.5/0.72	0.32/0.24/0.59	59.7/43.4/0.89	170/42.6/0.7	316/47.5/0.72	59.7/44.1/0.83	123/45.4/0.58
#5	0.13/0.11/1.14	0.14/0.12/1.2	0.86/0.71/1.03	0.24/0.16/1.21	0.14/0.11/1.16	0.15/0.11/1.16	0.23/0.16/1.27	0.28/0.18/0.99
#6	0/0/0.38	0/0/0.4	0.05/0.04/0.28	0.02/0.01/0.08	0/0/0.25	0/0/0.36	0.01/0.01/0.16	0.01/0.01/0.13
#7	24.7/20/0.56	52.9/43.5/1.08	0.4/0.32/0.46	551/416/1.51	37.5/29.7/0.81	74.3/61.6/1.18	303/239/1.47	813/592/1.45
#8	79.2/62/0.84	182/154/1.27	3.39/2.53/1.26	3924/2855/1.57	112/90.9/1.06	145/120/1.21	2147/1406/1.56	6887/4598/1.51
#9	0.32/0.26/1.55	0.32/0.26/1.61	1.03/0.82/1.65	0.39/0.35/1.64	0.32/0.25/1.41	0.34/0.26/1.29	0.41/0.36/1.62	0.38/0.3/1.38
#10	0.25/0.19/0.39	0.16/0.12/0.8	0.44/0.34/0.12	0.64/0.24/0.33	0.23/0.19/0.36	0.3/0.22/0.28	0.69/0.25/0.29	0.55/0.26/0.28
#11	1.43/0.97/1.14	1.37/0.95/1.12	0.78/0.53/1.05	2.24/1.37/1.07	1.14/0.86/1.25	1.77/1.22/1.1	2.42/1.38/0.99	1.85/1.2/1.12
#12	0.97/0.71/0.77	0.91/0.69/0.76	0.47/0.35/0.78	173/25.6/1.12	0.84/0.66/0.84	54.8/8.22/0.92	129/19.3/1.09	120/18.5/1.1
#13	0.59/0.39/0.49	0.37/0.3/0.75	0.51/0.26/0.49	16/4.21/0.56	0.44/0.35/0.55	0.62/0.4/0.5	7.9/2.13/0.44	0.88/0.78/0.5
#14	0.68/0.48/0.63	0.69/0.51/0.68	0.4/0.3/0.61	1.59/1.21/1	0.83/0.64/0.75	2.76/2.2/1.01	1.47/1.08/0.85	2.13/1.57/0.87
#15	17.8/8.63/1.36	14/7.86/1.35	0.93/0.39/1.37	50.8/40.2/1.57	13.6/7.32/1.39	24.2/12.9/1.45	28.8/14.8/1.42	26.7/14.3/1.42
#16	16.8/11.1/1.57	17.1/11.2/1.58	1.18/0.72/1.46	36.6/29.9/1.55	14.9/11/1.59	21.1/13.8/1.52	19.7/14/1.44	21.5/14.2/1.51
#17	10.3/2.2/1.64	7.38/1.9/1.74	1.13/0.54/1.62	543/36.9/1.55	3.08/1.95/1.69	65.1/5.64/1.63	882/60.3/1.54	7.36/4.34/1.63
#18	0/0/0.5	0/0/0.45	0.11/0.08/0.42	0/0/0.69	0/0/0.57	0/0/0.79	0/0/0.68	0/0/0.86
#19	10.9/8.34/1.64	11.2/8.42/1.66	0.94/0.72/1.64	53.7/21.7/1.54	10.5/8.05/1.64	11/8.21/1.64	11.6/8.77/1.58	11.1/8.45/1.58
#20	3.76/3/1.11	3.88/3.13/1.21	0.78/0.62/1.06	5.28/4.3/1.02	3.79/2.99/1.13	3.58/2.92/1.19	3.81/3.04/1.03	3.55/2.9/1.02
#21	0/0/0.34	0/0/0.34	0.29/0.21/0.33	0/0/0.37	0/0/0.39	0/0/0.33	0/0/0.41	0/0/0.38
#22	0.05/0.04/1.52	0.05/0.04/1.77	0.89/0.69/1.42	0.05/0.04/1.38	0.06/0.04/1.55	0.07/0.05/1.44	0.06/0.04/1.57	0.06/0.05/1.35
#23	6.62/0.25/0.28	0.69/0.33/0.32	0.26/0.11/0.26	45.5/5.41/0.39	0.57/0.25/0.28	43.9/3.99/0.38	57.6/2.2/0.5	201/27/0.64
#24	0.89/0.68/1.18	0.83/0.64/1.26	1.05/0.81/1.21	1.31/0.95/1.2	0.89/0.69/1.26	0.92/0.7/1.19	1.13/0.85/1.19	1.06/0.82/1.21
#25	1.06/0.88/1.15	1.12/0.96/1.18	1.48/1.05/1.08	1.76/1.17/1.13	1.12/0.94/1.18	1.13/0.93/1.28	1.87/1.21/1.17	2.04/1.28/1.06
#26	3.45/2.99/1.9	3.45/2.99/1.91	1.01/0.88/1.82	3.46/3/1.88	3.46/2.99/1.87	3.48/3.02/1.87	3.48/3.01/1.82	3.48/3.02/1.84

D.3 KNN–based Regressor

Table 34: Per-dataset performance of KNN Regression under different split regimes and datasets.

#	#1	#2	#3	#4	#5	#6	#7	#8
#1	0/0/0.71	0/0/0.67	0.5/0.3/0.64	0/0/0.87	0/0/0.72	0/0/0.73	0/0/0.85	0/0/0.64
#2	0/0/0.7	0/0/0.75	0.53/0.26/0.57	0/0/0.96	0/0/0.67	0/0/0.71	0/0/0.96	0/0/0.65
#3	0/0/0.73	0/0/0.76	0.65/0.45/0.8	0/0/1.02	0/0/0.74	0/0/0.74	0/0/0.93	0/0/0.86
#4	84.7/55.2/0.81	90.3/59.5/0.85	0.44/0.3/0.71	93.2/57.9/0.96	85.1/55/0.85	88.8/58/0.87	91.8/57.5/0.9	80.8/56.1/0.77
#5	0.1/0.06/0.59	0.15/0.11/0.94	0.82/0.55/0.82	0.15/0.11/1.07	0.12/0.09/0.78	0.13/0.09/0.83	0.15/0.11/1.08	0.18/0.14/1.14
#6	0/0/0.37	0.01/0/0.46	0.09/0.06/0.42	0.05/0.04/0.26	0.01/0.01/0.37	0.02/0.01/0.63	0.02/0.01/0.36	0.02/0.02/0.26
#7	7.09/3.37/0.13	76.6/64.2/1.33	0.25/0.21/0.37	108/88/1.46	79.8/60.8/1.17	120/102/1.46	115/96.2/1.4	115/96.1/1.43
#8	24/11/0.17	195/154/1.27	1.12/0.89/1.09	234/194/1.31	181/139/1.22	208/168/1.22	209/170/1.29	223/178/1.28
#9	0.34/0.28/1.41	0.35/0.28/1.42	1.1/0.91/1.47	0.37/0.31/1.48	0.35/0.28/1.42	0.35/0.29/1.44	0.36/0.3/1.52	0.36/0.28/1.39
#10	0.25/0.19/0.39	0.3/0.24/0.93	0.68/0.54/0.2	0.4/0.31/0.37	0.25/0.2/0.38	0.34/0.27/0.43	0.39/0.31/0.34	0.43/0.33/0.36
#11	1.34/0.92/0.99	1.39/0.96/1.01	0.82/0.59/0.98	1.95/1.47/1.1	1.23/0.94/1.13	1.64/1.21/1.11	2.1/1.67/1.21	1.75/1.32/1.21
#12	0.85/0.62/0.68	0.83/0.61/0.7	0.44/0.32/0.73	1.35/0.99/0.78	0.77/0.58/0.69	0.97/0.69/0.72	1.37/1.01/0.87	1.63/1.19/0.86
#13	0.28/0.18/0.26	0.29/0.22/0.59	0.2/0.12/0.27	1.54/0.98/0.24	0.26/0.17/0.31	0.3/0.19/0.28	1.27/0.95/0.33	0.59/0.47/0.31
#14	0.61/0.41/0.55	0.66/0.47/0.68	0.38/0.28/0.6	0.83/0.58/0.82	1.08/0.8/0.82	2.15/1.68/1.07	1.02/0.72/0.81	2.1/1.53/0.89
#15	12.2/6.17/0.97	17.8/8.34/1.29	0.86/0.42/1.19	48.6/32.7/1.59	16.9/10.1/1.33	54.3/31.4/1.49	32.1/15.9/1.3	27.5/14.6/1.35
#16	12.7/7.91/1.06	14.7/9.58/1.35	1.23/0.69/1.43	22.1/17.3/1.37	19.2/13.8/1.46	26.6/18.9/1.47	22.5/16.1/1.45	26/18.9/1.43
#17	3.04/1.98/1.37	2.4/1.71/1.45	0.67/0.52/1.38	2.39/2/1.45	3.27/2.11/1.44	3.06/1.99/1.38	2.6/2.16/1.45	5.67/3.49/1.42
#18	0/0/0.21	0/0/0.16	0.03/0.02/0.08	0/0/0.45	0/0/0.2	0/0/0.88	0/0/0.46	0/0/0.75
#19	9.3/6.86/1.13	9.4/6.91/1.24	0.93/0.71/1.29	11/8.34/1.3	10.5/8.03/1.3	10.7/8.13/1.24	11.6/8.84/1.33	10.9/8.34/1.31
#20	4.13/3.15/1.04	4.19/3.31/1.19	0.85/0.68/1.1	5.91/4.78/1.11	4.27/3.4/1.12	4.35/3.45/1.19	4.24/3.36/1.1	4.15/3.28/1.11
#21	0/0/0.08	0/0/0.31	0.31/0.22/0.33	0/0/0.46	0/0/0.37	0/0/0.31	0/0/0.49	0/0/0.5
#22	0/0/0.13	0.05/0.04/1.05	0.82/0.58/0.96	0.05/0.04/1.13	0.05/0.04/1.06	0.03/0.02/0.82	0.05/0.04/1.12	0.05/0.04/1.13
#23	0.07/0.03/0.11	0.42/0.34/0.5	0.18/0.11/0.27	1.32/1.04/0.5	0.34/0.25/0.48	0.29/0.19/0.16	1.07/0.94/0.51	1.01/0.87/0.48
#24	0.4/0.3/0.64	0.45/0.35/0.73	0.52/0.4/0.73	0.76/0.57/0.9	0.54/0.42/0.8	0.49/0.38/0.76	0.65/0.48/0.88	0.63/0.48/0.87
#25	0.74/0.46/0.58	1.12/0.84/0.92	1/0.79/1.12	1.15/0.9/1.17	0.95/0.68/0.8	0.98/0.68/0.84	1.15/0.9/1.16	1.22/0.99/1.21
#26	4.07/3.38/1.46	3.79/3.19/1.5	1.12/0.94/1.49	3.77/3.19/1.5	3.78/3.18/1.51	3.89/3.26/1.49	3.77/3.19/1.51	3.75/3.17/1.51

D.4 SVM–based Regressor

Table 35: Per-dataset performance of SVM Regression under different split regimes and datasets.

#	#1	#2	#3	#4	#5	#6	#7	#8
#1	0/0/0.9	0/0/0.75	0.55/0.32/0.65	0/0/1.06	0/0/0.84	0/0/0.84	0/0/1.12	0/0/0.82
#2	0/0/0.92	0/0/0.88	0.6/0.31/0.78	0/0/1.28	0/0/0.85	0/0/0.92	0/0/1.27	0/0/0.87
#3	0/0/0.83	0/0/0.86	0.67/0.47/0.8	0/0/0.91	0/0/0.84	0/0/0.84	0/0/0.89	0/0/0.87
#4	106/51.6/0.77	74.6/51.6/0.77	0.37/0.28/0.7	74/53/0.95	68.2/47.5/0.73	76.9/52.1/0.79	75.2/52.7/0.81	70.4/48.6/0.64

#	#1	#2	#3	#4	#5	#6	#7	#8
#5	0.14/0.11/1.13	0.15/0.12/1.21	0.87/0.72/0.98	0.17/0.13/1.04	0.15/0.12/1.15	0.15/0.12/1.16	0.16/0.13/1.02	0.19/0.15/0.97
#6	0.01/0/0.44	0/0/0.44	0.06/0.05/0.35	0.01/0.01/0.08	0/0/0.31	0.01/0/0.43	0.01/0.01/0.14	0.01/0.01/0.14
#7	33.1/26.7/0.71	95/91.3/1.46	0.65/0.54/0.76	392/328/1.57	49.2/41.9/1	78.1/66/1.05	287/217/1.5	506/391/1.62
#8	56.1/43/0.59	62.5/51.1/0.74	3.28/2.46/1.2	675/497/1.34	70.3/53.4/0.74	220/160/0.93	653/485/1.33	656/468/1.34
#9	0.34/0.24/1.03	0.34/0.24/1.05	1.07/0.76/1.11	0.36/0.26/1.1	0.34/0.24/1.03	0.36/0.25/1.05	0.36/0.26/1.11	0.36/0.25/1
#10	0.29/0.2/0.39	0.18/0.13/0.79	0.52/0.38/0.13	0.56/0.28/0.34	0.24/0.19/0.37	0.34/0.23/0.29	0.57/0.26/0.3	0.45/0.29/0.33
#11	1.5/0.9/1	1.44/0.9/1.01	0.85/0.56/0.98	2.23/1.2/0.97	1.16/0.77/1.11	1.72/1.11/1	2.11/1.16/0.94	1.75/1.08/0.96
#12	1.02/0.78/0.83	0.97/0.74/0.87	0.51/0.39/0.87	7.26/3.33/1.1	0.87/0.68/0.88	1.4/0.91/0.83	4.91/2.18/1.01	5.86/2.78/1.04
#13	0.87/0.5/0.53	0.34/0.27/0.71	0.67/0.33/0.51	6.9/3.47/0.39	0.51/0.42/0.6	0.85/0.53/0.56	4.79/2.36/0.4	1.04/0.91/0.44
#14	1/0.72/0.83	1.13/0.79/0.85	0.55/0.41/0.83	1.58/1.24/1.02	1.33/1/0.88	2.13/1.68/1.07	1.39/1.04/0.94	2.47/1.84/1.08
#15	18.3/8.84/1.31	14/8.2/1.25	0.94/0.38/1.32	34.8/18.8/1.38	13.7/7.14/1.27	21.9/13/1.27	28.8/14.2/1.39	26.4/13.7/1.36
#16	17.4/11.6/1.54	17.4/12.5/1.53	1.2/0.68/1.54	18.6/13.5/1.56	14.5/9.95/1.53	19.7/13/1.49	19/13.1/1.54	21/13.9/1.5
#17	2.69/1.79/1.58	2.48/1.7/1.67	0.75/0.53/1.54	34.2/10.8/1.53	3.16/2.03/1.59	6.24/2.58/1.54	31.8/8.13/1.48	7.38/4.34/1.66
#18	0/0/0.62	0/0/0.51	0.18/0.13/0.45	0/0/0.63	0/0/0.69	0/0/0.96	0/0/0.69	0/0/0.94
#19	11/8.42/1.7	11/8.46/1.7	0.93/0.72/1.69	82.2/27.8/1.65	10.6/8.19/1.74	10.7/8.23/1.69	11.2/8.52/1.66	11.1/8.48/1.67
#20	3.88/3.09/1.12	3.95/3.18/1.2	0.78/0.61/1.13	5.45/4.43/0.98	3.86/3.12/1.17	3.66/2.97/1.16	4.09/3.29/1.05	3.81/3.05/1.05
#21	0/0/0.34	0/0/0.35	0.29/0.21/0.33	0/0/0.37	0/0/0.36	0/0/0.34	0/0/0.35	0/0/0.32
#22	0.05/0.04/1.41	0.05/0.04/1.45	0.89/0.65/1.25	0.05/0.04/1.36	0.06/0.04/1.41	0.07/0.05/1.41	0.05/0.04/1.5	0.05/0.04/1.26
#23	0.25/0.16/0.46	0.52/0.33/0.45	0.31/0.23/0.52	14.6/2.82/0.7	0.58/0.34/0.41	2.58/0.98/0.62	9.17/1.75/0.82	14.6/4.89/0.93
#24	1.02/0.81/1.59	0.94/0.74/1.59	1.18/0.95/1.52	1.32/1.05/1.53	0.99/0.8/1.63	1.03/0.81/1.55	1.14/0.91/1.54	1.17/0.91/1.5
#25	1.12/0.93/1.15	1.15/0.97/1.18	1.05/0.82/1.04	1.37/1.09/1.07	1.17/0.97/1.15	1.21/1/1.26	1.45/1.15/1.06	1.43/1.11/1.06
#26	3.48/3/1.74	3.45/2.99/1.85	1.03/0.89/1.62	3.5/3.03/1.63	3.49/3.01/1.69	3.49/3.02/1.74	3.53/3.05/1.65	3.53/3.04/1.61

D.5 DT–based Regressor

Table 36: Per-dataset performance of DT Regression under different split regimes and datasets.

#	#1	#2	#3	#4	#5	#6	#7	#8
#1	0/0/0.72	0/0/0.69	0.43/0.24/0.51	0/0/0.84	0/0/0.74	0/0/0.73	0/0/0.84	0/0/0.5
#2	0/0/0.74	0/0/0.78	0.52/0.25/0.54	0/0/0.92	0/0/0.7	0/0/0.75	0/0/0.92	0/0/0.73
#3	0/0/0.8	0/0/0.91	0.74/0.54/0.87	0/0/0.95	0/0/0.84	0/0/0.79	0/0/0.92	0/0/0.94
#4	94.4/61.5/0.85	101/66.1/0.93	0.54/0.33/0.7	93.4/60.2/0.94	98.4/63.2/0.89	102/66.9/0.95	94.6/62.3/0.94	94.4/61.8/0.86
#5	0.12/0.09/0.9	0.16/0.13/1.07	0.98/0.75/1.06	0.17/0.14/1.27	0.15/0.11/1.06	0.15/0.12/1.16	0.16/0.13/1.2	0.2/0.16/1.35
#6	0.01/0/0.42	0.01/0/0.45	0.08/0.06/0.38	0.06/0.04/0.23	0.01/0/0.34	0.01/0.01/0.47	0.01/0.01/0.27	0.01/0.01/0.17
#7	27.2/17/0.5	97.4/81.4/1.4	0.51/0.38/0.73	83.3/70/1.49	68.3/55.2/1.26	122/107/1.48	94.9/78.3/1.45	107/88.4/1.41
#8	86.4/61.7/0.95	151/125/1.36	1.24/0.97/1.27	187/150/1.29	156/124/1.21	148/116/1.24	187/152/1.28	172/137/1.32
#9	0.31/0.26/1.62	0.32/0.27/1.67	1/0.85/1.71	0.34/0.3/1.76	0.32/0.26/1.66	0.32/0.27/1.73	0.34/0.3/1.76	0.34/0.27/1.48
#10	0.24/0.18/0.38	0.28/0.23/0.89	0.58/0.46/0.17	0.3/0.24/0.3	0.23/0.18/0.36	0.3/0.23/0.32	0.3/0.24/0.28	0.32/0.25/0.3
#11	1.32/0.94/1.05	1.33/0.94/1.04	0.76/0.59/1.01	1.94/1.51/1.18	1.24/0.95/1.15	2.03/1.62/1.28	2/1.52/1.26	1.69/1.33/1.18
#12	0.95/0.69/0.73	0.99/0.74/0.8	0.49/0.37/0.76	1.46/1.08/0.8	0.91/0.7/0.77	1.01/0.75/0.76	1.38/1.03/0.79	1.39/1.03/0.79
#13	0.39/0.26/0.37	0.51/0.37/0.8	0.22/0.14/0.33	1.97/1.12/0.31	0.76/0.52/0.59	0.67/0.41/0.39	2.8/2.12/0.58	0.92/0.67/0.42
#14	0.79/0.54/0.67	0.91/0.63/0.8	0.46/0.32/0.67	1/0.69/0.97	1.21/0.91/0.96	2.23/1.75/1.14	1.22/0.93/0.94	2.23/1.67/1
#15	16.5/8.68/1.22	16.1/9.93/1.25	0.99/0.55/1.34	65.1/53.3/1.65	16.6/9.49/1.39	33.2/23/1.52	33.8/18.1/1.44	34/22/1.43
#16	15.8/10.6/1.36	16.2/11.6/1.46	1.3/0.81/1.44	21.4/17.3/1.43	18.3/13.4/1.47	35.4/29.8/1.56	22.5/17/1.46	23.6/18.3/1.47
#17	2.54/1.78/1.44	2.16/1.59/1.46	0.64/0.5/1.4	1.81/1.46/1.4	2.9/1.98/1.5	2.64/1.83/1.43	1.98/1.6/1.46	5.19/3.37/1.45
#18	0/0/0.23	0/0/0.18	0.04/0.02/0.08	0/0/0.34	0/0/0.25	0/0/0.92	0/0/0.43	0/0/0.53
#19	8.9/6.52/1.13	9.02/6.65/1.18	0.81/0.61/1.14	9.4/6.82/1.17	8.53/6.32/1.13	8.62/6.3/1.11	9.15/6.75/1.16	9.01/6.65/1.17
#20	3.76/2.99/1.08	3.97/3.19/1.22	0.83/0.67/1.11	5.62/4.55/1.13	4.13/3.3/1.13	4.08/3.26/1.16	4.14/3.29/1.12	4.04/3.26/1.11
#21	0/0/0.23	0/0/0.33	0.3/0.22/0.36	0/0/0.42	0/0/0.41	0/0/0.36	0/0/0.42	0/0/0.47
#22	0.03/0.02/0.77	0.05/0.04/1.16	0.72/0.54/1	0.05/0.04/1.18	0.05/0.04/1.1	0.04/0.03/1.02	0.05/0.04/1.16	0.06/0.04/1.21
#23	0.08/0.05/0.18	0.39/0.31/0.44	0.19/0.11/0.24	1.03/0.86/0.38	0.3/0.23/0.38	0.28/0.18/0.21	0.93/0.75/0.35	0.91/0.74/0.36
#24	0.87/0.69/1.19	0.84/0.67/1.18	0.99/0.79/1.15	1.17/0.91/1.3	0.9/0.71/1.24	0.92/0.71/1.2	1/0.78/1.27	1.04/0.81/1.29
#25	0.94/0.7/0.87	1.21/0.96/1.1	1.08/0.88/1.3	1.34/1.07/1.21	1.11/0.86/1.01	1.18/0.93/1.06	1.27/1.03/1.22	1.27/1.05/1.3
#26	3.54/3.04/1.7	3.55/3.05/1.68	1.02/0.88/1.76	3.57/3.06/1.65	3.53/3.03/1.68	3.55/3.05/1.68	3.56/3.06/1.63	3.55/3.06/1.57

D.6 RF–based Regressor

Table 37: Per-dataset performance of RF Regression under different split regimes and datasets.

#	#1	#2	#3	#4	#5	#6	#7	#8
#1	0/0/0.68	0/0/0.62	0.43/0.25/0.54	0/0/0.81	0/0/0.69	0/0/0.7	0/0/0.81	0/0/0.54
#2	0/0/0.7	0/0/0.73	0.46/0.21/0.51	0/0/0.84	0/0/0.66	0/0/0.71	0/0/0.84	0/0/0.79
#3	0/0/0.73	0/0/0.79	0.66/0.51/0.89	0/0/0.93	0/0/0.76	0/0/0.74	0/0/0.9	0/0/0.93
#4	84.3/55.1/0.82	89.2/58.3/0.87	0.42/0.3/0.75	85.7/52.3/0.9	84.1/54.1/0.84	87/58.1/0.93	84.9/53.5/0.88	81.4/53.5/0.84
#5	0.11/0.09/0.89	0.14/0.13/1.15	0.85/0.7/1.05	0.15/0.13/1.34	0.13/0.11/1.08	0.13/0.11/1.13	0.15/0.13/1.34	0.18/0.16/1.45
#6	0/0/0.36	0/0/0.39	0.06/0.04/0.23	0.04/0.03/0.22	0/0/0.26	0.01/0/0.37	0.01/0.01/0.24	0.01/0.01/0.14
#7	21.7/12.9/0.36	75.4/63.2/1.47	0.43/0.28/0.46	83/69.2/1.44	64.5/52.6/1.26	111/95.4/1.49	80.4/67/1.43	82.7/68.8/1.42
#8	67/45/0.62	142/118/1.44	0.95/0.77/1.27	170/134/1.29	124/95.2/1.19	138/116/1.37	156/126/1.31	149/121/1.36
#9	0.31/0.26/1.64	0.32/0.27/1.71	0.99/0.84/1.74	0.33/0.3/1.79	0.32/0.26/1.71	0.32/0.27/1.79	0.33/0.29/1.8	0.33/0.27/1.46
#10	0.23/0.18/0.37	0.23/0.19/0.92	0.67/0.54/0.19	0.29/0.23/0.3	0.22/0.18/0.35	0.28/0.22/0.3	0.28/0.22/0.27	0.31/0.25/0.28
#11	1.25/0.89/1.02	1.26/0.9/1.02	0.69/0.55/1	1.76/1.38/1.2	1.05/0.81/1.16	1.47/1.17/1.21	1.63/1.24/1.16	1.58/1.31/1.21
#12	0.75/0.54/0.61	0.74/0.55/0.65	0.38/0.28/0.66	1.19/0.85/0.67	0.68/0.51/0.66	0.8/0.58/0.63	1.14/0.82/0.67	1.16/0.82/0.65
#13	0.27/0.18/0.25	0.33/0.26/0.62	0.2/0.13/0.27	3.17/2.08/0.4	0.34/0.25/0.37	0.42/0.26/0.29	1.88/1.4/0.52	0.93/0.8/0.49
#14	0.61/0.41/0.55	0.67/0.47/0.67	0.34/0.22/0.56	0.78/0.54/0.84	0.89/0.65/0.78	2.1/1.71/1.07	0.92/0.68/0.81	2.1/1.55/0.91
#15	15.6/8.05/1.24	15.9/8.15/1.38	0.88/0.42/1.38	46.6/35.8/1.59	14.2/7.89/1.44	28.2/18.1/1.58	29.9/16.6/1.45	29.9/18.5/1.45
#16	15.1/9.97/1.38	15.6/10.6/1.52	1.16/0.7/1.47	18.5/13.2/1.46	15.2/11.3/1.47	20.5/15.2/1.52	19.6/14.4/1.47	20.4/14.8/1.47

#	#1	#2	#3	#4	#5	#6	#7	#8
#17	2.47/1.73/1.47	2.11/1.53/1.49	0.6/0.46/1.46	1.69/1.36/1.45	2.81/1.9/1.57	2.58/1.76/1.46	1.9/1.55/1.49	5.23/3.3/1.49
#18	0/0/0.14	0/0/0.14	0.03/0.02/0.08	0/0/0.26	0/0/0.15	0/0/0.61	0/0/0.26	0/0/0.4
#19	8.28/6/1.12	8.33/6.04/1.15	0.72/0.53/1.1	8.61/6.12/1.12	7.85/5.73/1.1	8.07/5.81/1.1	8.45/6.08/1.12	8.17/5.94/1.1
#20	3.64/2.89/1.06	3.82/3.08/1.17	0.78/0.63/1.06	5.42/4.41/1.1	3.91/3.12/1.13	3.94/3.17/1.15	3.7/2.97/1.07	3.66/2.96/1.11
#21	0/0/0.17	0/0/0.23	0.19/0.14/0.24	0/0/0.34	0/0/0.27	0/0/0.23	0/0/0.36	0/0/0.37
#22	0.02/0.02/0.69	0.04/0.04/1.26	0.67/0.51/1.07	0.04/0.03/1.22	0.04/0.03/1.13	0.04/0.03/1.12	0.04/0.03/1.26	0.04/0.03/1.23
#23	0.07/0.03/0.12	0.35/0.24/0.34	0.11/0.08/0.18	1/0.82/0.35	0.2/0.14/0.28	0.29/0.18/0.17	0.88/0.73/0.35	0.81/0.65/0.32
#24	0.73/0.59/1.15	0.7/0.57/1.2	0.87/0.7/1.25	0.97/0.77/1.32	0.77/0.63/1.26	0.75/0.61/1.22	0.9/0.72/1.31	0.84/0.67/1.33
#25	0.83/0.65/0.85	1.12/0.93/1.13	0.89/0.76/1.26	1.12/0.94/1.25	0.98/0.81/1.03	1.02/0.85/1.11	1.11/0.93/1.25	1.15/0.98/1.29
#26	3.47/3/1.76	3.47/3/1.76	1.01/0.88/1.79	3.51/3.03/1.69	3.49/3.01/1.75	3.5/3.03/1.76	3.5/3.03/1.74	3.49/3.03/1.75

D.7 AdaBoost–based Regressor

Table 38: Per-dataset performance of AdaBoost Regression under different split regimes and datasets.

#	#1	#2	#3	#4	#5	#6	#7	#8
#1	0/0/0.77	0/0/0.76	0.44/0.27/0.59	0/0/0.87	0/0/0.79	0/0/0.79	0/0/0.88	0/0/0.59
#2	0/0/0.71	0/0/0.73	0.43/0.23/0.56	0/0/0.84	0/0/0.68	0/0/0.72	0/0/0.84	0/0/0.8
#3	0/0/0.75	0/0/0.79	0.62/0.49/0.85	0/0/0.92	0/0/0.77	0/0/0.75	0/0/0.89	0/0/0.92
#4	69.4/50.1/0.82	72.6/51.8/0.86	0.42/0.3/0.78	70.9/47.4/0.92	69.1/49.2/0.86	72.3/53.2/0.93	70.1/48.6/0.9	70.9/50.6/0.84
#5	0.12/0.1/1.12	0.15/0.14/1.34	0.91/0.83/1.33	0.15/0.14/1.43	0.14/0.12/1.29	0.14/0.12/1.33	0.15/0.13/1.43	0.2/0.19/1.52
#6	0/0/0.32	0/0/0.34	0.05/0.04/0.2	0.03/0.02/0.17	0/0/0.22	0/0/0.34	0.01/0/0.2	0.01/0.01/0.11
#7	9.53/6.97/0.23	73.9/60.9/1.5	0.3/0.25/0.49	85.3/69.2/1.43	64.3/52.7/1.2	115/99.4/1.47	69.5/58.9/1.48	73.8/62.5/1.44
#8	31.9/23/0.35	145/118/1.32	1.02/0.8/1.1	191/152/1.28	126/95.7/1.11	158/124/1.27	181/145/1.31	168/135/1.3
#9	0.33/0.31/1.73	0.34/0.32/1.73	1.05/0.98/1.71	0.35/0.33/1.68	0.33/0.31/1.74	0.33/0.31/1.71	0.35/0.33/1.68	0.34/0.31/1.76
#10	0.23/0.18/0.37	0.28/0.23/0.96	0.61/0.49/0.17	0.29/0.23/0.3	0.23/0.18/0.36	0.28/0.22/0.3	0.28/0.22/0.28	0.31/0.25/0.28
#11	1.43/1.22/1.38	1.47/1.26/1.39	0.74/0.61/1.2	1.93/1.65/1.4	1.18/1.02/1.37	1.64/1.4/1.41	1.88/1.62/1.45	1.74/1.52/1.42
#12	0.7/0.5/0.57	0.69/0.51/0.6	0.36/0.26/0.61	1.12/0.81/0.66	0.62/0.46/0.62	0.75/0.54/0.6	1.05/0.77/0.65	1.07/0.77/0.63
#13	0.29/0.22/0.32	0.27/0.21/0.6	0.2/0.15/0.33	2.74/1.66/0.34	0.29/0.22/0.35	0.32/0.23/0.31	1.63/1.13/0.4	0.72/0.61/0.34
#14	0.51/0.36/0.53	0.58/0.41/0.64	0.3/0.21/0.52	0.72/0.5/0.76	0.76/0.57/0.71	1.92/1.54/0.99	0.81/0.59/0.71	1.92/1.4/0.83
#15	13/7.35/1.23	14.4/7.86/1.41	0.82/0.42/1.32	84.6/39.9/1.56	14.6/8.13/1.36	31.9/21/1.56	32.8/16.4/1.41	32.1/17.3/1.45
#16	14.1/9.84/1.41	15.1/10.7/1.53	1.19/0.71/1.47	21.1/15.7/1.34	16.1/11.7/1.48	19.1/14.2/1.44	20.8/15.2/1.48	22.2/15.3/1.48
#17	2.53/1.74/1.46	2.19/1.54/1.48	0.6/0.46/1.47	1.91/1.56/1.45	3.18/1.95/1.53	2.68/1.79/1.47	2/1.63/1.46	5.62/3.4/1.45
#18	0/0/0.13	0/0/0.14	0.03/0.02/0.09	0/0/0.18	0/0/0.14	0/0/0.77	0/0/0.18	0/0/0.28
#19	8.19/6.13/0.97	8.08/6.02/0.99	0.68/0.5/0.94	8.42/5.99/0.97	7.88/5.95/0.96	7.77/5.79/0.94	8.08/5.94/0.97	7.8/5.82/0.96
#20	3.7/2.96/1.11	3.91/3.16/1.25	0.79/0.64/1.08	5.47/4.49/1.13	3.95/3.18/1.18	4.11/3.32/1.17	3.72/3.02/1.12	3.81/3.06/1.12
#21	0/0/0.15	0/0/0.21	0.16/0.12/0.21	0/0/0.3	0/0/0.24	0/0/0.21	0/0/0.31	0/0/0.29
#22	0.02/0.02/0.64	0.04/0.03/1.17	0.57/0.41/0.89	0.04/0.03/1.18	0.04/0.03/0.99	0.03/0.03/1.02	0.04/0.03/1.18	0.04/0.03/1.17
#23	0.07/0.04/0.12	0.42/0.29/0.31	0.13/0.09/0.19	1.01/0.85/0.36	0.22/0.17/0.3	0.29/0.19/0.17	0.99/0.8/0.35	0.95/0.77/0.32
#24	0.59/0.48/0.96	0.59/0.49/1.05	0.71/0.58/1.05	0.85/0.67/1.16	0.66/0.54/1.11	0.64/0.53/1.07	0.8/0.64/1.16	0.76/0.61/1.22
#25	0.9/0.76/1.06	1.13/1/1.29	0.93/0.81/1.43	1.16/1.01/1.41	1.02/0.89/1.21	1.09/0.93/1.29	1.12/0.97/1.4	1.21/1.06/1.48
#26	3.45/2.99/1.89	3.45/2.99/1.88	1.01/0.87/1.88	3.48/3.01/1.82	3.47/3/1.89	3.48/3.02/1.88	3.48/3.02/1.83	3.49/3.02/1.8

D.8 GB–based Regressor

Table 39: Per-dataset performance of GB Regression under different split regimes and datasets.

#	#1	#2	#3	#4	#5	#6	#7	#8
#1	0/0/0.68	0/0/0.67	0.42/0.22/0.45	0/0/0.81	0/0/0.71	0/0/0.71	0/0/0.79	0/0/0.47
#2	0/0/0.68	0/0/0.7	0.43/0.21/0.49	0/0/0.84	0/0/0.66	0/0/0.7	0/0/0.85	0/0/0.69
#3	0/0/0.63	0/0/0.64	0.55/0.39/0.72	0/0/0.78	0/0/0.62	0/0/0.63	0/0/0.75	0/0/0.71
#4	51.9/37.1/0.63	59.8/41.8/0.69	0.34/0.25/0.64	53.9/38.3/0.8	51.6/36.5/0.67	55/39.3/0.73	53.3/38/0.72	49.4/37/0.54
#5	0.12/0.09/0.93	0.14/0.12/1.17	0.85/0.69/1.03	0.14/0.11/1.2	0.13/0.11/1.08	0.13/0.11/1.11	0.15/0.12/1.23	0.19/0.16/1.34
#6	0/0/0.34	0/0/0.37	0.05/0.04/0.21	0.03/0.02/0.16	0/0/0.23	0/0/0.32	0.01/0/0.15	0.01/0/0.11
#7	8.23/6.2/0.22	69.6/57.7/1.43	0.22/0.18/0.3	83.5/69.4/1.46	57.9/45.4/1.13	109/93.7/1.5	83.6/67.5/1.42	83.8/68.7/1.36
#8	25.4/17.9/0.27	128/107/1.31	0.93/0.75/1.08	162/126/1.31	126/88.6/1.04	154/125/1.21	157/126/1.25	149/124/1.37
#9	0.31/0.26/1.65	0.32/0.27/1.68	0.99/0.84/1.71	0.33/0.3/1.78	0.32/0.27/1.69	0.32/0.27/1.77	0.33/0.29/1.81	0.33/0.26/1.35
#10	0.23/0.18/0.37	0.27/0.22/0.96	0.66/0.53/0.2	0.28/0.22/0.31	0.22/0.18/0.35	0.28/0.22/0.29	0.28/0.22/0.27	0.3/0.24/0.28
#11	1.13/0.74/0.81	1.16/0.77/0.8	0.69/0.47/0.81	1.69/1.21/1.03	0.92/0.64/0.92	1.47/1.17/1.17	1.45/0.99/0.83	1.44/1.06/0.89
#12	0.65/0.47/0.53	0.66/0.47/0.56	0.35/0.25/0.57	1.02/0.74/0.64	0.57/0.42/0.56	0.71/0.51/0.57	0.98/0.71/0.6	1.01/0.74/0.6
#13	0.25/0.16/0.24	0.21/0.15/0.54	0.19/0.11/0.26	1.19/0.63/0.18	0.22/0.14/0.27	0.26/0.17/0.24	0.6/0.35/0.19	0.33/0.23/0.19
#14	0.49/0.35/0.51	0.56/0.4/0.61	0.28/0.2/0.49	0.58/0.41/0.67	0.69/0.52/0.62	1.82/1.45/0.95	0.65/0.47/0.64	1.5/1.14/0.72
#15	14.3/8.1/1.25	16.6/8.59/1.34	0.88/0.44/1.29	89.9/76.6/1.7	16.2/9.68/1.43	57.2/40.4/1.61	36.2/20.1/1.44	37.4/20.4/1.46
#16	14.6/9.95/1.35	15.4/10.5/1.49	1.28/0.8/1.42	24.6/19.2/1.37	17.1/12.2/1.47	25.1/18.6/1.43	22.3/16.5/1.46	25.1/18.6/1.42
#17	2.53/1.74/1.43	2.08/1.53/1.48	0.61/0.47/1.44	1.9/1.55/1.43	2.85/1.93/1.47	2.61/1.76/1.41	2.04/1.67/1.43	5.24/3.45/1.43
#18	0/0/0.14	0/0/0.13	0.02/0.02/0.05	0/0/0.15	0/0/0.15	0/0/0.49	0/0/0.16	0/0/0.24
#19	7.48/5.33/0.96	7.5/5.39/1	0.65/0.47/0.94	7.87/5.44/0.95	7.06/5.11/0.94	7.26/5.15/0.94	7.64/5.41/0.96	7.37/5.27/0.96
#20	3.64/2.89/1.06	3.83/3.07/1.16	0.77/0.62/1.09	5.55/4.5/1.1	3.97/3.1/1.1	4.19/3.35/1.17	3.74/2.99/1.07	3.78/3.03/1.08
#21	0/0/0.21	0/0/0.22	0.16/0.12/0.23	0/0/0.28	0/0/0.24	0/0/0.23	0/0/0.25	0/0/0.25
#22	0.03/0.02/0.82	0.05/0.04/1.22	0.64/0.45/0.87	0.04/0.03/1.08	0.04/0.03/1	0.04/0.03/0.93	0.04/0.03/1.06	0.04/0.03/1.19
#23	0.07/0.03/0.13	0.28/0.18/0.21	0.1/0.06/0.17	0.74/0.59/0.24	0.14/0.1/0.17	0.16/0.09/0.14	0.59/0.46/0.21	0.59/0.43/0.22
#24	0.74/0.59/1.1	0.7/0.56/1.14	0.9/0.7/1.16	1.04/0.78/1.2	0.79/0.63/1.23	0.76/0.59/1.16	1/0.77/1.24	0.91/0.74/1.27
#25	0.88/0.7/0.91	1.1/0.91/1.12	0.87/0.71/1.14	1.14/0.94/1.31	1/0.81/1.03	1.06/0.87/1.12	1.11/0.91/1.25	1.3/1.09/1.32
#26	3.46/2.99/1.83	3.46/2.99/1.78	1.01/0.88/1.78	3.47/3.01/1.8	3.47/3.01/1.83	3.49/3.02/1.83	3.48/3.02/1.79	3.49/3.02/1.73

D.9 XGBoost–based Regressor

Table 40: Per-dataset performance of XGBoost Regression under different split regimes and datasets.

#	#1	#2	#3	#4	#5	#6	#7	#8
#1	0/0/0.64	0/0/0.66	0.42/0.22/0.46	0/0/0.78	0/0/0.66	0/0/0.67	0/0/0.78	0/0/0.46
#2	0/0/0.64	0/0/0.68	0.41/0.19/0.44	0/0/0.8	0/0/0.61	0/0/0.65	0/0/0.8	0/0/0.67
#3	0/0/0.61	0/0/0.64	0.55/0.4/0.73	0/0/0.73	0/0/0.61	0/0/0.62	0/0/0.73	0/0/0.73
#4	52.1/37.3/0.63	59.2/41.7/0.68	0.34/0.24/0.64	56.4/39.2/0.8	52.1/36.9/0.67	55.9/39.7/0.71	55.7/38.7/0.72	50.6/37.9/0.56
#5	0.11/0.08/0.79	0.14/0.12/1.07	0.82/0.65/0.98	0.14/0.12/1.11	0.12/0.1/0.98	0.13/0.1/0.98	0.14/0.12/1.14	0.17/0.14/1.28
#6	0/0/0.37	0/0/0.4	0.06/0.04/0.29	0.04/0.03/0.2	0/0/0.25	0.01/0/0.37	0.01/0.01/0.26	0.01/0.01/0.13
#7	13.1/9.69/0.29	74/61.3/1.43	0.29/0.23/0.44	81.4/67.7/1.41	65.6/51.8/1.18	110/94.1/1.46	80.6/67/1.42	86.7/71.7/1.4
#8	33.7/24.4/0.35	150/125/1.34	0.96/0.76/1.19	176/139/1.26	128/95/1.07	150/123/1.31	145/119/1.31	138/116/1.37
#9	0.31/0.26/1.6	0.32/0.27/1.68	0.99/0.84/1.69	0.33/0.3/1.78	0.32/0.27/1.67	0.32/0.27/1.77	0.33/0.3/1.79	0.33/0.27/1.45
#10	0.23/0.18/0.37	0.42/0.37/1.32	0.67/0.54/0.19	0.3/0.23/0.31	0.23/0.18/0.35	0.28/0.22/0.29	0.29/0.23/0.28	0.32/0.26/0.29
#11	1.15/0.77/0.89	1.39/1.06/1.13	0.66/0.5/0.88	2.03/1.5/1.15	0.96/0.7/0.98	1.95/1.56/1.23	1.59/1.16/1.03	1.67/1.26/1.02
#12	0.65/0.46/0.53	0.64/0.46/0.56	0.34/0.24/0.57	1.08/0.76/0.63	0.58/0.43/0.56	0.72/0.51/0.57	1.02/0.72/0.62	1.03/0.74/0.61
#13	0.27/0.17/0.24	0.35/0.27/0.61	0.2/0.12/0.26	2.37/1.32/0.26	0.3/0.23/0.37	0.28/0.18/0.25	1.33/0.88/0.34	0.57/0.48/0.33
#14	0.52/0.36/0.51	0.56/0.39/0.61	0.29/0.2/0.5	0.65/0.46/0.73	0.74/0.56/0.7	1.76/1.41/0.93	0.79/0.57/0.69	1.81/1.35/0.8
#15	13.3/7.47/1.2	14.6/8.63/1.35	0.85/0.43/1.3	86.3/67.1/1.67	16.5/9.18/1.4	43.7/31/1.6	33.1/18.8/1.46	33.6/21/1.48
#16	13.6/9.2/1.29	14.9/9.95/1.46	1.2/0.72/1.43	22.1/16.9/1.37	16.5/12/1.44	19.9/15/1.41	23.4/17.1/1.43	23.4/17.7/1.45
#17	2.8/1.79/1.43	2.22/1.54/1.49	0.6/0.47/1.44	1.68/1.34/1.44	3.06/2.01/1.5	2.82/1.81/1.42	1.76/1.42/1.45	5.45/3.3/1.47
#18	0/0/0.15	0/0/0.12	0.03/0.02/0.09	0/0/0.14	0/0/0.16	0/0/0.72	0/0/0.15	0/0/0.19
#19	6.45/4.36/0.78	6.81/4.96/0.85	0.58/0.39/0.79	7.25/4.71/0.81	5.97/4.12/0.77	6.26/4.21/0.77	6.7/4.52/0.8	6.38/4.34/0.79
#20	3.62/2.87/1.05	3.88/3.12/1.14	0.77/0.62/1.08	5.51/4.48/1.13	3.84/3.08/1.11	3.88/3.07/1.15	3.69/2.97/1.07	3.75/3.03/1.08
#21	0/0/0.19	0/0/0.23	0.19/0.13/0.26	0/0/0.3	0/0/0.26	0/0/0.23	0/0/0.3	0/0/0.29
#22	0.03/0.02/0.7	0.05/0.04/1.07	0.64/0.49/1.03	0.04/0.03/1.08	0.04/0.03/0.93	0.03/0.03/0.94	0.04/0.03/1.14	0.04/0.03/1.14
#23	0.07/0.04/0.13	0.31/0.23/0.26	0.11/0.08/0.17	1.04/0.86/0.38	0.25/0.19/0.28	0.25/0.16/0.16	0.82/0.69/0.33	0.72/0.57/0.28
#24	0.54/0.42/0.82	0.59/0.47/0.95	0.66/0.51/0.9	0.83/0.61/0.97	0.63/0.49/0.99	0.59/0.47/0.93	0.77/0.6/1.06	0.72/0.57/1.06
#25	0.78/0.59/0.78	1.1/0.88/1.04	0.95/0.8/1.27	1.09/0.88/1.18	0.94/0.74/0.94	0.98/0.77/0.99	1.07/0.87/1.16	1.11/0.92/1.2
#26	3.46/2.99/1.82	3.46/2.99/1.82	1.02/0.88/1.73	3.48/3.01/1.78	3.47/3/1.83	3.49/3.02/1.81	3.48/3.01/1.79	3.5/3.03/1.75

D.10 LightGBM–based Regressor

Table 41: Per-dataset performance of LightGBM Regression under different split regimes and datasets.

#	#1	#2	#3	#4	#5	#6	#7	#8
#1	0/0/0.65	0/0/0.6	0.41/0.22/0.46	0/0/0.8	0/0/0.67	0/0/0.67	0/0/0.79	0/0/0.46
#2	0/0/0.64	0/0/0.67	0.42/0.2/0.45	0/0/0.81	0/0/0.6	0/0/0.65	0/0/0.81	0/0/0.63
#3	0/0/0.56	0/0/0.57	0.54/0.38/0.66	0/0/0.73	0/0/0.55	0/0/0.56	0/0/0.7	0/0/0.7
#4	47.8/34.9/0.6	53.1/38.2/0.63	0.33/0.24/0.62	52.7/37.7/0.77	47.4/34.8/0.64	50.6/37/0.71	51.1/37.1/0.7	46.7/35.1/0.52
#5	0.1/0.07/0.68	0.14/0.12/1.05	0.77/0.58/0.86	0.14/0.12/1.11	0.12/0.09/0.89	0.13/0.1/0.9	0.15/0.12/1.1	0.18/0.15/1.38
#6	0/0/0.23	0/0/0.31	0.04/0.03/0.2	0.03/0.02/0.12	0/0/0.16	0/0/0.28	0.01/0/0.16	0.01/0/0.08
#7	4.01/2.7/0.12	68.1/55.3/1.37	0.15/0.11/0.19	86.8/70.5/1.41	55.4/41.6/1.09	110/92.7/1.45	77.6/64.2/1.43	96.5/79/1.37
#8	11.8/7.12/0.12	134/112/1.22	0.93/0.76/1.08	176/138/1.23	140/96.9/0.97	149/119/1.19	143/115/1.26	142/116/1.33
#9	0.31/0.26/1.58	0.32/0.27/1.67	0.99/0.84/1.66	0.33/0.3/1.77	0.32/0.26/1.66	0.32/0.27/1.75	0.33/0.29/1.8	0.33/0.26/1.41
#10	0.23/0.18/0.37	0.27/0.22/0.95	0.74/0.6/0.22	0.29/0.23/0.3	0.22/0.18/0.35	0.29/0.23/0.31	0.29/0.22/0.27	0.31/0.25/0.28
#11	1.04/0.61/0.59	1.13/0.74/0.74	0.64/0.4/0.59	1.63/1.18/0.98	0.85/0.56/0.7	1.61/1.2/1.14	1.4/0.99/0.78	1.42/0.98/0.79
#12	0.63/0.44/0.51	0.63/0.44/0.54	0.34/0.24/0.55	1.02/0.72/0.62	0.55/0.4/0.55	0.71/0.5/0.56	0.96/0.69/0.6	1.04/0.74/0.62
#13	0.26/0.16/0.24	0.35/0.26/0.59	0.19/0.12/0.26	1.75/0.94/0.21	0.28/0.21/0.33	0.32/0.2/0.25	0.94/0.64/0.23	0.44/0.35/0.21
#14	0.46/0.31/0.46	0.51/0.36/0.55	0.27/0.18/0.46	0.61/0.42/0.7	0.65/0.48/0.59	1.8/1.45/0.95	0.72/0.52/0.67	1.65/1.23/0.75
#15	12.2/7.27/1.09	17/9.32/1.33	0.88/0.51/1.26	99.2/85.6/1.69	17.5/10.8/1.38	45.3/35.3/1.59	34.1/20.3/1.44	33.4/20/1.43
#16	12.9/8.8/1.14	14.6/10.4/1.33	1.27/0.81/1.39	22.8/17.5/1.39	18.7/13.6/1.41	21.6/16.7/1.4	24.3/18.7/1.43	26/19.9/1.44
#17	2.66/1.87/1.38	2.27/1.68/1.41	0.67/0.52/1.37	2.37/1.98/1.44	3/2.08/1.43	2.82/1.9/1.37	2.17/1.8/1.42	5.59/3.67/1.42
#18	0/0/0.08	0/0/0.08	0.02/0.01/0.04	0/0/0.15	0/0/0.07	0/0/0.41	0/0/0.15	0/0/0.25
#19	6.06/4.05/0.74	6.21/4.38/0.82	0.54/0.37/0.77	6.81/4.43/0.79	5.61/3.85/0.73	5.9/3.95/0.74	6.38/4.32/0.77	5.94/4.03/0.75
#20	3.73/2.93/1.04	4.02/3.2/1.19	0.82/0.66/1.12	5.71/4.64/1.12	4.09/3.28/1.12	4.3/3.43/1.17	3.87/3.11/1.12	4.01/3.22/1.1
#21	0/0/0.12	0/0/0.2	0.16/0.11/0.2	0/0/0.27	0/0/0.22	0/0/0.19	0/0/0.26	0/0/0.26
#22	0.01/0.01/0.43	0.05/0.04/1.08	0.36/0.25/0.59	0.03/0.02/0.98	0.02/0.02/0.62	0.03/0.02/0.63	0.03/0.02/0.82	0.03/0.02/0.89
#23	0.06/0.02/0.08	0.3/0.2/0.23	0.1/0.07/0.15	0.99/0.78/0.35	0.21/0.15/0.26	0.2/0.11/0.1	0.68/0.54/0.27	0.66/0.5/0.23
#24	0.32/0.24/0.53	0.36/0.28/0.61	0.42/0.32/0.59	0.64/0.46/0.76	0.46/0.34/0.72	0.42/0.32/0.67	0.6/0.45/0.86	0.6/0.45/0.87
#25	0.7/0.49/0.65	1.1/0.87/0.98	0.88/0.71/1.15	1.1/0.88/1.21	0.91/0.7/0.87	0.94/0.71/0.89	1.08/0.86/1.17	1.26/1.05/1.35
#26	3.5/3.01/1.7	3.48/3/1.72	1.03/0.89/1.66	3.52/3.04/1.69	3.5/3.02/1.71	3.52/3.04/1.69	3.52/3.04/1.68	3.53/3.05/1.65

D.11 RealMLP Regressor

Table 42: Per-dataset performance of RealMLP Regression under different split regimes and datasets.

#	#1	#2	#3	#4	#5	#6	#7	#8
#1	0/0/0.64	0/0/0.64	0.41/0.2/0.42	0/0/0.82	0/0/0.65	0/0/0.65	0/0/0.83	0/0/0.49
#2	0/0/0.65	0/0/0.68	0.44/0.21/0.46	0/0/0.87	0/0/0.59	0/0/0.64	0/0/0.83	0/0/0.59
#3	0/0/0.61	0/0/0.61	0.58/0.39/0.68	0/0/0.78	0/0/0.6	0/0/0.62	0/0/0.74	0/0/0.69
#4	42.7/31.7/0.54	43.8/32.5/0.55	0.26/0.18/0.46	45.2/33.8/0.75	42.6/31.6/0.57	45.1/32.9/0.59	45.5/33.8/0.65	42.7/31.7/0.47

#	#1	#2	#3	#4	#5	#6	#7	#8
#5	0.1/0.07/0.65	0.15/0.11/0.93	0.78/0.56/0.79	0.16/0.12/1.05	0.12/0.09/0.8	0.13/0.09/0.85	0.16/0.12/1.04	0.16/0.12/1.07
#6	0/0/0.26	0/0/0.29	0.03/0.03/0.15	0.02/0.01/0.1	0/0/0.17	0/0/0.27	0/0/0.09	0.01/0/0.07
#7	7.14/5.33/0.2	55.5/44.9/0.99	0.17/0.13/0.19	107/83.7/1.26	41.8/29.2/0.69	81.4/69.6/1.25	113/86.6/1.17	113/87.6/1.17
#8	39.9/22.3/0.34	110/89/1.05	1.28/0.97/1.12	221/183/1.31	141/103/1.03	142/114/1.15	210/171/1.3	215/173/1.29
#9	0.31/0.26/1.6	0.32/0.27/1.68	0.99/0.84/1.7	0.33/0.29/1.78	0.32/0.26/1.64	0.32/0.28/1.71	0.33/0.3/1.76	0.34/0.29/1.62
#10	0.23/0.18/0.37	0.27/0.22/0.97	0.49/0.39/0.13	0.34/0.25/0.3	0.22/0.18/0.35	0.29/0.23/0.3	0.28/0.22/0.28	0.31/0.24/0.28
#11	1.09/0.68/0.8	1.2/0.75/0.83	0.67/0.42/0.79	1.77/1.14/0.86	0.89/0.57/0.85	1.58/1.07/0.98	1.63/0.91/0.79	1.79/1.09/0.82
#12	0.66/0.46/0.53	0.64/0.46/0.55	0.34/0.25/0.56	1.03/0.73/0.64	0.59/0.43/0.57	0.72/0.51/0.58	1/0.71/0.6	1/0.71/0.58
#13	0.25/0.16/0.24	0.21/0.13/0.43	0.18/0.11/0.25	0.92/0.55/0.18	0.22/0.14/0.27	0.26/0.17/0.24	0.56/0.36/0.17	0.3/0.2/0.15
#14	0.5/0.34/0.49	0.57/0.4/0.58	0.28/0.19/0.47	0.77/0.53/0.77	0.68/0.5/0.64	1.63/1.24/0.8	0.75/0.53/0.66	1.59/1.15/0.71
#15	12.6/7.28/1.19	16/8.85/1.31	0.88/0.41/1.28	60.2/31.3/1.48	20.1/9.64/1.34	67/39/1.46	31.3/17.8/1.42	30.3/15.4/1.41
#16	13.4/9.21/1.3	15.1/10.1/1.43	1.23/0.75/1.44	26.9/21.1/1.44	17.4/12.7/1.49	26.4/19.6/1.48	24/17.7/1.38	24.7/18.5/1.46
#17	2.58/1.72/1.42	2.18/1.54/1.45	0.59/0.46/1.42	1.67/1.33/1.4	3.02/1.91/1.47	2.7/1.75/1.4	2.83/2.16/1.48	5.52/3.29/1.49
#18	0/0/0.11	0/0/0.08	0.01/0.01/0.03	0/0/0.1	0/0/0.11	0/0/0.36	0/0/0.11	0/0/0.13
#19	5.93/3.96/0.73	5.95/4.03/0.8	0.51/0.35/0.74	6.65/4.26/0.77	5.37/3.7/0.71	5.77/3.81/0.73	6.2/4.11/0.75	5.74/3.92/0.73
#20	3.65/2.89/1.05	3.88/3.09/1.14	0.77/0.62/1.04	5.8/4.74/1.17	4.01/3.19/1.14	3.93/3.17/1.2	4.04/3.21/1.11	3.86/3.15/1.18
#21	0/0/0.11	0/0/0.21	0.19/0.13/0.19	0/0/0.3	0/0/0.23	0/0/0.21	0/0/0.28	0/0/0.28
#22	0.01/0.01/0.43	0.04/0.03/0.96	0.55/0.36/0.74	0.04/0.03/0.93	0.03/0.02/0.65	0.02/0.02/0.6	0.03/0.02/0.8	0.03/0.02/0.81
#23	0.06/0.03/0.1	0.25/0.16/0.23	0.09/0.05/0.13	0.48/0.35/0.15	0.18/0.11/0.17	0.12/0.06/0.09	0.31/0.2/0.14	0.33/0.22/0.09
#24	0.06/0.04/0.14	0.07/0.05/0.16	0.1/0.07/0.2	0.21/0.13/0.28	0.09/0.06/0.17	0.1/0.06/0.19	0.12/0.07/0.21	0.15/0.09/0.25
#25	0.73/0.49/0.62	1.1/0.82/0.91	0.92/0.71/1.01	1.23/0.95/1.12	0.91/0.66/0.77	0.93/0.67/0.86	1.21/0.93/1.11	1.33/1.03/1.15
#26	3.45/2.98/1.95	3.45/2.98/1.96	1.01/0.87/1.94	3.47/3.01/1.89	3.46/2.99/1.92	3.48/3.01/1.94	3.47/3.01/1.91	3.48/3.02/1.92

D.12 ResNet Regressor

Table 43: Per-dataset performance of ResNet Regression under different split regimes and datasets.

#	#1	#2	#3	#4	#5	#6	#7	#8
#1	0/0/0.68	0/0/0.62	0.42/0.23/0.49	0/0/0.91	0/0/0.67	0/0/0.7	0/0/0.86	0/0/0.56
#2	0/0/0.66	0/0/0.63	0.5/0.24/0.56	0/0/0.96	0/0/0.61	0/0/0.66	0/0/0.97	0/0/0.59
#3	0/0/0.61	0/0/0.62	0.51/0.33/0.6	0/0/0.71	0/0/0.6	0/0/0.61	0/0/0.7	0/0/0.66
#4	42.4/31.1/0.53	43.9/31.5/0.54	0.26/0.19/0.46	48.1/36.7/0.82	42.5/31.8/0.57	44.4/32/0.57	45.5/33.3/0.64	47.7/34.6/0.48
#5	0.1/0.07/0.66	0.13/0.1/0.85	0.76/0.54/0.74	0.14/0.1/0.99	0.12/0.08/0.77	0.12/0.09/0.8	0.15/0.11/0.94	0.16/0.11/0.89
#6	0/0/0.34	0/0/0.33	0.04/0.03/0.18	0.01/0.01/0.06	0/0/0.19	0/0/0.31	0.01/0/0.11	0.01/0.01/0.11
#7	8.98/7.39/0.26	46.3/31.8/0.87	0.19/0.16/0.3	189/158/1.33	22.4/16.4/0.61	51.3/41.6/0.85	144/125/1.33	134/112/1.28
#8	28.1/22/0.38	45.1/35.8/0.53	1.18/0.83/1.04	318/236/1.14	49.3/39/0.58	136/100/0.92	352/263/1.17	151/123/1.18
#9	0.31/0.27/1.66	0.32/0.27/1.7	1/0.85/1.72	0.34/0.31/1.72	0.32/0.26/1.65	0.32/0.28/1.74	0.34/0.31/1.73	0.36/0.3/1.4
#10	0.23/0.18/0.38	0.18/0.14/0.79	0.47/0.37/0.13	0.35/0.23/0.28	0.23/0.18/0.35	0.28/0.22/0.29	0.43/0.22/0.27	0.34/0.25/0.28
#11	1.11/0.71/0.81	1.13/0.73/0.81	0.57/0.37/0.81	1.6/0.94/0.82	0.91/0.59/0.85	1.47/0.87/0.83	1.49/0.91/0.81	1.35/0.85/0.83
#12	0.67/0.48/0.54	0.65/0.46/0.55	0.36/0.26/0.58	2.64/1.19/0.78	0.59/0.44/0.57	1.02/0.65/0.63	4.15/1.72/0.77	1.78/1.08/0.73
#13	0.26/0.18/0.26	0.19/0.12/0.38	0.19/0.12/0.27	1.87/0.92/0.19	0.23/0.16/0.29	0.27/0.18/0.26	0.84/0.47/0.19	0.33/0.25/0.19
#14	0.49/0.34/0.49	0.53/0.38/0.56	0.28/0.2/0.48	0.72/0.5/0.72	0.68/0.5/0.62	1.76/1.35/0.88	0.65/0.46/0.61	1.47/1.06/0.68
#15	13.4/7.61/1.2	13.7/8.98/1.3	0.8/0.39/1.19	45.4/26.9/1.48	15.8/8.24/1.32	48.8/26.5/1.41	31.9/17.6/1.39	29.1/15.7/1.41
#16	14.7/9.88/1.36	16.1/10.6/1.43	1.28/0.8/1.45	33.3/26/1.43	17.4/12.4/1.47	31.2/21.3/1.47	22.7/17.5/1.45	26.3/19.7/1.49
#17	2.61/1.73/1.51	2.19/1.54/1.46	0.67/0.52/1.46	39.2/8.57/1.47	3.04/1.91/1.52	2.81/1.82/1.45	12.3/3.79/1.44	7.38/4.07/1.56
#18	0/0/0.22	0/0/0.21	0.07/0.06/0.22	0/0/0.31	0/0/0.24	0/0/0.56	0/0/0.27	0/0/0.45
#19	9.05/6.53/1.06	8.34/6.19/1.1	0.8/0.59/1.09	62.9/21/0.95	7.35/5.34/0.94	8.07/5.75/0.96	9.55/6.94/0.96	8.05/5.8/0.94
#20	3.63/2.88/1.05	3.84/3.07/1.13	0.76/0.6/1.03	5.75/4.68/1.13	4/3.23/1.12	3.89/3.11/1.18	3.94/3.16/1.11	3.8/3.03/1.11
#21	0/0/0.18	0/0/0.23	0.17/0.12/0.2	0/0/0.27	0/0/0.24	0/0/0.22	0/0/0.24	0/0/0.23
#22	0.03/0.02/0.74	0.05/0.04/1.13	0.96/0.74/1.05	0.04/0.03/1.07	0.04/0.03/0.92	0.04/0.03/0.98	0.04/0.03/1.05	0.05/0.03/1.06
#23	0.07/0.04/0.13	0.16/0.11/0.22	0.1/0.06/0.13	7.41/0.75/0.19	0.11/0.07/0.16	0.22/0.11/0.17	3.13/0.38/0.2	6.01/0.86/0.2
#24	0.11/0.08/0.23	0.1/0.07/0.21	0.17/0.13/0.29	0.28/0.18/0.35	0.12/0.09/0.25	0.14/0.1/0.28	0.16/0.11/0.27	0.2/0.14/0.33
#25	0.73/0.5/0.63	1.03/0.78/0.83	0.86/0.66/0.94	1.04/0.81/0.94	0.86/0.63/0.76	0.9/0.68/0.88	1.07/0.84/1	1.13/0.9/0.98
#26	3.45/2.99/1.9	3.45/2.99/1.93	1.01/0.87/1.88	3.47/3.01/1.89	3.46/2.99/1.89	3.48/3.02/1.89	3.47/3.01/1.84	3.48/3.02/1.84

D.13 FT–Transformer Regressor

Table 44: Per-dataset performance of FT–Transformer Regression under different split regimes and datasets.

#	#1	#2	#3	#4	#5	#6	#7	#8
#1	0/0/0.71	0/0/0.62	0.42/0.23/0.47	0/0/0.86	0/0/0.65	0/0/0.7	0/0/0.87	0/0/0.59
#2	0/0/0.68	0/0/0.64	0.45/0.23/0.51	0/0/1.03	0/0/0.62	0/0/0.66	0/0/1.06	0/0/0.61
#3	0/0/0.66	0/0/0.67	0.58/0.4/0.71	0/0/0.73	0/0/0.65	0/0/0.66	0/0/0.73	0/0/0.7
#4	45.4/32.6/0.54	45.7/33/0.55	0.27/0.19/0.49	45.8/32.8/0.7	43.8/31.6/0.56	47/33.9/0.59	46.3/33.6/0.64	42/30.5/0.44
#5	0.11/0.08/0.79	0.14/0.11/0.93	0.83/0.61/0.8	0.14/0.11/0.97	0.12/0.09/0.89	0.13/0.09/0.93	0.15/0.11/1.07	0.15/0.11/0.9
#6	0/0/0.35	0/0/0.39	0.05/0.04/0.23	0.02/0.02/0.13	0/0/0.25	0/0/0.35	0.01/0/0.18	0.01/0.01/0.12
#7	43.2/34.1/0.86	97.5/83/1.44	0.86/0.69/1.08	114/97.4/1.45	85.2/72.5/1.46	108/89.5/1.5	115/98.2/1.44	112/95.5/1.43
#8	123/104/1.47	179/148/1.45	1.05/0.88/1.45	169/138/1.41	141/119/1.54	133/115/1.57	179/146/1.38	177/146/1.35
#9	0.31/0.26/1.6	0.32/0.27/1.66	0.99/0.85/1.7	0.33/0.3/1.76	0.32/0.26/1.62	0.32/0.27/1.71	0.33/0.3/1.77	0.34/0.27/1.44
#10	0.24/0.19/0.38	0.21/0.17/0.84	0.48/0.39/0.13	0.3/0.23/0.3	0.23/0.18/0.35	0.28/0.22/0.3	0.29/0.23/0.28	0.32/0.25/0.29
#11	1.12/0.71/0.8	1.16/0.71/0.81	0.6/0.4/0.8	1.68/1.17/0.89	0.92/0.6/0.83	1.44/0.99/1.03	1.44/0.9/0.79	1.45/0.92/0.84
#12	0.68/0.49/0.56	0.67/0.5/0.57	0.35/0.26/0.58	1.08/0.77/0.65	0.63/0.47/0.59	0.75/0.54/0.58	1.06/0.77/0.61	1.16/0.83/0.64
#13	0.25/0.16/0.23	0.19/0.12/0.41	0.19/0.11/0.25	1.19/0.69/0.2	0.22/0.14/0.26	0.27/0.18/0.24	0.6/0.4/0.19	0.33/0.24/0.17
#14	0.56/0.4/0.55	0.61/0.46/0.61	0.32/0.23/0.51	0.73/0.52/0.71	0.79/0.59/0.65	1.57/1.23/0.85	0.79/0.58/0.64	1.55/1.19/0.69
#15	16.1/8.49/1.31	14.7/8.53/1.37	0.89/0.41/1.42	55/40.7/1.59	15/8.06/1.41	32.6/19.7/1.51	30.8/14.3/1.42	29.5/18.8/1.42
#16	16.2/10.8/1.44	16.7/11.2/1.53	1.21/0.69/1.5	19.9/15.8/1.53	17.1/12.8/1.5	23.7/15.7/1.51	20.4/14.8/1.49	22.6/15.7/1.49

#	#1	#2	#3	#4	#5	#6	#7	#8
#17	2.63/1.75/1.5	2.19/1.55/1.46	0.61/0.47/1.48	1.72/1.37/1.38	3.05/1.92/1.54	2.75/1.78/1.45	1.65/1.31/1.34	5.46/3.26/1.51
#18	0/0/0.18	0/0/0.24	0.03/0.02/0.1	0/0/0.27	0/0/0.17	0/0/0.56	0/0/0.29	0/0/0.41
#19	8.23/5.91/1	8.18/6.01/1.06	0.71/0.52/1.02	7.13/4.75/0.82	7.93/5.81/1.01	8.12/5.87/1.02	7.96/5.74/1.01	7.93/5.76/1.01
#20	3.67/2.92/1.06	3.85/3.09/1.14	0.75/0.6/1.04	5.6/4.55/1.12	4/3.2/1.09	3.89/3.08/1.14	3.91/3.15/1.08	3.59/2.91/1.05
#21	0/0/0.16	0/0/0.22	0.19/0.14/0.23	0/0/0.26	0/0/0.23	0/0/0.21	0/0/0.25	0/0/0.25
#22	0.02/0.02/0.63	0.05/0.04/1.04	0.57/0.39/0.83	0.04/0.03/0.97	0.03/0.02/0.75	0.03/0.02/0.79	0.03/0.02/0.96	0.03/0.02/1.01
#23	0.07/0.03/0.12	0.19/0.13/0.23	0.11/0.05/0.14	0.62/0.43/0.17	0.12/0.08/0.12	0.15/0.08/0.12	0.5/0.34/0.15	0.5/0.32/0.15
#24	0.15/0.11/0.29	0.15/0.12/0.3	0.21/0.15/0.35	0.35/0.24/0.48	0.18/0.13/0.31	0.19/0.14/0.36	0.24/0.17/0.36	0.25/0.17/0.39
#25	0.86/0.64/0.79	1.1/0.86/0.95	0.83/0.66/0.97	1.09/0.84/1	0.97/0.74/0.89	1.05/0.84/1.06	1.08/0.82/0.98	1.1/0.84/0.99
#26	3.45/2.99/1.91	3.45/2.98/1.92	1.01/0.87/1.87	3.47/3.01/1.9	3.46/2.99/1.9	3.48/3.02/1.86	3.47/3.01/1.9	3.48/3.02/1.89